



**UNCUYO**  
UNIVERSIDAD  
NACIONAL DE CUYO



**FACULTAD  
DE INGENIERÍA**

## **Compilador TinyS**

Autores: Paulina Suden - Tomás Rando

Profesora: Carolina Olivera

Asignatura: Compiladores

Fecha de presentación: 15/01/2026

|                                                                |            |
|----------------------------------------------------------------|------------|
| <b>1. Introducción</b>                                         | <b>3</b>   |
| <b>2. Desarrollo</b>                                           | <b>3</b>   |
| 2.1. Compilación de las etapas                                 | 3          |
| 2.2. Ejecución de las etapas                                   | 4          |
| 2.3. Análisis Léxico                                           | 5          |
| 2.3.1. Alfabeto de entrada                                     | 5          |
| 2.3.2. Expresiones regulares                                   | 5          |
| 2.3.3. Tokens                                                  | 6          |
| 2.3.4. Excepciones                                             | 7          |
| 2.3.5. Casos de testeo                                         | 8          |
| 2.4. Análisis Sintáctico                                       | 11         |
| 2.4.1. Gramática EBNF                                          | 11         |
| 2.4.2. Gramática BNF                                           | 14         |
| 2.4.3. Gramática sin recursividad a izquierda                  | 18         |
| 2.4.4. Gramática factorizada                                   | 22         |
| 2.4.5. Eliminación de ambigüedad y transformación en LL(1)     | 26         |
| 2.4.6. Primeros y Siguientes                                   | 32         |
| 2.4.7. Gramática LL(1)                                         | 47         |
| 2.4.8. Excepciones                                             | 47         |
| 2.4.9. Casos de testeo                                         | 48         |
| 2.5. Análisis Semántico - Chequeo de declaraciones             | 56         |
| 2.5.1. Estructura y diagrama de clases de la tabla de símbolos | 56         |
| 2.5.2. Esquema de traducción                                   | 57         |
| 2.5.3. Ejemplo de JSON                                         | 65         |
| 2.5.4. Errores semánticos detectados                           | 66         |
| 2.5.5. Casos de testeo                                         | 68         |
| 2.6. Análisis Semántico - Chequeo de sentencias                | 77         |
| 2.6.1. Estructura y diagrama de clases del AST                 | 77         |
| 2.6.2. Esquema de traducción                                   | 77         |
| 2.6.3. Ejemplo de JSON                                         | 90         |
| 2.6.4. Errores semánticos detectados                           | 92         |
| 2.6.5. Casos de testeo                                         | 96         |
| 2.7. Generación de código                                      | 111        |
| 2.7.1. Estructura del registro de activación                   | 111        |
| 2.7.2. Gestión y resolución de métodos                         | 112        |
| 2.7.3. Estructuras de las clases                               | 113        |
| 2.7.4. Decisiones de diseño                                    | 115        |
| 2.7.5. Proceso de generación de código MIPS                    | 115        |
| 2.7.6. Errores detectados                                      | 116        |
| 2.7.7. Casos de testeo                                         | 116        |
| <b>3. Gestión de los tiempos</b>                               | <b>121</b> |
| <b>4. Conclusión</b>                                           | <b>121</b> |
| <b>5. Referencias y/o bibliografía</b>                         | <b>121</b> |
| <b>6. Anexos</b>                                               | <b>122</b> |

# 1. Introducción

En el presente informe, se describe el proceso de construcción del compilador para el lenguaje TinyS, explicando de manera detallada las distintas etapas: Análisis Léxico, Análisis Sintáctico, Análisis Semántico (junto a la Tabla de Símbolos y el AST) y, por último, la Generación de Código en MIPS. También se explica cómo ejecutar las distintas etapas a través de la terminal del usuario.

El compilador fue desarrollado en el lenguaje de programación Java utilizando el jdk 21. No se utilizaron librerías externas, a excepción de JUnit 5, la cual fue utilizada para automatizar los tests.

## 2. Desarrollo

### 2.1. Compilación de las etapas

Las etapas se entregan por separado en archivos zip distintos. Dentro de cada uno de ellos se encuentra la carpeta /Compiladores, la cual contiene la carpeta /Compilador, con el código fuente de la etapa; la carpeta /tc con los tests de la etapa, los mismos divididos en failing y passing; la documentación en forma de Javadoc y el ejecutable .jar de la etapa.

Para compilar las diferentes etapas del proyecto, se utilizó Maven 3+. Para ello, se debe ingresar por terminal a la carpeta /Compilador, la cual contiene el código fuente de la misma, y se debe colocar el siguiente comando:

```
mvn clean package -DskipTests
```

Ante esto, se generará en la carpeta /Compilador la carpeta /target, la cual contendrá en su interior el ejecutable .jar con el nombre de la etapa. A continuación se presenta un ejemplo con la etapa 1 de los pasos mencionados (Figura 1 y 2).

```

PS C:\Users\tomas\Desktop\Compiladores\Compilador> mvn clean package -DskipTests
[INFO] Scanning for projects...
[INFO] -----< com.uncuyo.compiladores:Compilador >-----
[INFO] Building Compilador 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- clean:3.2.0:clean (default-clean) @ Compilador ---
[INFO] Deleting C:\Users\tomas\Desktop\Compiladores\Compilador\target
[INFO] --- resources:3.3.1:resources (default-resources) @ Compilador ---
[INFO] skip non existing resourceDirectory C:\Users\tomas\Desktop\Compiladores\Compilador\src\main\resources
[INFO] --- compiler:3.11.0:compile (default-compile) @ Compilador ---
[INFO] Changes detected - recompiling the module! :source
[INFO] Compiling 9 source files with javac [debug target 21] to target\classes
[INFO] --- resources:3.3.1:testResources (default-testResources) @ Compilador ---
[INFO] skip non existing resourceDirectory C:\Users\tomas\Desktop\Compiladores\Compilador\src\test\resources
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ Compilador ---
[INFO] Changes detected - recompiling the module! :dependency
[INFO] Compiling 1 source file with javac [debug target 21] to target\test-classes
[INFO] --- surefire:3.2.5:test (default-test) @ Compilador ---
[INFO] Tests are skipped.
[INFO] --- jar:3.3.0:jar (default-jar) @ Compilador ---
[INFO] Building jar: C:\Users\tomas\Desktop\Compiladores\Compilador\target\etapa1.jar
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.721 s
[INFO] Finished at: 2025-12-20T12:37:51-03:00
[INFO] -----

```

Figura 1. Resultado del comando mvn clean package -DskipTests

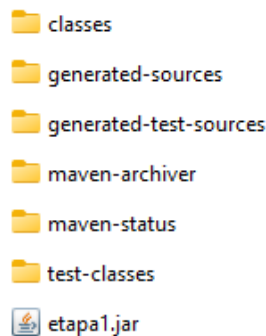


Figura 2. Carpeta /target con el .jar generado

## 2.2. Ejecución de las etapas

Utilizando el archivo .jar de la etapa, se procede a la ejecución del mismo a través de la terminal utilizando el siguiente comando:

```
java -jar etapaX.jar <ARCHIVO_FUENTE>
```

Por ejemplo, para la etapa 1, se podría ejecutar el test “input.s” de la siguiente manera:

```
java -jar etapa1.jar input.s
```

Vale aclarar que en esta etapa en particular, también es posible indicar el archivo de salida:

```
java -jar etapa1.jar input.s output.txt
```

## 2.3. Análisis Léxico

### 2.3.1. Alfabeto de entrada

$$\Sigma = \{ [A-Z], [a-z], [0-9], \acute{a}, \acute{e}, \acute{i}, \acute{o}, \acute{u}, \acute{A}, \acute{E}, \acute{I}, \acute{O}, \acute{U}, \ddot{U}, \ddot{u}, (, ), \{, \}, [, ], ., ', :, ;, \&, |, =, !, +, -, *, /, \%, <, >, _ \}$$

### 2.3.2. Expresiones regulares

Else: e.l.s.e

If: i.f

Start: s.t.a.r.t

Class: c.l.a.s.s

True: t.r.u.e

False: f.a.l.s.e

Nil: n.i.l

Void: v.o.i.d

Int: I.n.t

Bool: B.o.o.l

Double: D.o.u.b.l.e

Str: S.t.r

While: w.h.i.l.e

Div: d.i.v

Pub: p.u.b

St: s.t

Fn: f.n

Array: A.r.r.a.y

Ret: r.e.t

Self: s.e.l.f

IO: I.O

Impl: i.m.p.l

New: n.e.w

Object: O.b.j.e.c.t

Comentario simple: `/.Σ`

Comentario multilínea: `/.Σ.*./`

Identificadores de objetos: `[a-z] . ( [a-z] | [A-Z] | _ | [0-9] )*`

Identificadores de clase: `[A-Z] . ( [a-z] | [A-Z] | _ | [0-9] )* . ([a-z] | [A-Z])`

Enteros: `[0-9]`

Dobles: `[0-9]+.''.[0-9]+`

Cadenas: `"Σ."`

Operadores unarios: `( + | + . + | - . - | - | ! | '(.l.n.t.)'`

Operador asignación: `=`

Operadores relacional: `(< | > ) | (< | > ) . =`

Operadores lógicos: `((= | ! ) . = | & . & | ' | ' )`

Operador binario: `( * | / | % | + | - )`

Paréntesis abierto: `(`

Paréntesis cerrado: `)`

Llave abierta: `{`

Llave cerrada: `}`

Corchete abierto: `[`

Corchete cerrado: `]`

Punto: `.`

Coma: `,`

Punto y coma: `;`

Dos puntos: `:`

### 2.3.3. Tokens

En esta sección se describen los tokens utilizados durante el desarrollo del análisis léxico del lenguaje TinyS.

#### Palabras claves:

pstart, pclass, pimpl, pelse, pfalse, pif, pret, pwhile, ptrue, pnil, pn timer, pfn, pst, ppub, pself, pdiv, pbool, pstr, pint, pdouble, pobject, pio, parray, pvoid,

#### Operadores aritméticos (+, /, \*, -, %, ++, -):

op\_sum, op\_div, op\_mult, op\_sub, op\_mod, op\_increment, op\_decrement,

#### Operadores lógicos (&&, || y !)

op\_and, op\_or, op\_not,

**Operadores relacionales (<, >, ==, >=, <=, !=)**

op\_rel\_less, op\_rel\_greater, op\_rel\_equal, op\_rel\_greaterequal,  
op\_rel\_lessequal, op\_rel\_notequal,

**Operador de asignación (=):**

op\_equal,

**Símbolos especiales ([, ], (, ), {, }, ., ', ;, :):**

brackets1, brackets2, parentheses1, parentheses2, braces1, braces2, dot,  
comma, semicolon, colon,

**Constantes:**

const\_int, const\_double, const\_string,

**Identificadores de clase y de objetos:**

id\_class, id\_obj,

**EOF (End of file):**

end\_of\_file;

## 2.3.4. Excepciones

Con respecto a las excepciones, se decidió manejar cada etapa con una única excepción por cada una, modificando el error que reportan. En cuanto al análisis léxico, la excepción que generamos es “LexicalException”, la misma reporta los siguientes errores léxicos:

- Carácter ilegal
- Identificador de clase incorrecto
- Número fuera de límites
- End Of File inesperado
- Carácter nulo inválido en cadena
- Símbolo inválido
- Tamaño máximo de cadena superado (1024 caracteres)

- Apertura incorrecta en comentario
- Operador booleano no válido
- Operador de asignación no válido
- Símbolo especial no reconocido

Las excepciones en todos los casos se reportan con el siguiente formato:

ERROR: LÉXICO

| NÚMERO DE LÍNEA (NÚMERO DE COLUMNA) | DESCRIPCIÓN: |

| LÍNEA X (COLUMNA Y ) | mensaje |

### 2.3.5. Casos de testeo

Tabla 1. Casos de testeo del analizador léxico.

| Escenario                     | Nombre Test | Salida Esperada                                                                                                                                                     |
|-------------------------------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Prueba dos puntos (:)         | colon.s     | CORRECTO: ANALISIS LEXICO                                                                                                                                           |
| Operadores y cadenas          | input.s     | CORRECTO: ANALISIS LEXICO                                                                                                                                           |
| Int y doubles                 | input1.s    | CORRECTO: ANALISIS LEXICO                                                                                                                                           |
| Cadena sin cerrar             | input2.s    | ERROR: LÉXICO<br>  NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LINEA 1 (COLUMNA 29)  <br>END OF FILE<br>INESPERADO. SE ESPERABA CIERRE DE COMILLAS |
| Comentarios e identificadores | input3.s    | ERROR: LÉXICO<br>  NÚMERO DE LÍNEA                                                                                                                                  |



|                                              |          |                                                                                                                                                                    |
|----------------------------------------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                              |          | (NÚMERO DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LINEA 1 (COLUMNA 29)  <br>END OF FILE<br>INESPERADO. SE<br>ESPERABA CIERRE DE<br>COMILLAS                             |
| Identificador inválido                       | input4.s | ERROR: LÉXICO<br>  NÚMERO DE LÍNEA<br>(NÚMERO DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 1 (COLUMNA 44)  <br>IDENTIFICADOR DE CLASE<br>INCORRECTO<br>SoyUnaClase48 |
| Comentario sin<br>cerrar                     | input5.s | ERROR: LÉXICO<br>  NÚMERO DE LÍNEA<br>(NÚMERO DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 1 (COLUMNA 45)  <br>END OF FILE INESPERADO<br>EN COMENTARIO<br>MULTILÍNEA |
| Límite de la longitud<br>del entero superada | input6.s | ERROR: LÉXICO<br>  NÚMERO DE LÍNEA<br>(NÚMERO DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 1 (COLUMNA 30)  <br>NÚMERO FUERA DE<br>LÍMITES                            |
| Entero negativo                              | input7.s | CORRECTO: ANALISIS<br>LEXICO                                                                                                                                       |

|                                                                         |           |                                                                                                                                                                             |
|-------------------------------------------------------------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| String con más de 1024 caracteres                                       | input8.s  | ERROR: LÉXICO<br>  NÚMERO DE LÍNEA<br>(NÚMERO DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LINEA 1 (COLUMNA 1025)<br>  SE SUPERÓ EL TAMAÑO<br>MÁXIMO DE CADENA (1024<br>CARACTERES) |
| Comentario simple                                                       | input9.s  | CORRECTO: ANALISIS<br>LEXICO                                                                                                                                                |
| Fibonacci. Pruebas de la mayoría de los casos.                          | input10.s | CORRECTO: ANALISIS<br>LEXICO                                                                                                                                                |
| Int y un identificador de clase concatenado. Se reconocen por separado. | input11.s | CORRECTO: ANALISIS<br>LEXICO                                                                                                                                                |
| Carácter ilegal en comentario simple                                    | input12.s | ERROR: LÉXICO<br>  NÚMERO DE LÍNEA<br>(NÚMERO DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 1 (COLUMNA 8)  <br>SÍMBOLO INVALIDO: $\pi$                                         |
| Carácter ilegal fuera de comentario                                     | input13.s | ERROR: LÉXICO<br>  NÚMERO DE LÍNEA<br>(NÚMERO DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 1 (COLUMNA 7)  <br>SÍMBOLO INVALIDO: $\pi$                                         |

|                                          |           |                                                                                                                                                           |
|------------------------------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Carácter ilegal en comentario multilínea | input14.s | ERROR: LÉXICO<br>  NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 2 (COLUMNA 19)  <br>SÍMBOLO INVALIDO: $\pi$                         |
| Carácter nulo en literal cadena          | input15.s | ERROR: LÉXICO<br>  NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LINEA 1 (COLUMNA 9)  <br>CARACTER NULO INVALIDO EN CADENA: $\backslash 0$ |
| Cadena con carácter de salto de línea    | input16.s | CORRECTO: ANALISIS LEXICO                                                                                                                                 |

## 2.4. Análisis Sintáctico

### 2.4.1. Gramática EBNF

$\langle \text{Program} \rangle ::= \langle \text{Lista-Definiciones} \rangle \langle \text{Start} \rangle$   
 $\langle \text{Start} \rangle ::= \text{start} \langle \text{Bloque-Método} \rangle$   
 $\langle \text{Lista-Definiciones} \rangle ::= (\langle \text{Class} \rangle | \langle \text{Impl} \rangle)^*$   
 $\langle \text{Class} \rangle ::= \text{class idclass} \langle \text{Herencia} \rangle? \{ \langle \text{Atributo} \rangle^* \}$   
 $\langle \text{Impl} \rangle ::= \text{impl idclass} \{ \langle \text{Miembro} \rangle^+ \}$   
 $\langle \text{Herencia} \rangle ::= : \langle \text{Tipo} \rangle$   
 $\langle \text{Miembro} \rangle ::= \langle \text{Metodo} \rangle | \langle \text{Constructor} \rangle$   
 $\langle \text{Constructor} \rangle ::= . \langle \text{Argumentos-Formales} \rangle \langle \text{Bloque-Método} \rangle$   
 $\langle \text{Atributo} \rangle ::= \langle \text{Visibilidad} \rangle? \langle \text{Tipo} \rangle \langle \text{Lista-Declaracion-Variables} \rangle ;$   
 $\langle \text{Método} \rangle ::= \langle \text{Forma-Metodo} \rangle? \text{fn} \langle \text{Tipo-Metodo} \rangle? \text{idMetAt}$   
 $\langle \text{Argumentos-Formales} \rangle \langle \text{Bloque-Método} \rangle$   
 $\langle \text{Visibilidad} \rangle ::= \text{pub}$   
 $\langle \text{Forma-Metodo} \rangle ::= \text{st}$

<Bloque-Método> ::= { <Decl-Var-Locales>\* <Sentencia>\* }  
 <Decl-Var-Locales> ::= <Tipo> <Lista-Declaracion-Variables> ;  
 <Lista-Declaracion-Variables> ::= idMetAt  
 <Lista-Declaracion-Variables> ::= idMetAt , <Lista-Declaracion-Variables>  
 <Argumentos-Formales> ::= ( <Lista-Argumentos-Formales>? )  
 <Lista-Argumentos-Formales> ::= <Argumento-Formal> ,  
 <Lista-Argumentos-Formales>  
 <Lista-Argumentos-Formales> ::= <Argumento-Formal>  
 <Argumento-Formal> ::= <Tipo> idMetAt  
 <Tipo-Método> ::= <Tipo>  
 <Tipo-Método> ::= void  
 <Tipo> ::= <Tipo-Primitivo>  
 <Tipo> ::= <Tipo-Arreglo>  
 <Tipo> ::= <Tipo-Referencia>  
 <Tipo-Primitivo> ::= Str  
 <Tipo-Primitivo> ::= Bool  
 <Tipo-Primitivo> ::= Int  
 <Tipo-Primitivo> ::= Double  
 <Tipo-Referencia> ::= idclass  
 <Tipo-Arreglo> ::= Array <Tipo-Primitivo>  
 <ExpresionParentizada> ::= ( <Expresión> ) <Encadenado>?  
 <AccesoSelf> ::= self <Encadenado>?  
 <AccesoVar> ::= id <Encadenado>?  
 <AccesoVar> ::= id [ <Expresión> ] <Encadenado>?  
 <Llamada-Método> ::= id <Argumentos-Actuales> <Encadenado>?  
 <Llamada-Método-Estático> ::= idclass . <Llamada-Método> <Encadenado>?  
 <Llamada-Conclassor> ::= new idclass <Argumentos-Actuales> <Encadenado>?  
 <Llamada-Conclassor> ::= new <Tipo-Primitivo> [ <Expresión> ]  
 <Argumentos-Actuales> ::= ( <Lista-Expresiones>? )  
 <Lista-Expresiones> ::= <Expresión>  
 <Lista-Expresiones> ::= <Expresión> , <Lista-Expresiones>  
 <Encadenado> ::= . <Llamada-Método-Encadenado>  
 <Encadenado> ::= . <Acceso-Variable-Encadenado>  
 <Llamada-Método-Encadenado> ::= id <Argumentos-Actuales> <Encadenado>?  
 <Acceso-Variable-Encadenado> ::= id <Encadenado>?  
 <Acceso-Variable-Encadenado> ::= id [ <Expresión> ] <Encadenado>?  
 <Sentencia> ::= ;

```

| <Asignación>;
| <Sentencia-Simple>;
| if (<Expresión>) <Sentencia>
| if (<Expresión>) <Sentencia> else <Sentencia>
| while ( <Expresión> ) <Sentencia>
| <Bloque>
| ret <Expresión>?
<Bloque> ::= { <Sentencia>* }
<Asignación> ::= <AccesoVar-Simple> = <Expresión>
| <AccesoSelf-Simple> = <Expresión>
<AccesoVar-Simple> ::= id <Encadenado-Simple>*
| id [ <Expresión> ]
<AccesoSelf-Simple> ::= self <Encadenado-Simple>*
<Encadenado-Simple> ::= . id
<Sentencia-Simple> ::= (<Expresión>)
<ExpOr > ::= <ExpOr > || <ExpAnd> | <ExpAnd>
<ExpAnd> ::= <ExpAnd> && <ExpIguar> | <ExpIguar>
<ExpIguar> ::= <ExpIguar> <OpIguar> <ExpCompuesta> | <ExpCompuesta>
<ExpCompuesta> ::= <ExpAd> <OpCompuesto> <ExpAd> | <ExpAd>
<ExpAd> ::= <ExpAd> <OpAd> <ExpMul> | <ExpMul>
<ExpMul> ::= <ExpMul> <OpMul> <ExpUn> | <ExpUn>
<ExpUn> ::= <OpUnario> <ExpUn> | <Operando>
<OpIguar> ::= == | !=
<OpCompuesto> ::= < > | <= > | >=
<OpAd> ::= + | -
<OpUnario> ::= + | - | ! | ++ | -- | (Int)
<OpMul> ::= * | / | % | div
<Operando> ::= <Literal> | <Primario> <Encadenado>?
<Literal> ::= nil | true | false | intLiteral | StrLiteral | doubleLiteral
<Primario> ::= <ExpresionParentizada>
| <AccesoSelf >
| <AccesoVar >
| <Llamada-Método>
| <Llamada-Método-Estático>
| <Llamada-Conclassor >

```

## 2.4.2. Gramática BNF

$\langle \text{Program} \rangle ::= \langle \text{Lista-Definiciones} \rangle \langle \text{Start} \rangle$   
 $\langle \text{Start} \rangle ::= \text{pstart} \langle \text{Bloque-Método} \rangle$   
 $\langle \text{Lista-Definiciones} \rangle ::= \langle \text{Class} \rangle \langle \text{Lista-Definiciones} \rangle$   
 $\langle \text{Lista-Definiciones} \rangle ::= \langle \text{Impl} \rangle \langle \text{Lista-Definiciones} \rangle$   
 $\langle \text{Lista-Definiciones} \rangle ::= \lambda$   
 $\langle \text{Class} \rangle ::= \text{pclass id\_class} \langle \text{Herencia} \rangle \{ \langle \text{Atributos} \rangle \}$   
 $\langle \text{Class} \rangle ::= \text{pclass id\_class} \{ \langle \text{Atributos} \rangle \}$   
 $\langle \text{Atributos} \rangle ::= \langle \text{Atributo} \rangle \langle \text{Atributos} \rangle$   
 $\langle \text{Atributos} \rangle ::= \lambda$   
 $\langle \text{Impl} \rangle ::= \text{pimpl id\_class} \{ \langle \text{Miembro} \rangle \langle \text{Miembros} \rangle \}$   
 $\langle \text{Miembros} \rangle ::= \langle \text{Miembro} \rangle \langle \text{Miembros} \rangle$   
 $\langle \text{Miembros} \rangle ::= \lambda$   
 $\langle \text{Herencia} \rangle ::= \text{colon} \langle \text{Tipo} \rangle$   
 $\langle \text{Miembro} \rangle ::= \langle \text{Metodo} \rangle$   
 $\langle \text{Miembro} \rangle ::= \langle \text{Constructor} \rangle$   
 $\langle \text{Constructor} \rangle ::= \text{dot} \langle \text{Argumentos-Formales} \rangle \langle \text{Bloque-Método} \rangle$   
 $\langle \text{Atributo} \rangle ::= \langle \text{Visibilidad} \rangle \langle \text{Tipo} \rangle \langle \text{Lista-Declaracion-Variables} \rangle \text{semicolon}$   
 $\langle \text{Atributo} \rangle ::= \langle \text{Tipo} \rangle \langle \text{Lista-Declaracion-Variables} \rangle \text{semicolon}$   
 $\langle \text{Método} \rangle ::= \langle \text{Forma-Método} \rangle \text{pfn} \langle \text{Tipo-Método} \rangle \text{id\_obj}$   
 $\langle \text{Argumentos-Formales} \rangle \langle \text{Bloque-Método} \rangle$   
 $\langle \text{Método} \rangle ::= \langle \text{Forma-Método} \rangle \text{pfn id\_obj} \langle \text{Argumentos-Formales} \rangle$   
 $\langle \text{Bloque-Método} \rangle$   
 $\langle \text{Método} \rangle ::= \text{pfn} \langle \text{Tipo-Método} \rangle \text{id\_obj} \langle \text{Argumentos-Formales} \rangle$   
 $\langle \text{Bloque-Método} \rangle$   
 $\langle \text{Método} \rangle ::= \text{pfn id\_obj} \langle \text{Argumentos-Formales} \rangle \langle \text{Bloque-Método} \rangle$   
 $\langle \text{Visibilidad} \rangle ::= \text{ppub}$   
 $\langle \text{Forma-Método} \rangle ::= \text{pst}$   
 $\langle \text{Bloque-Metodo} \rangle ::= \text{braces1} \langle \text{Más-Decl-Var-Locales} \rangle \langle \text{Sentencias} \rangle \text{braces2}$   
 $\langle \text{Sentencias} \rangle ::= \langle \text{Sentencia} \rangle \langle \text{Sentencias} \rangle$   
 $\langle \text{Sentencias} \rangle ::= \lambda$   
 $\langle \text{Mas-Decl-Var-Locales} \rangle ::= \langle \text{Decl-Var-Locales} \rangle \langle \text{Mas-Decl-Var-Locales} \rangle$   
 $\langle \text{Más-Decl-Var-Locales} \rangle ::= \lambda$   
 $\langle \text{Decl-Var-Locales} \rangle ::= \langle \text{Tipo} \rangle \langle \text{Lista-Declaracion-Variables} \rangle \text{semicolon}$   
 $\langle \text{Lista-Declaracion-Variables} \rangle ::= \text{id\_obj}$   
 $\langle \text{Lista-Declaracion-Variables} \rangle ::= \text{id\_obj comma} \langle \text{Lista-Declaracion-Variables} \rangle$

<Argumentos-Formales> ::= parentheses1 <Lista-Argumentos-Formales> )  
 parentheses2  
 <Argumentos-Formales> ::= parentheses1 parentheses2  
 <Lista-Argumentos-Formales> ::= <Argumento-Formal> comma  
 <Lista-Argumentos-Formales>  
 <Lista-Argumentos-Formales> ::= <Argumento-Formal>  
 <Argumento-Formal> ::= <Tipo> id\_obj  
 <Tipo-Método> ::= <Tipo>  
 <Tipo-Método> ::= pvoid  
 <Tipo> ::= <Tipo-Primitivo>  
 <Tipo> ::= parray <Tipo-Primitivo>  
 <Tipo> ::= id\_class  
 <Tipo-Primitivo> ::= pstr  
 <Tipo-Primitivo> ::= pbool  
 <Tipo-Primitivo> ::= pint  
 <Tipo-Primitivo> ::= pdouble  
 <ExpresionParentizada> ::= parentheses1 <ExpOr> parentheses2  
 <Encadenado>  
 <ExpresionParentizada> ::= parentheses1 <ExpOr> parentheses2  
 <AccesoSelf> ::= pself <Encadenado>  
 <AccesoSelf> ::= pself  
 <AccesoVar> ::= id <Encadenado>  
 <AccesoVar> ::= id  
 <AccesoVar> ::= id brackets1 <ExpOr> brackets2 <Encadenado>  
 <AccesoVar> ::= id brackets1 <ExpOr> brackets2  
 <Llamada-Método> ::= id <Argumentos-Actuales> <Encadenado>  
 <Llamada-Método> ::= id <Argumentos-Actuales>  
 <Llamada-Método-Estático> ::= id\_class dot <Llamada-Método> <Encadenado>  
 <Llamada-Método-Estático> ::= id\_class dot <Llamada-Método>  
 <Llamada-Conclassor> ::= pnew id\_class <Argumentos-Actuales>  
 <Encadenado>  
 <Llamada-Conclassor> ::= pnew id\_class <Argumentos-Actuales>  
 <Llamada-Conclassor> ::= pnew <Tipo-Primitivo> brackets1 <ExpOr> brackets2  
 <Argumentos-Actuales> ::= parentheses1 <Lista-Expresiones> parentheses2  
 <Argumentos-Actuales> ::= parentheses1 parentheses2  
 <Lista-Expresiones> ::= <ExpOr>  
 <Lista-Expresiones> ::= <ExpOr> comma <Lista-Expresiones>

<Encadenado> ::= dot <Llamada-Método-Encadenado>  
 <Encadenado> ::= dot <Acceso-Variable-Encadenado>  
 <Llamada-Método-Encadenado> ::= id <Argumentos-Actuales> <Encadenado>  
 <Llamada-Método-Encadenado> ::= id <Argumentos-Actuales>  
 <Acceso-Variable-Encadenado> ::= id <Encadenado>  
 <Acceso-Variable-Encadenado> ::= id  
 <Acceso-Variable-Encadenado> ::= id brackets1 <ExpOr> brackets2  
 <Encadenado>  
 <Acceso-Variable-Encadenado> ::= id brackets1 <ExpOr> brackets2  
 <Sentencia> ::= semicolon  
 <Sentencia> ::= <Asignación> semicolon  
 <Sentencia> ::= <Sentencia-Simple> semicolon  
 <Sentencia> ::= pif parentheses1 <ExpOr> parentheses2 <Sentencia>  
 <Sentencia> ::= pif parentheses1 <ExpOr> parentheses2 <Sentencia> pelse  
 <Sentencia>  
 <Sentencia> ::= pwhile parentheses1 <ExpOr> parentheses2 <Sentencia>  
 <Sentencia> ::= <Bloque>  
 <Sentencia> ::= ret semicolon  
 <Sentencia> ::= ret <ExpOr> semicolon  
 <Bloque> ::= braces1 <Sentencias> braces2  
 <Asignación> ::= <AccesoVar-Simple> op\_equal <ExpOr>  
 <Asignación> ::= <AccesoSelf-Simple> op\_equal <ExpOr>  
 <AccesoVar-Simple> ::= id <Encadenados-Simples>  
 <AccesoVar-Simple> ::= id brackets1 <ExpOr> brackets2  
 <Encadenados-Simples> ::= <Encadenado-Simple> <Encadenados-Simples>  
 <Encadenados-Simples> ::=  $\lambda$   
 <AccesoSelf-Simple> ::= pself <Encadenados-Simples>  
 <Encadenado-Simple> ::= dot id  
 <Sentencia-Simple> ::= parentheses1 <ExpOr> parentheses2  
 <ExpOr> ::= <ExpOr> op\_or <ExpAnd>  
 <ExpOr> ::= <ExpAnd>  
 <ExpAnd> ::= <ExpAnd> op\_and <Explgual>  
 <ExpAnd> ::= <Explgual>  
 <Explgual> ::= <Explgual> <Oplgual> <ExpCompuesta>  
 <Explgual> ::= <ExpCompuesta>  
 <ExpCompuesta> ::= <ExpAd> <OpCompuesto> <ExpAd>  
 <ExpCompuesta> ::= <ExpAd>



$\langle \text{ExpAd} \rangle ::= \langle \text{ExpAd} \rangle \langle \text{OpAd} \rangle \langle \text{ExpMul} \rangle$   
 $\langle \text{ExpAd} \rangle ::= \langle \text{ExpMul} \rangle$   
 $\langle \text{ExpMul} \rangle ::= \langle \text{ExpMul} \rangle \langle \text{OpMul} \rangle \langle \text{ExpUn} \rangle$   
 $\langle \text{ExpMul} \rangle ::= \langle \text{ExpUn} \rangle$   
 $\langle \text{ExpUn} \rangle ::= \langle \text{OpUnario} \rangle \langle \text{ExpUn} \rangle$   
 $\langle \text{ExpUn} \rangle ::= \langle \text{Operando} \rangle$   
 $\langle \text{OpIguar} \rangle ::= \text{op\_rel\_equal}$   
 $\langle \text{OpIguar} \rangle ::= \text{op\_rel\_notequal}$   
 $\langle \text{OpCompuesto} \rangle ::= \text{op\_rel\_less}$   
 $\langle \text{OpCompuesto} \rangle ::= \text{op\_rel\_greater}$   
 $\langle \text{OpCompuesto} \rangle ::= \text{op\_rel\_lessequal}$   
 $\langle \text{OpCompuesto} \rangle ::= \text{op\_rel\_greaterequal}$   
 $\langle \text{OpAd} \rangle ::= \text{op\_sum}$   
 $\langle \text{OpAd} \rangle ::= \text{op\_sub}$   
 $\langle \text{OpUnario} \rangle ::= \text{op\_sum}$   
 $\langle \text{OpUnario} \rangle ::= \text{op\_sub}$   
 $\langle \text{OpUnario} \rangle ::= \text{op\_not}$   
 $\langle \text{OpUnario} \rangle ::= \text{op\_increment}$   
 $\langle \text{OpUnario} \rangle ::= \text{op\_decrement}$   
 $\langle \text{OpUnario} \rangle ::= \text{parentheses1 pint parentheses2}$   
 $\langle \text{OpMul} \rangle ::= \text{op\_mult}$   
 $\langle \text{OpMul} \rangle ::= \text{op\_div}$   
 $\langle \text{OpMul} \rangle ::= \text{op\_mod}$   
 $\langle \text{OpMul} \rangle ::= \text{op\_pdiv}$   
 $\langle \text{Operando} \rangle ::= \langle \text{Literal} \rangle$   
 $\langle \text{Operando} \rangle ::= \langle \text{Primario} \rangle$   
 $\langle \text{Operando} \rangle ::= \langle \text{Primario} \rangle \langle \text{Encadenado} \rangle$   
 $\langle \text{Literal} \rangle ::= \text{pnil}$   
 $\langle \text{Literal} \rangle ::= \text{ptrue}$   
 $\langle \text{Literal} \rangle ::= \text{pfalse}$   
 $\langle \text{Literal} \rangle ::= \text{const\_int}$   
 $\langle \text{Literal} \rangle ::= \text{const\_string}$   
 $\langle \text{Literal} \rangle ::= \text{const\_double}$   
 $\langle \text{Primario} \rangle ::= \langle \text{ExpresionParentizada} \rangle$   
 $\langle \text{Primario} \rangle ::= \langle \text{AccesoSelf} \rangle$   
 $\langle \text{Primario} \rangle ::= \langle \text{AccesoVar} \rangle$   
 $\langle \text{Primario} \rangle ::= \langle \text{Llamada-Método} \rangle$

<Primario> ::= <Llamada-Método-Estático>

<Primario> ::= <Llamada-Conclassor>

### 2.4.3. Gramática sin recursividad a izquierda

⟨Program⟩ ::= ⟨Lista-Definiciones⟩ ⟨Start⟩

⟨Start⟩ ::= pstart ⟨Bloque-Metodo⟩

<Lista-Definiciones> ::= <Class> <Lista-Definiciones>

<Lista-Definiciones> ::= <Impl> <Lista-Definiciones>

<Lista-Definiciones> ::= λ

⟨Class⟩ ::= pclass id\_class ⟨Herencia⟩ braces1 ⟨Atributos⟩ braces2

⟨Class⟩ ::= pclass id\_class braces1 ⟨Atributos⟩ braces2

<Atributos> ::= <Atributo> <Atributos>

<Atributos> ::= λ

⟨Impl⟩ ::= pimpl id\_class { ⟨Miembro⟩ <Miembros> }

<Miembros> ::= <Miembro> <Miembros>

<Miembros> ::= λ

⟨Herencia⟩ ::= colon ⟨Tipo⟩

⟨Miembro⟩ ::= ⟨Metodo⟩

⟨Miembro⟩ ::= ⟨Constructor⟩

⟨Constructor⟩ ::= dot ⟨Argumentos-Formales⟩ ⟨Bloque-Metodo⟩

⟨Atributo⟩ ::= ⟨Visibilidad⟩ ⟨Tipo⟩ ⟨Lista-Declaracion-Variables⟩ semicolon

⟨Atributo⟩ ::= ⟨Tipo⟩ ⟨Lista-Declaracion-Variables⟩ semicolon

⟨Metodo⟩ ::= ⟨Forma-Metodo⟩ pfn ⟨Tipo-Metodo⟩ id\_obj

⟨Argumentos-Formales⟩ ⟨Bloque-Metodo⟩

⟨Metodo⟩ ::= ⟨Forma-Metodo⟩ pfn id\_obj ⟨Argumentos-Formales⟩

⟨Bloque-Metodo⟩

⟨Metodo⟩ ::= pfn ⟨Tipo-Metodo⟩ id\_obj ⟨Argumentos-Formales⟩

⟨Bloque-Metodo⟩

⟨Metodo⟩ ::= pfn id\_obj ⟨Argumentos-Formales⟩ ⟨Bloque-Metodo⟩

⟨Visibilidad⟩ ::= ppub

⟨Forma-Metodo⟩ ::= pst

⟨Bloque-Metodo⟩ ::= braces1 ⟨Mas-Decl-Var-Locales⟩ ⟨Sentencias⟩ braces2

<Sentencias> ::= <Sentencia> <Sentencias>

<Sentencias> ::= λ

⟨Mas-Decl-Var-Locales⟩ ::= ⟨Decl-Var-Locales⟩ ⟨Mas-Decl-Var-Locales⟩

<Mas-Decl-Var-Locales> ::=  $\lambda$   
 <Decl-Var-Locales> ::= <Tipo> <Lista-Declaracion-Variables> semicolon  
 <Lista-Declaracion-Variables> ::= id\_obj  
 <Lista-Declaracion-Variables> ::= id\_obj comma <Lista-Declaracion-Variables>  
 <Argumentos-Formales> ::= parentheses1 <Lista-Argumentos-Formales>  
 parentheses2  
 <Argumentos-Formales> ::= parentheses1 parentheses2  
 <Lista-Argumentos-Formales> ::= <Argumento-Formal> comma  
 <Lista-Argumentos-Formales>  
 <Lista-Argumentos-Formales> ::= <Argumento-Formal>  
 <Argumento-Formal> ::= <Tipo> id\_obj  
 <Tipo-Metodo> ::= <Tipo>  
 <Tipo-Metodo> ::= pvoid  
 <Tipo> ::= <Tipo-Primitivo>  
 <Tipo> ::= parray <Tipo-Primitivo>  
 <Tipo> ::= id\_class  
 <Tipo-Primitivo> ::= pstr  
 <Tipo-Primitivo> ::= pbool  
 <Tipo-Primitivo> ::= pint  
 <Tipo-Primitivo> ::= pdouble  
 <ExpresionParentizada> ::= parentheses1 <ExpOr> parentheses2  
 <Encadenado>  
 <ExpresionParentizada> ::= parentheses1 <ExpOr> parentheses2  
 <AccesoSelf> ::= pself <Encadenado>  
 <AccesoSelf> ::= pself  
 <AccesoVar> ::= id <Encadenado>  
 <AccesoVar> ::= id  
 <AccesoVar> ::= id brackets1 <ExpOr> brackets2 <Encadenado>  
 <AccesoVar> ::= id brackets1 <ExpOr> brackets2  
 <Llamada-Metodo> ::= id <Argumentos-Actuales> <Encadenado>  
 <Llamada-Metodo> ::= id <Argumentos-Actuales>  
 <Llamada-Metodo-Estatico> ::= id\_class dot <Llamada-Metodo> <Encadenado>  
 <Llamada-Metodo-Estatico> ::= id\_class dot <Llamada-Metodo>  
 <Llamada-Conclassor> ::= pnew id\_class <Argumentos-Actuales>  
 <Encadenado>  
 <Llamada-Conclassor> ::= pnew id\_class <Argumentos-Actuales>  
 <Llamada-Conclassor> ::= pnew <Tipo-Primitivo> brackets1 <ExpOr> brackets2

<Argumentos-Actuales> ::= parentheses1 <Lista-Expresiones> parentheses2  
 <Argumentos-Actuales> ::= parentheses1 parentheses2  
 <Lista-Expresiones> ::= <ExpOr>  
 <Lista-Expresiones> ::= <ExpOr> comma <Lista-Expresiones>  
 <Encadenado> ::= dot <Llamada-Metodo-Encadenado>  
 <Encadenado> ::= dot <Acceso-Variable-Encadenado>  
 <Llamada-Metodo-Encadenado> ::= id <Argumentos-Actuales> <Encadenado>  
 <Llamada-Metodo-Encadenado> ::= id <Argumentos-Actuales>  
  
 <Acceso-Variable-Encadenado> ::= id <Encadenado>  
 <Acceso-Variable-Encadenado> ::= id  
 <Acceso-Variable-Encadenado> ::= id brackets1 <ExpOr> brackets2  
 <Encadenado>  
 <Acceso-Variable-Encadenado> ::= id brackets1 <ExpOr> brackets2  
 <Sentencia> ::= semicolon  
 <Sentencia> ::= <Asignación> semicolon  
 <Sentencia> ::= <Sentencia-Simple> semicolon  
 <Sentencia> ::= pif parentheses1 <ExpOr> parentheses2 <Sentencia>  
 <Sentencia> ::= pif parentheses1 <ExpOr> parentheses2 <Sentencia> pelse  
 <Sentencia>  
 <Sentencia> ::= pwhile parentheses1 <ExpOr> parentheses2 <Sentencia>  
 <Sentencia> ::= <Bloque>  
 <Sentencia> ::= ret semicolon  
 <Sentencia> ::= ret <ExpOr> semicolon  
 <Bloque> ::= braces1 <Sentencias> braces2  
 <Asignación> ::= <AccesoVar-Simple> op\_equal <ExpOr>  
 <Asignación> ::= <AccesoSelf-Simple> op\_equal <ExpOr>  
 <AccesoVar-Simple> ::= id <Encadenados-Simples>  
 <AccesoVar-Simple> ::= id brackets1 <ExpOr> brackets2  
 <Encadenados-Simples> ::= <Encadenado-Simple> <Encadenados-Simples>  
 <Encadenados-Simples> ::=  $\lambda$   
 <AccesoSelf-Simple> ::= pself <Encadenados-Simples>  
 <Encadenado-Simple> ::= dot id  
 <Sentencia-Simple> ::= parentheses1 <ExpOr> parentheses2  
 <ExpOr> ::= <ExpAnd> <ExpOr2>  
 <ExpOr2> ::= op\_or <ExpAnd> <ExpOr2>  
 <ExpOr2> ::=  $\lambda$

$\langle \text{ExpAnd} \rangle ::= \langle \text{ExpIgu} \rangle \langle \text{ExpAnd2} \rangle$   
 $\langle \text{ExpAnd2} \rangle ::= \text{op\_and} \langle \text{ExpIgu} \rangle \langle \text{ExpAnd2} \rangle$   
 $\langle \text{ExpAnd2} \rangle ::= \lambda$   
 $\langle \text{ExpIgu} \rangle ::= \langle \text{ExpCompuesta} \rangle \langle \text{ExpIgu2} \rangle$   
 $\langle \text{ExpIgu2} \rangle ::= \langle \text{OpIgu} \rangle \langle \text{ExpCompuesta} \rangle \langle \text{ExpIgu2} \rangle$   
 $\langle \text{ExpIgu2} \rangle ::= \lambda$   
 $\langle \text{ExpCompuesta} \rangle ::= \langle \text{ExpAd} \rangle \langle \text{OpCompuesto} \rangle \langle \text{ExpAd} \rangle$   
 $\langle \text{ExpCompuesta} \rangle ::= \langle \text{ExpAd} \rangle$   
 $\langle \text{ExpAd} \rangle ::= \langle \text{ExpMul} \rangle \langle \text{ExpAd2} \rangle$   
 $\langle \text{ExpAd2} \rangle ::= \langle \text{OpAd} \rangle \langle \text{ExpMul} \rangle \langle \text{ExpAd2} \rangle$   
 $\langle \text{ExpAd2} \rangle ::= \lambda$   
 $\langle \text{ExpMul} \rangle ::= \langle \text{ExpUn} \rangle \langle \text{ExpMul2} \rangle$   
 $\langle \text{ExpMul2} \rangle ::= \langle \text{OpMul} \rangle \langle \text{ExpUn} \rangle \langle \text{ExpMul2} \rangle$   
 $\langle \text{ExpMul2} \rangle ::= \lambda$   
 $\langle \text{ExpUn} \rangle ::= \langle \text{OpUnario} \rangle \langle \text{ExpUn} \rangle$   
 $\langle \text{ExpUn} \rangle ::= \langle \text{Operando} \rangle$   
 $\langle \text{OpIgu} \rangle ::= \text{op\_rel\_equal}$   
 $\langle \text{OpIgu} \rangle ::= \text{op\_rel\_notequal}$   
 $\langle \text{OpCompuesto} \rangle ::= \text{op\_rel\_less}$   
 $\langle \text{OpCompuesto} \rangle ::= \text{op\_rel\_greater}$   
 $\langle \text{OpCompuesto} \rangle ::= \text{op\_rel\_lessequal}$   
 $\langle \text{OpCompuesto} \rangle ::= \text{op\_rel\_greaterequal}$   
 $\langle \text{OpAd} \rangle ::= \text{op\_sum}$   
 $\langle \text{OpAd} \rangle ::= \text{op\_sub}$   
 $\langle \text{OpUnario} \rangle ::= \text{op\_sum}$   
 $\langle \text{OpUnario} \rangle ::= \text{op\_sub}$   
 $\langle \text{OpUnario} \rangle ::= \text{op\_not}$   
 $\langle \text{OpUnario} \rangle ::= \text{op\_increment}$   
 $\langle \text{OpUnario} \rangle ::= \text{op\_decrement}$   
 $\langle \text{OpUnario} \rangle ::= \text{parentheses1 pint parentheses2}$   
 $\langle \text{OpMul} \rangle ::= \text{op\_mult}$   
 $\langle \text{OpMul} \rangle ::= \text{op\_div}$   
 $\langle \text{OpMul} \rangle ::= \text{op\_mod}$   
 $\langle \text{OpMul} \rangle ::= \text{op\_pdiv}$   
 $\langle \text{Operando} \rangle ::= \langle \text{Literal} \rangle$   
 $\langle \text{Operando} \rangle ::= \langle \text{Primario} \rangle$   
 $\langle \text{Operando} \rangle ::= \langle \text{Primario} \rangle \langle \text{Encadenado} \rangle$

<Literal> ::= pnil  
 <Literal> ::= ptrue  
 <Literal> ::= pfalse  
 <Literal> ::= const\_int  
 <Literal> ::= const\_string  
 <Literal> ::= const\_double  
 <Primario> ::= <ExpresionParentizada>  
 <Primario> ::= <AccesoSelf>  
 <Primario> ::= <AccesoVar>  
 <Primario> ::= <Llamada-Método>  
 <Primario> ::= <Llamada-Método-Estático>  
 <Primario> ::= <Llamada-Conclassor>

#### 2.4.4. Gramática factorizada

<Program> ::= <Lista-Definiciones> <Start>  
 <Start> ::= pstart <Bloque-Metodo>  
 <Lista-Definiciones> ::= <Class> <Lista-Definiciones>  
 <Lista-Definiciones> ::= <Impl> <Lista-Definiciones>  
 <Lista-Definiciones> ::=  $\lambda$   
 <Class> ::= pclass id\_class <Class2>  
 <Class2> ::= <Herencia> braces1 <Atributos> braces2  
 <Class2> ::= braces1 <Atributos> braces2  
 <Atributos> ::= <Atributo> <Atributos>  
 <Atributos> ::=  $\lambda$   
 <Impl> ::= pimpl id\_class braces1 <Miembro> <Miembros> braces2  
 <Miembros> ::= <Miembro> <Miembros>  
 <Miembros> ::=  $\lambda$   
 <Herencia> ::= colon <Tipo>  
 <Miembro> ::= <Metodo>  
 <Miembro> ::= <Constructor>  
 <Constructor> ::= dot <Argumentos-Formales> <Bloque-Metodo>  
 <Atributo> ::= <Visibilidad> <Tipo> <Lista-Declaracion-Variables> semicolon  
 <Atributo> ::= <Tipo> <Lista-Declaracion-Variables> semicolon  
 <Metodo> ::= <Forma-Metodo> pfn <Metodo2>  
 <Metodo2> ::= <Tipo-Metodo> id\_obj <Argumentos-Formales> <Bloque-Metodo>  
 <Metodo2> ::= id\_obj <Argumentos-Formales> <Bloque-Metodo>

<Metodo> ::= pfn <Metodo2>  
 <Visibilidad> ::= ppub  
 <Forma-Metodo> ::= pst  
 <Bloque-Metodo> ::= braces1 <Mas-Decl-Var-Locales> <Sentencias> braces2  
 <Sentencias> ::= <Sentencia> <Sentencias>  
 <Sentencias> ::=  $\lambda$   
 <Mas-Decl-Var-Locales> ::= <Decl-Var-Locales> <Mas-Decl-Var-Locales>  
 <Mas-Decl-Var-Locales> ::=  $\lambda$   
 <Decl-Var-Locales> ::= <Tipo> <Lista-Declaracion-Variables> semicolon  
 <Lista-Declaracion-Variables> ::= id\_obj <Lista-Declaracion-Variables2>  
 <Lista-Declaracion-Variables2> ::= comma <Lista-Declaracion-Variables>  
 <Lista-Declaracion-Variables2> ::=  $\lambda$   
 <Argumentos-Formales> ::= parentheses1 <Argumentos-Formales2>  
 <Argumentos-Formales2> ::= <Lista-Argumentos-Formales> parentheses2  
 <Argumentos-Formales2> ::= parentheses2  
 <Lista-Argumentos-Formales> ::= <Argumento-Formal>  
 <Lista-Argumentos-Formales2>  
 <Lista-Argumentos-Formales2> ::= comma <Lista-Argumentos-Formales>  
 <Lista-Argumentos-Formales2> ::=  $\lambda$   
 <Argumento-Formal> ::= <Tipo> id\_obj  
 <Tipo-Metodo> ::= <Tipo>  
 <Tipo-Metodo> ::= pvoid  
 <Tipo> ::= <Tipo-Primitivo>  
 <Tipo> ::= parray <Tipo-Primitivo>  
 <Tipo> ::= id\_class  
 <Tipo-Primitivo> ::= pstr  
 <Tipo-Primitivo> ::= pbool  
 <Tipo-Primitivo> ::= pint  
 <Tipo-Primitivo> ::= pdouble  
 <ExpresionParentizada> ::= parentheses1 <ExpOr> parentheses2  
 <ExpresionParentizada2>  
 <ExpresionParentizada2> ::= <Encadenado>  
 <ExpresionParentizada2> ::=  $\lambda$   
 <AccesoSelf> ::= pself <EncadenadoAux>  
 <AccesoSelf2> ::= <Encadenado>  
 <AccesoSelf2> ::=  $\lambda$   
 <AccesoVar2> ::= <AccesoVar3>

$\langle \text{AccesoVar2} \rangle ::= \text{brackets1} \langle \text{ExpOr} \rangle \text{brackets2} \langle \text{AccesoVar3} \rangle$   
 $\langle \text{AccesoVar3} \rangle ::= \langle \text{Encadenado} \rangle$   
 $\langle \text{AccesoVar3} \rangle ::= \lambda$   
 $\langle \text{Llamada-Metodo} \rangle ::= \text{id} \langle \text{Argumentos-Actuales} \rangle \langle \text{EncadenadoAux} \rangle$   
 $\langle \text{Llamada-Metodo2} \rangle ::= \langle \text{Encadenado} \rangle$   
 $\langle \text{Llamada-Metodo2} \rangle ::= \lambda$   
 $\langle \text{Llamada-Metodo-Estatico} \rangle ::= \text{id\_class} \text{dot} \langle \text{Llamada-Metodo} \rangle$   
 $\langle \text{EncadenadoAux} \rangle$   
 $\langle \text{Llamada-Conclassor} \rangle ::= \text{pnew} \langle \text{Llamada-Conclassor2} \rangle$   
 $\langle \text{Llamada-Conclassor2} \rangle ::= \text{id\_class} \langle \text{Argumentos-Actuales} \rangle$   
 $\langle \text{EncadenadoAux} \rangle$   
 $\langle \text{Llamada-Conclassor2} \rangle ::= \langle \text{Tipo-Primitivo} \rangle \text{brackets1} \langle \text{ExpOr} \rangle \text{brackets2}$   
 $\langle \text{Argumentos-Actuales} \rangle ::= \text{parentheses1} \langle \text{Argumentos-Actuales2} \rangle$   
 $\langle \text{Argumentos-Actuales2} \rangle ::= \langle \text{Lista-Expresiones} \rangle \text{parentheses2}$   
 $\langle \text{Argumentos-Actuales2} \rangle ::= \text{parentheses2}$   
 $\langle \text{Lista-Expresiones} \rangle ::= \langle \text{ExpOr} \rangle \langle \text{Lista-Expresiones2} \rangle$   
 $\langle \text{Lista-Expresiones2} \rangle ::= \text{comma} \langle \text{Lista-Expresiones} \rangle$   
 $\langle \text{Lista-Expresiones2} \rangle ::= \lambda$   
 $\langle \text{Encadenado} \rangle ::= \text{dot id} \langle \text{Encadenado2} \rangle$   
 $\langle \text{Encadenado2} \rangle ::= \langle \text{Llamada-Metodo-Encadenado} \rangle$   
 $\langle \text{Encadenado2} \rangle ::= \langle \text{Acceso-Variable-Encadenado} \rangle$   
 $\langle \text{Llamada-Metodo-Encadenado} \rangle ::= \langle \text{Argumentos-Actuales} \rangle$   
 $\langle \text{EncadenadoAux} \rangle$   
 $\langle \text{Acceso-Variable-Encadenado} \rangle ::= \langle \text{Acceso-Variable-Encadenado2} \rangle$   
 $\langle \text{Acceso-Variable-Encadenado2} \rangle ::= \langle \text{EncadenadoAux} \rangle$   
 $\langle \text{Acceso-Variable-Encadenado2} \rangle ::= \text{brackets1} \langle \text{ExpOr} \rangle \text{brackets2}$   
 $\langle \text{EncadenadoAux} \rangle$   
 $\langle \text{Sentencia} \rangle ::= \text{semicolon}$   
 $\langle \text{Sentencia} \rangle ::= \langle \text{Asignación} \rangle \text{semicolon}$   
 $\langle \text{Sentencia} \rangle ::= \langle \text{Sentencia-Simple} \rangle \text{semicolon}$   
 $\langle \text{Sentencia} \rangle ::= \text{pif} \text{parentheses1} \langle \text{ExpOr} \rangle \text{parentheses2} \langle \text{Sentencia} \rangle$   
 $\langle \text{Sentencialf} \rangle$   
 $\langle \text{Sentencia} \rangle ::= \text{pwhile} \text{parentheses1} \langle \text{ExpOr} \rangle \text{parentheses2} \langle \text{Sentencia} \rangle$   
 $\langle \text{Sentencia} \rangle ::= \langle \text{Bloque} \rangle$   
 $\langle \text{Sentencia} \rangle ::= \text{ret} \langle \text{SentenciaRet} \rangle$   
 $\langle \text{SentenciaRet} \rangle ::= \text{semicolon}$   
 $\langle \text{SentenciaRet} \rangle ::= \langle \text{ExpOr} \rangle \text{semicolon}$



<Sentencialf> ::= pelse <Sentencia>  
 <Sentencialf> ::=  $\lambda$   
 <Bloque> ::= braces1 <Sentencias> braces2  
 <Asignación> ::= <AccesoVar-Simple> op\_equal <ExpOr>  
 <Asignación> ::= <AccesoSelf-Simple> op\_equal <ExpOr>  
 <AccesoVar-Simple> ::= id <AccesoVar-Simple2>  
 <AccesoVar-Simple2> ::= <Encadenados-Simples>  
 <AccesoVar-Simple2> ::= brackets1 <ExpOr> brackets2  
 <Encadenados-Simples> ::= <Encadenado-Simple> <Encadenados-Simples>  
 <Encadenados-Simples> ::=  $\lambda$   
 <AccesoSelf-Simple> ::= pself <Encadenados-Simples>  
 <Encadenado-Simple> ::= dot id  
 <Sentencia-Simple> ::= parentheses1 <ExpOr> parentheses2  
 <ExpOr> ::= <ExpAnd> <ExpOr2>  
 <ExpOr2> ::= op\_or <ExpAnd> <ExpOr2>  
 <ExpOr2> ::=  $\lambda$   
 <ExpAnd> ::= <ExpIlgual> <ExpAnd2>  
 <ExpAnd2> ::= op\_and <ExpIlgual> <ExpAnd2>  
 <ExpAnd2> ::=  $\lambda$   
 <ExpIlgual> ::= <ExpCompuesta> <ExpIlgual2>  
 <ExpIlgual2> ::= <OpIlgual> <ExpCompuesta> <ExpIlgual2>  
 <ExpIlgual2> ::=  $\lambda$   
 <ExpCompuesta> ::= <ExpAd> <ExpCompuesta2>  
 <ExpCompuesta2> ::= <OpCompuesto> <ExpAd>  
 <ExpCompuesta2> ::=  $\lambda$   
 <ExpAd> ::= <ExpMul> <ExpAd2>  
 <ExpAd2> ::= <OpAd> <ExpMul> <ExpAd2>  
 <ExpAd2> ::=  $\lambda$   
 <ExpMul> ::= <ExpUn> <ExpMul2>  
 <ExpMul2> ::= <OpMul> <ExpUn> <ExpMul2>  
 <ExpMul2> ::=  $\lambda$   
 <ExpUn> ::= <OpUnario> <ExpUn>  
 <ExpUn> ::= <Operando>  
 <OpIlgual> ::= op\_rel\_equal  
 <OpIlgual> ::= op\_rel\_notequal  
 <OpCompuesto> ::= op\_rel\_less  
 <OpCompuesto> ::= op\_rel\_greater

<OpCompuesto> ::= op\_rel\_lessequal  
 <OpCompuesto> ::= op\_rel\_greaterequal  
 <OpAd> ::= op\_sum  
 <OpAd> ::= op\_sub  
 <OpUnario> ::= op\_sum  
 <OpUnario> ::= op\_sub  
 <OpUnario> ::= op\_not  
 <OpUnario> ::= op\_increment  
 <OpUnario> ::= op\_decrement  
 <OpUnario> ::= parentheses1 pint parentheses2  
 <OpMul> ::= op\_mult  
 <OpMul> ::= op\_div  
 <OpMul> ::= op\_mod  
 <OpMul> ::= op\_pdiv  
 <Operando> ::= <Literal>  
 <Operando> ::= <Primario> <EncadenadoAux>  
 <Literal> ::= pnil  
 <Literal> ::= ptrue  
 <Literal> ::= pfalse  
 <Literal> ::= const\_int  
 <Literal> ::= const\_string  
 <Literal> ::= const\_double  
 <Primario> ::= <ExpresionParentizada>  
 <Primario> ::= <AccesoSelf>  
 <Primario> ::= id <Primario2>  
 <Primario> ::= <Llamada-Método-Estático>  
 <Primario> ::= <Llamada-Conclassor>  
 <Primario2> ::= <Argumentos-Actuales> <EncadenadoAux>  
 <Primario2> ::= <AccesoVar2>

#### 2.4.5. Eliminación de ambigüedad y transformación en LL(1)

Se encontró un conflicto en los Primeros de dos derivaciones de la misma regla ya que estos compartían terminales. El problema aparecía en la regla de <ExpUn>, ya que <OpUnario> derivaba en (Int) y <Operando> derivaba en <Primario> y este a su vez en <ExpresionParentizada>, por lo

que compartían el terminal ‘(. Para solucionarlo, se “elevó” la responsabilidad de generar las expresiones con paréntesis al no terminal <ExpUn>, por lo que se creó una regla adicional <ExpUnAux> que generaba las expresiones parentizadas que estaban en <Primario> y el operador de casteo a Int.

Además, otro problema que surgió fue la ambigüedad en el no terminal <Operando>. Originalmente, la regla tenía la forma <Primario> <Encadenado> o  $\lambda$ , sin embargo, cada una de las derivaciones de <Primario> a su vez podía terminar en <Encadenado> o  $\lambda$ , a excepción de una de las derivaciones de <Llamada-Conclator2>, por ello, existían varios árboles de derivación distintos para generar la misma regla desde <Operando>. Para solucionarlo, se agregó <Encadenado> y  $\lambda$  desde la regla faltante de <Llamada-Conclator2> y se eliminó <Encadenado> y  $\lambda$  de <Operando> ya que las mismas cadenas iban a poder seguir siendo generadas desde <Primario>.

Problemas similares surgieron luego en las reglas <Llamada-Método-Estático> y <ExpresionParentizada2> por lo que se aplicaron enfoques similares para arreglar los mismos.

Por último, se encontró el clásico problema de ambigüedad en las reglas referidas al if-else, por lo que allí se eligió reducir en lugar de desplazar para arreglar el mismo.

A continuación, se presenta la gramática final con todos los cambios mencionados.

```

<Program> ::= <Lista-Definiciones> <Start>
<Start> ::= pstart <Bloque-Metodo>
<Lista-Definiciones> ::= <Class> <Lista-Definiciones>
<Lista-Definiciones> ::= <Impl> <Lista-Definiciones>
<Lista-Definiciones> ::=  $\lambda$ 
<Class> ::= pclass id_class <Class2>
<Class2> ::= <Herencia> braces1 <Atributos> braces2
<Class2> ::= braces1 <Atributos> braces2
<Atributos> ::= <Atributo> <Atributos>
<Atributos> ::=  $\lambda$ 

```

<Impl> ::= pimpl id\_class braces1 <Miembro> <Miembros> braces2  
 <Miembros> ::= <Miembro> <Miembros>  
 <Miembros> ::=  $\lambda$   
 <Herencia> ::= colon <Tipo>  
 <Miembro> ::= <Metodo>  
 <Miembro> ::= <Constructor>  
 <Constructor> ::= dot <Argumentos-Formales> <Bloque-Metodo>  
 <Atributo> ::= <Visibilidad> <Tipo> <Lista-Declaracion-Variables> semicolon  
 <Atributo> ::= <Tipo> <Lista-Declaracion-Variables> semicolon  
 <Metodo> ::= <Forma-Metodo> pfn <Metodo2>  
 <Metodo> ::= pfn <Metodo2>  
 <Metodo2> ::= <Tipo-Metodo> id\_obj <Argumentos-Formales> <Bloque-Metodo>  
 <Metodo2> ::= id\_obj <Argumentos-Formales> <Bloque-Metodo>  
 <Visibilidad> ::= ppub  
 <Forma-Metodo> ::= pst  
 <Bloque-Metodo> ::= braces1 <Mas-Decl-Var-Locales> <Sentencias>  
 <Sentencias> ::= <Sentencia> <Sentencias>  
 <Sentencias> ::= braces2  
 <Mas-Decl-Var-Locales> ::= <Decl-Var-Locales> <Mas-Decl-Var-Locales>  
 <Mas-Decl-Var-Locales> ::=  $\lambda$   
 <Decl-Var-Locales> ::= <Tipo> <Lista-Declaracion-Variables> semicolon  
 <Lista-Declaracion-Variables> ::= id\_obj <Lista-Declaracion-Variables2>  
 <Lista-Declaracion-Variables2> ::= comma <Lista-Declaracion-Variables>  
 <Lista-Declaracion-Variables2> ::=  $\lambda$   
 <Argumentos-Formales> ::= parentheses1 <Argumentos-Formales2>  
 <Argumentos-Formales2> ::= <Lista-Argumentos-Formales> parentheses2  
 <Argumentos-Formales2> ::= parentheses2  
 <Lista-Argumentos-Formales> ::= <Argumento-Formal>  
 <Lista-Argumentos-Formales2>  
 <Lista-Argumentos-Formales2> ::= comma <Lista-Argumentos-Formales>  
 <Lista-Argumentos-Formales2> ::=  $\lambda$   
 <Argumento-Formal> ::= <Tipo> id\_obj  
 <Tipo-Metodo> ::= <Tipo>  
 <Tipo-Metodo> ::= pvoid  
 <Tipo> ::= <Tipo-Primitivo>  
 <Tipo> ::= parray <Tipo-Primitivo>  
 <Tipo> ::= id\_class

<Tipo-Primitivo> ::= pstr  
 <Tipo-Primitivo> ::= pbool  
 <Tipo-Primitivo> ::= pint  
 <Tipo-Primitivo> ::= pdouble  
 <ExpresionParentizada2> ::= <ExpOr> parentheses2  
 <ExpresionParentizada3>  
 <ExpresionParentizada3> ::= <Encadenado>  
 <ExpresionParentizada3> ::=  $\lambda$   
 <AccesoSelf> ::= pself <AccesoSelf2>  
 <AccesoSelf2> ::= <Encadenado>  
 <AccesoSelf2> ::=  $\lambda$   
 <AccesoVar2> ::= <AccesoVar3>  
 <AccesoVar2> ::= brackets1 <ExpOr> brackets2 <AccesoVar3>  
 <AccesoVar3> ::= <Encadenado>  
 <AccesoVar3> ::=  $\lambda$   
 <Llamada-Metodo> ::= id <Argumentos-Actuales> <Llamada-Metodo2>  
 <Llamada-Metodo2> ::= <Encadenado>  
 <Llamada-Metodo2> ::=  $\lambda$   
 <Llamada-Metodo-Estatico> ::= id\_class dot <Llamada-Metodo>  
 <Llamada-Conclassor> ::= pnnew <Llamada-Conclassor2>  
 <Llamada-Conclassor2> ::= id\_class <Argumentos-Actuales>  
 <Llamada-Conclassor3>  
 <Llamada-Conclassor2> ::= <Tipo-Primitivo> brackets1 <ExpOr> brackets2  
 <Llamada-Conclassor3>  
 <Llamada-Conclassor3> ::= <Encadenado>  
 <Llamada-Conclassor3> ::=  $\lambda$   
 <Argumentos-Actuales> ::= parentheses1 <Argumentos-Actuales2>  
 <Argumentos-Actuales2> ::= <Lista-Expresiones> parentheses2  
 <Argumentos-Actuales2> ::= parentheses2  
 <Lista-Expresiones> ::= <ExpOr> <Lista-Expresiones2>  
 <Lista-Expresiones2> ::= comma <Lista-Expresiones>  
 <Lista-Expresiones2> ::=  $\lambda$   
 <Encadenado> ::= dot id <Encadenado2>  
 <Encadenado2> ::= <Llamada-Metodo-Encadenado>  
 <Encadenado2> ::= <Acceso-Variable-Encadenado>  
 <Llamada-Metodo-Encadenado> ::= <Argumentos-Actuales>  
 <Llamada-Metodo-Encadenado2>

<Llamada-Metodo-Encadenado2> ::= <Encadenado>  
 <Llamada-Metodo-Encadenado2> ::=  $\lambda$   
 <Acceso-Variable-Encadenado> ::= <Acceso-Variable-Encadenado3>  
 <Acceso-Variable-Encadenado> ::= brackets1 <ExpOr> brackets2  
 <Acceso-Variable-Encadenado3>  
 <Acceso-Variable-Encadenado3> ::= <Encadenado>  
 <Acceso-Variable-Encadenado3> ::=  $\lambda$   
 <Sentencia> ::= semicolon  
 <Sentencia> ::= <Asignación> semicolon  
 <Sentencia> ::= <Sentencia-Simple> semicolon  
 <Sentencia> ::= pif parentheses1 <ExpOr> parentheses2 <Sentencia>  
 <Sentencialf>  
 <Sentencia> ::= pwhile parentheses1 <ExpOr> parentheses2 <Sentencia>  
 <Sentencia> ::= <Bloque>  
 <Sentencia> ::= ret <SentenciaRet>  
 <SentenciaRet> ::= semicolon  
 <SentenciaRet> ::= <ExpOr> semicolon  
 <Sentencialf> ::= pelse <Sentencia>  
 <Sentencialf> ::=  $\lambda$   
 <Bloque> ::= braces1 <Sentencias>  
 <Asignación> ::= <AccesoVar-Simple> op\_equal <ExpOr>  
 <Asignación> ::= <AccesoSelf-Simple> op\_equal <ExpOr>  
 <AccesoVar-Simple> ::= id <AccesoVar-Simple2>  
 <AccesoVar-Simple2> ::= <Encadenados-Simples>  
 <AccesoVar-Simple2> ::= brackets1 <ExpOr> brackets2  
 <Encadenados-Simples> ::= <Encadenado-Simple> <Encadenados-Simples>  
 <Encadenados-Simples> ::=  $\lambda$   
 <AccesoSelf-Simple> ::= pself <Encadenados-Simples>  
 <Encadenado-Simple> ::= dot id  
 <Sentencia-Simple> ::= parentheses1 <ExpOr> parentheses2  
 <ExpOr> ::= <ExpAnd> <ExpOr2>  
 <ExpOr2> ::= op\_or <ExpAnd> <ExpOr2>  
 <ExpOr2> ::=  $\lambda$   
 <ExpAnd> ::= <ExpIguual> <ExpAnd2>  
 <ExpAnd2> ::= op\_and <ExpIguual> <ExpAnd2>  
 <ExpAnd2> ::=  $\lambda$   
 <ExpIguual> ::= <ExpCompuesta> <ExpIguual2>

$\langle \text{ExpIguar2} \rangle ::= \langle \text{OpIguar} \rangle \langle \text{ExpCompuesta} \rangle \langle \text{ExpIguar2} \rangle$   
 $\langle \text{ExpIguar2} \rangle ::= \lambda$   
 $\langle \text{ExpCompuesta} \rangle ::= \langle \text{ExpAd} \rangle \langle \text{ExpCompuesta2} \rangle$   
 $\langle \text{ExpCompuesta2} \rangle ::= \langle \text{OpCompuesto} \rangle \langle \text{ExpAd} \rangle$   
 $\langle \text{ExpCompuesta2} \rangle ::= \lambda$   
 $\langle \text{ExpAd} \rangle ::= \langle \text{ExpMul} \rangle \langle \text{ExpAd2} \rangle$   
 $\langle \text{ExpAd2} \rangle ::= \langle \text{OpAd} \rangle \langle \text{ExpMul} \rangle \langle \text{ExpAd2} \rangle$   
 $\langle \text{ExpAd2} \rangle ::= \lambda$   
 $\langle \text{ExpMul} \rangle ::= \langle \text{ExpUn} \rangle \langle \text{ExpMul2} \rangle$   
 $\langle \text{ExpMul2} \rangle ::= \langle \text{OpMul} \rangle \langle \text{ExpUn} \rangle \langle \text{ExpMul2} \rangle$   
 $\langle \text{ExpMul2} \rangle ::= \lambda$   
 $\langle \text{ExpUn} \rangle ::= \langle \text{OpUnario} \rangle \langle \text{ExpUn} \rangle$   
 $\langle \text{ExpUn} \rangle ::= \langle \text{Operando} \rangle$   
 $\langle \text{ExpUn} \rangle ::= \text{parenthesis1} \langle \text{ExpUnAux} \rangle$   
 $\langle \text{ExpUnAux} \rangle ::= \text{pint parentheses2} \langle \text{ExpUn} \rangle$   
 $\langle \text{ExpUnAux} \rangle ::= \langle \text{ExpresionParentizada2} \rangle$   
 $\langle \text{OpIguar} \rangle ::= \text{op\_rel\_equal}$   
 $\langle \text{OpIguar} \rangle ::= \text{op\_rel\_notequal}$   
 $\langle \text{OpCompuesto} \rangle ::= \text{op\_rel\_less}$   
 $\langle \text{OpCompuesto} \rangle ::= \text{op\_rel\_greater}$   
 $\langle \text{OpCompuesto} \rangle ::= \text{op\_rel\_lessequal}$   
 $\langle \text{OpCompuesto} \rangle ::= \text{op\_rel\_greaterequal}$   
 $\langle \text{OpAd} \rangle ::= \text{op\_sum}$   
 $\langle \text{OpAd} \rangle ::= \text{op\_sub}$   
 $\langle \text{OpUnario} \rangle ::= \text{op\_sum}$   
 $\langle \text{OpUnario} \rangle ::= \text{op\_sub}$   
 $\langle \text{OpUnario} \rangle ::= \text{op\_not}$   
 $\langle \text{OpUnario} \rangle ::= \text{op\_increment}$   
 $\langle \text{OpUnario} \rangle ::= \text{op\_decrement}$   
 $\langle \text{OpMul} \rangle ::= \text{op\_mult}$   
 $\langle \text{OpMul} \rangle ::= \text{op\_div}$   
 $\langle \text{OpMul} \rangle ::= \text{op\_mod}$   
 $\langle \text{OpMul} \rangle ::= \text{pdiv}$   
 $\langle \text{Operando} \rangle ::= \langle \text{Literal} \rangle$   
 $\langle \text{Operando} \rangle ::= \langle \text{Primario} \rangle$   
 $\langle \text{Literal} \rangle ::= \text{pnil}$   
 $\langle \text{Literal} \rangle ::= \text{ptrue}$

$\langle \text{Literal} \rangle ::= \text{pfalse}$   
 $\langle \text{Literal} \rangle ::= \text{const\_int}$   
 $\langle \text{Literal} \rangle ::= \text{const\_string}$   
 $\langle \text{Literal} \rangle ::= \text{const\_double}$   
 $\langle \text{Primario} \rangle ::= \langle \text{AccesoSelf} \rangle$   
 $\langle \text{Primario} \rangle ::= \text{id } \langle \text{Primario2} \rangle$   
 $\langle \text{Primario} \rangle ::= \langle \text{Llamada-Método-Estático} \rangle$   
 $\langle \text{Primario} \rangle ::= \langle \text{Llamada-Conclassor} \rangle$   
 $\langle \text{Primario2} \rangle ::= \langle \text{Argumentos-Actuales} \rangle \langle \text{Primario3} \rangle$   
 $\langle \text{Primario2} \rangle ::= \langle \text{AccesoVar2} \rangle$   
 $\langle \text{Primario3} \rangle ::= \langle \text{Encadenado} \rangle$   
 $\langle \text{Primario3} \rangle ::= \lambda$

#### 2.4.6. Primeros y Siguietes

Los primeros y siguietes fueron calculados utilizando las siguientes reglas:

- Si  $S \rightarrow aB$ ,  $a \in \text{Primeros}(S)$
- Si  $S \rightarrow AB$ ,  $\text{Primeros}(A) \subseteq \text{Primeros}(S)$
- Si  $S$  es la primera regla de la gramática,  $\$ \in \text{Siguietes}(S)$
- Si  $A \rightarrow BS$ ,  $\text{Siguietes}(S) \supseteq \text{Siguietes}(A)$
- Si  $A \rightarrow BSC$  y  $\lambda \notin \text{Primeros}(C)$ ,  $\text{Siguietes}(S) \supseteq \text{Primeros}(C)$
- Si  $A \rightarrow BSCD$  y  $\lambda \in \text{Primeros}(C)$ ,  $\text{Siguietes}(S) \supseteq \text{Primeros}(C) \cup \text{Primeros}(D)$
- Si  $A \rightarrow BSC$  y  $\lambda \in \text{Primeros}(C)$ ,  $\text{Siguietes}(S) \supseteq \text{Primeros}(C) \cup \text{Siguietes}(A)$
- En los siguietes no se coloca lambda

Tabla 2. Primeros y siguietes de la gramática.

| Regla                                                                                                                            | Primeros                     | Siguietes |
|----------------------------------------------------------------------------------------------------------------------------------|------------------------------|-----------|
| $\langle \text{Program} \rangle ::=$<br>$\langle \text{Lista-Definiciones} \rangle \langle \text{Start} \rangle$                 | { $\lambda$ , pclass, pimpl} | {éxito}   |
| $\langle \text{Start} \rangle ::= \text{pstart}$<br>$\langle \text{Bloque-Metodo} \rangle$                                       | {pstart}                     | {exito}   |
| $\langle \text{Lista-Definiciones} \rangle ::=$<br>$\langle \text{Class} \rangle$<br>$\langle \text{Lista-Definiciones} \rangle$ | { $\lambda$ , pclass, pimpl} | {pstart}  |



|                                                                                              |                                                                         |                                                                  |
|----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|------------------------------------------------------------------|
| <Lista-Definiciones> ::=<br><Impl><br><Lista-Definiciones>                                   |                                                                         |                                                                  |
| <Lista-Definiciones> ::= $\lambda$                                                           |                                                                         |                                                                  |
| <Class> ::= <b>pclass</b><br><b>id_class</b> <Class2>                                        | {pclass}                                                                | {pclass, pimpl, pstart}                                          |
| <Class2> ::= <Herencia><br><b>braces1</b> <Atributos><br><b>braces2</b>                      | {braces1, colon}                                                        | {pclass, pimpl, pstart}                                          |
| <Class2> ::= <b>braces1</b><br><Atributos> <b>braces2</b>                                    |                                                                         |                                                                  |
| <Atributos> ::= <Atributo><br><Atributos>                                                    | { $\lambda$ , pstr, pbool, pint,<br>pdouble, parray, id_class,<br>ppub} | {braces2}                                                        |
| <Atributos> ::= $\lambda$                                                                    |                                                                         |                                                                  |
| <Impl> ::= <b>pimpl id_class</b><br><b>braces1</b> <Miembro><br><Miembros> <b>braces2</b>    | {pimpl}                                                                 | {pclass, pimpl, pstart}                                          |
| <Miembros> ::=<br><Miembro> <Miembros>                                                       | {pfn, pst, dot, $\lambda$ }                                             | {braces2}                                                        |
| <Miembros> ::= $\lambda$                                                                     |                                                                         |                                                                  |
| <Herencia> ::= <b>colon</b> <Tipo>                                                           | {colon}                                                                 | {braces1}                                                        |
| <Miembro> ::= <Metodo>                                                                       | {pfn, pst, dot}                                                         | {pfn, pst, dot, braces2}                                         |
| <Miembro> ::=<br><Constructor>                                                               |                                                                         |                                                                  |
| <Constructor> ::= <b>dot</b><br><Argumentos-Formales><br><Bloque-Metodo>                     | {dot}                                                                   | {pfn, pst, dot, braces2}                                         |
| <Atributo> ::= <Visibilidad><br><Tipo><br><Lista-Declaracion-Variable<br>s> <b>semicolon</b> | {pstr, pbool, pint, pdouble,<br>parray, id_class, ppub}                 | {pstr, pbool, pint, pdouble, parray, id_class,<br>ppub, braces2} |
| <Atributo> ::= <Tipo><br><Lista-Declaracion-Variable<br>s> <b>semicolon</b>                  |                                                                         |                                                                  |

|                                                                                           |                                                                                 |                                                                                                                            |
|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <Metodo> ::=<br><Forma-Metodo> <b>pfn</b><br><Metodo2>                                    | {pfn, pst}                                                                      | {pfn, pst, dot, braces2}                                                                                                   |
| <Metodo> ::= <b>pfn</b><br><Metodo2>                                                      |                                                                                 |                                                                                                                            |
| <Metodo2> ::=<br><Tipo-Metodo> <b>id_obj</b><br><Argumentos-Formales><br><Bloque-Metodo>  | {pstr, pbool, pint, pdouble,<br>parray, id_class, pvoid,<br>id_obj}             | {pfn, pst, dot, braces2}                                                                                                   |
| <Metodo2> ::= <b>id_obj</b><br><Argumentos-Formales><br><Bloque-Metodo>                   |                                                                                 |                                                                                                                            |
| <Visibilidad> ::= <b>ppub</b>                                                             | {ppub}                                                                          | {pstr, pbool, pint, pdouble, parray, id_class}                                                                             |
| <Forma-Metodo> ::= <b>pst</b>                                                             | {pst}                                                                           | {pfn}                                                                                                                      |
| <Bloque-Metodo> ::=<br><b>braces1</b><br><Mas-Decl-Var-Locales><br><Sentencias>           | {braces1}                                                                       | {exito, pfn, pst, dot, braces2}                                                                                            |
| <Sentencias> ::=<br><Sentencia><br><Sentencias>                                           | {semicolon,pif, pwhile, id,<br>pself, ret,<br>parenthesis1,braces1,<br>braces2} | {exito, pfn, pst, dot, braces2, pelse,<br>semicolon,pif, pwhile, id, pself, ret,<br>parenthesis1,braces1}                  |
| <Sentencias> ::= braces2                                                                  |                                                                                 |                                                                                                                            |
| <Mas-Decl-Var-Locales> ::=<br><Decl-Var-Locales><br><Mas-Decl-Var-Locales>                | {λ, pstr, pbool, pint,<br>pdouble, parray, id_class}                            | {semicolon,pif, pwhile, id, pself, ret,<br>parenthesis1,braces1, braces2}                                                  |
| <Mas-Decl-Var-Locales> ::=<br>λ                                                           |                                                                                 |                                                                                                                            |
| <Decl-Var-Locales> ::=<br><Tipo><br><Lista-Declaracion-Variable<br>s> semicolon           | {pstr, pbool, pint, pdouble,<br>parray, id_class}                               | {pstr, pbool, pint, pdouble, parray, id_class,<br>semicolon,pif, pwhile, id, pself, ret,<br>parenthesis1,braces1, braces2} |
| <Lista-Declaracion-Variable<br>s> ::= <b>id_obj</b><br><Lista-Declaracion-Variabl<br>es2> | {id_obj}                                                                        | {semicolon}                                                                                                                |

|                                                                                       |                                                              |                       |
|---------------------------------------------------------------------------------------|--------------------------------------------------------------|-----------------------|
| <Lista-Declaracion-Variables2> ::= <b>comma</b><br><Lista-Declaracion-Variables>      | {comma, λ}                                                   | {semicolon}           |
| <Lista-Declaracion-Variables2> ::= λ                                                  |                                                              |                       |
| <Argumentos-Formales> ::= <b>parentheses1</b><br><Argumentos-Formales2>               | {parentheses1}                                               | {braces1}             |
| <Argumentos-Formales2> ::=<br><Lista-Argumentos-Formales> <b>parentheses2</b>         | {pstr, pbool, pint, pdouble, parray, id_class, parentheses2} | {braces1}             |
| <Argumentos-Formales2> ::= <b>parentheses2</b>                                        |                                                              |                       |
| <Lista-Argumentos-Formales> ::=<br><Argumento-Formal><br><Lista-Argumentos-Formales2> | {pstr, pbool, pint, pdouble, parray, id_class}               | {parentheses2}        |
| <Lista-Argumentos-Formales2> ::= <b>comma</b><br><Lista-Argumentos-Formales>          | {comma, λ}                                                   | {parentheses2}        |
| <Lista-Argumentos-Formales2> ::= λ                                                    |                                                              |                       |
| <Argumento-Formal> ::=<br><Tipo> <b>id_obj</b>                                        | {pstr, pbool, pint, pdouble, parray, id_class}               | {comma, parentheses2} |
| <Tipo-Metodo> ::= <Tipo>                                                              | {pstr, pbool, pint, pdouble, parray, id_class, pvoid}        | {id_obj}              |
| <Tipo-Metodo> ::= <b>pvoid</b>                                                        |                                                              |                       |
| <Tipo> ::= <Tipo-Primitivo>                                                           | {pstr, pbool, pint, pdouble, parray, id_class}               | {id_obj, braces1}     |
| <Tipo> ::= <b>parray</b><br><Tipo-Primitivo>                                          |                                                              |                       |
| <Tipo> ::= <b>id_class</b>                                                            |                                                              |                       |

|                                                                                |                                                                                                                                                                                |                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ⟨Tipo-Primitivo⟩ ::= <b>pstr</b>                                               | {pstr, pbool, pint, pdouble}                                                                                                                                                   | {id_obj, braces1, brackets1}                                                                                                                                                                                                   |
| ⟨Tipo-Primitivo⟩ ::= <b>pbool</b>                                              |                                                                                                                                                                                |                                                                                                                                                                                                                                |
| ⟨Tipo-Primitivo⟩ ::= <b>pint</b>                                               |                                                                                                                                                                                |                                                                                                                                                                                                                                |
| ⟨Tipo-Primitivo⟩ ::= <b>pdouble</b>                                            |                                                                                                                                                                                |                                                                                                                                                                                                                                |
| ⟨ExpresionParentizada2⟩<br>::= <ExpOr> parentheses2<br><ExpresionParentizada3> | {pnil, ptrue, pfalse,<br>const_int, const_string,<br>const_double,<br>parentheses1, pself, id,<br>pnew, id_class, op_sum,<br>op_sub, op_not,<br>op_increment,<br>op_decrement} | {op_mult, op_div, op_mod, op_pdiv, op_sum,<br>op_sub, op_rel_less, op_rel_greater,<br>op_rel_lessequal, op_rel_greaterequal,<br>op_rel_equal, op_rel_notequal, op_and,<br>op_or, parentheses2, brackets2, semicolon,<br>comma} |
| <ExpresionParentizada3><br>::= <Encadenado>                                    | {dot, λ}                                                                                                                                                                       | {op_mult, op_div, op_mod, op_pdiv, op_sum,<br>op_sub, op_rel_less, op_rel_greater,<br>op_rel_lessequal, op_rel_greaterequal,<br>op_rel_equal, op_rel_notequal, op_and,<br>op_or, parentheses2, brackets2, semicolon,<br>comma} |
| <ExpresionParentizada3><br>::= λ                                               |                                                                                                                                                                                |                                                                                                                                                                                                                                |
| ⟨AccesoSelf⟩ ::= <b>pself</b><br>⟨AccesoSelf2⟩                                 | {pself}                                                                                                                                                                        | {op_mult, op_div, op_mod, op_pdiv, op_sum,<br>op_sub, op_rel_less, op_rel_greater,<br>op_rel_lessequal, op_rel_greaterequal,<br>op_rel_equal, op_rel_notequal, op_and,<br>op_or, parentheses2, brackets2, semicolon,<br>comma} |
| ⟨AccesoSelf2⟩::=<br><Encadenado>                                               | {dot, λ}                                                                                                                                                                       | {op_mult, op_div, op_mod, op_pdiv, op_sum,<br>op_sub, op_rel_less, op_rel_greater,<br>op_rel_lessequal, op_rel_greaterequal,<br>op_rel_equal, op_rel_notequal, op_and,<br>op_or, parentheses2, brackets2, semicolon,<br>comma} |
| ⟨AccesoSelf2⟩ ::= λ                                                            |                                                                                                                                                                                |                                                                                                                                                                                                                                |
| ⟨AccesoVar2⟩ ::=<br><AccesoVar3>                                               | {dot, λ, brackets1}                                                                                                                                                            | {op_mult, op_div, op_mod, op_pdiv, op_sum,<br>op_sub, op_rel_less, op_rel_greater,<br>op_rel_lessequal, op_rel_greaterequal,                                                                                                   |

|                                                                                                                            |                   |                                                                                                                                                                                                                 |
|----------------------------------------------------------------------------------------------------------------------------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                            |                   | op_rel_equal, op_rel_notequal, op_and, op_or, parentheses2, brackets2, semicolon, comma}                                                                                                                        |
| $\langle \text{AccesoVar2} \rangle ::=$<br><b>brackets1</b> <ExpOr><br><b>brackets2</b> <AccesoVar3>                       |                   |                                                                                                                                                                                                                 |
| $\langle \text{AccesoVar3} \rangle ::=$<br><Encadenado>                                                                    | {dot, $\lambda$ } | {op_mult, op_div, op_mod, op_pdiv, op_sum, op_sub, op_rel_less, op_rel_greater, op_rel_lessequal, op_rel_greaterequal, op_rel_equal, op_rel_notequal, op_and, op_or, parentheses2, brackets2, semicolon, comma} |
| $\langle \text{AccesoVar3} \rangle ::= \lambda$                                                                            |                   |                                                                                                                                                                                                                 |
| $\langle \text{Llamada-Metodo} \rangle ::= \text{id}$<br><Argumentos-Actuales><br>$\langle \text{Llamada-Metodo2} \rangle$ | {id}              | {op_mult, op_div, op_mod, op_pdiv, op_sum, op_sub, op_rel_less, op_rel_greater, op_rel_lessequal, op_rel_greaterequal, op_rel_equal, op_rel_notequal, op_and, op_or, parentheses2, brackets2, semicolon, comma} |
| $\langle \text{Llamada-Metodo2} \rangle ::=$<br><Encadenado>                                                               | {dot, $\lambda$ } | {op_mult, op_div, op_mod, op_pdiv, op_sum, op_sub, op_rel_less, op_rel_greater, op_rel_lessequal, op_rel_greaterequal, op_rel_equal, op_rel_notequal, op_and, op_or, parentheses2, brackets2, semicolon, comma} |
| $\langle \text{Llamada-Metodo2} \rangle ::= \lambda$                                                                       |                   |                                                                                                                                                                                                                 |
| $\langle \text{Llamada-Metodo-Estatico} \rangle ::= \text{id\_class dot}$<br>$\langle \text{Llamada-Metodo} \rangle$       | {id_class}        | {op_mult, op_div, op_mod, op_pdiv, op_sum, op_sub, op_rel_less, op_rel_greater, op_rel_lessequal, op_rel_greaterequal, op_rel_equal, op_rel_notequal, op_and, op_or, parentheses2, brackets2, semicolon, comma} |
| $\langle \text{Llamada-Conclassor} \rangle ::=$<br><b>pnew</b><br><Llamada-Conclassor2>                                    | {pnew}            | {op_mult, op_div, op_mod, op_pdiv, op_sum, op_sub, op_rel_less, op_rel_greater, op_rel_lessequal, op_rel_greaterequal, op_rel_equal, op_rel_notequal, op_and,                                                   |

|                                                                                                                        |                                                                                                                                                                                                 |                                                                                                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                        |                                                                                                                                                                                                 | op_or, parentheses2, brackets2, semicolon, comma}                                                                                                                                                                                      |
| <Llamada-Conclassor2><br>::= id_class<br><Argumentos-Actuales><br><Llamada-Conclassor3>                                | {id_class, pstr, pbool, pint, pdouble}                                                                                                                                                          | {op_mult, op_div, op_mod, op_pdiv, op_sum,<br>op_sub, op_rel_less, op_rel_greater,<br>op_rel_lessequal, op_rel_greaterequal,<br>op_rel_equal, op_rel_notequal, op_and,<br>op_or, parentheses2, brackets2, semicolon,<br>comma}         |
| <Llamada-Conclassor2> ::= <b>&lt;Tipo-Primitivo&gt; brackets1</b><br><ExpOr> <b>brackets2</b><br><Llamada-Conclassor3> |                                                                                                                                                                                                 |                                                                                                                                                                                                                                        |
| <Llamada-Conclassor3><br>::= <Encadenado>                                                                              | {dot, λ}                                                                                                                                                                                        | {op_mult, op_div, op_mod, op_pdiv, op_sum,<br>op_sub, op_rel_less, op_rel_greater,<br>op_rel_lessequal, op_rel_greaterequal,<br>op_rel_equal, op_rel_notequal, op_and,<br>op_or, parentheses2, brackets2, semicolon,<br>comma}         |
| <Llamada-Conclassor3><br>::= λ                                                                                         |                                                                                                                                                                                                 |                                                                                                                                                                                                                                        |
| <Argumentos-Actuales> ::= <b>parentheses1</b><br><Argumentos-Actuales2><br>>                                           | {parentheses1}                                                                                                                                                                                  | {dot, op_mult, op_div, op_mod, op_pdiv,<br>op_sum, op_sub, op_rel_less,<br>op_rel_greater, op_rel_lessequal,<br>op_rel_greaterequal, op_rel_equal,<br>op_rel_notequal, op_and, op_or,<br>parentheses2, brackets2, semicolon,<br>comma} |
| <Argumentos-Actuales2><br>::= <Lista-Expresiones><br><b>parentheses2</b>                                               | {pnil, ptrue, pfalse,<br>const_int, const_string,<br>const_double,<br>parentheses1, pself, id,<br>pnew, id_class, op_sum,<br>op_sub, op_not,<br>op_increment,<br>op_decrement,<br>parentheses2} | {dot, op_mult, op_div, op_mod, op_pdiv,<br>op_sum, op_sub, op_rel_less,<br>op_rel_greater, op_rel_lessequal,<br>op_rel_greaterequal, op_rel_equal,<br>op_rel_notequal, op_and, op_or,<br>parentheses2, brackets2, semicolon,<br>comma} |
| <Argumentos-Actuales2><br>::= <b>parentheses2</b>                                                                      |                                                                                                                                                                                                 |                                                                                                                                                                                                                                        |

|                                                                                                         |                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>⟨Lista-Expresiones⟩ ::=</p> <p>&lt;ExpOr&gt;</p> <p>⟨Lista-Expresiones2⟩</p>                         | <p>{pnil, ptrue, pfalse,</p> <p>const_int, const_string,</p> <p>const_double,</p> <p>parentheses1, pself, id,</p> <p>pnew, id_class, op_sum,</p> <p>op_sub, op_not,</p> <p>op_increment,</p> <p>op_decrement}</p> | <p>{parenthesis2}</p>                                                                                                                                                                                                                                     |
| <p>⟨Lista-Expresiones2⟩ ::=</p> <p>comma</p> <p>⟨Lista-Expresiones⟩</p>                                 | <p>{comma, λ}</p>                                                                                                                                                                                                 | <p>{parenthesis2}</p>                                                                                                                                                                                                                                     |
| <p>⟨Lista-Expresiones2⟩ ::= λ</p>                                                                       |                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                           |
| <p>⟨Encadenado⟩ ::= <b>dot id</b></p> <p>&lt;Encadenado2&gt;</p>                                        | <p>{dot}</p>                                                                                                                                                                                                      | <p>{op_mult, op_div, op_mod, op_pdiv, op_sum,</p> <p>op_sub, op_rel_less, op_rel_greater,</p> <p>op_rel_lessequal, op_rel_greaterequal,</p> <p>op_rel_equal, op_rel_notequal, op_and,</p> <p>op_or, parentheses2, brackets2, semicolon,</p> <p>comma}</p> |
| <p>&lt;Encadenado2&gt; ::=</p> <p>⟨Llamada-Metodo-Encadenado⟩</p>                                       | <p>{parenthesis1, dot,</p> <p>brackets1, λ}</p>                                                                                                                                                                   | <p>{op_mult, op_div, op_mod, op_pdiv, op_sum,</p> <p>op_sub, op_rel_less, op_rel_greater,</p> <p>op_rel_lessequal, op_rel_greaterequal,</p> <p>op_rel_equal, op_rel_notequal, op_and,</p> <p>op_or, parentheses2, brackets2, semicolon,</p> <p>comma}</p> |
| <p>&lt;Encadenado2&gt; ::=</p> <p>⟨Acceso-Variable-Encadenado⟩</p>                                      |                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                           |
| <p>⟨Llamada-Metodo-Encadenado⟩ ::=</p> <p>⟨Argumentos-Actuales⟩</p> <p>⟨Llamada-Metodo-Encadenado2⟩</p> | <p>{parenthesis1}</p>                                                                                                                                                                                             | <p>{op_mult, op_div, op_mod, op_pdiv, op_sum,</p> <p>op_sub, op_rel_less, op_rel_greater,</p> <p>op_rel_lessequal, op_rel_greaterequal,</p> <p>op_rel_equal, op_rel_notequal, op_and,</p> <p>op_or, parentheses2, brackets2, semicolon,</p> <p>comma}</p> |
| <p>⟨Llamada-Metodo-Encadenado2⟩ ::= &lt;Encadenado&gt;</p>                                              | <p>{dot, λ}</p>                                                                                                                                                                                                   | <p>{op_mult, op_div, op_mod, op_pdiv, op_sum,</p> <p>op_sub, op_rel_less, op_rel_greater,</p> <p>op_rel_lessequal, op_rel_greaterequal,</p> <p>op_rel_equal, op_rel_notequal, op_and,</p>                                                                 |

|                                                                                                  |                                                               |                                                                                                                                                                                                                 |
|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                  |                                                               | op_or, parentheses2, brackets2, semicolon, comma}                                                                                                                                                               |
| ⟨Llamada-Metodo-Encadenado2⟩ ::= λ                                                               |                                                               |                                                                                                                                                                                                                 |
| ⟨Acceso-Variable-Encadenado⟩ ::= <Acceso-Variable-Encadenado3>                                   | {dot, λ. brackets1}                                           | {op_mult, op_div, op_mod, op_pdiv, op_sum, op_sub, op_rel_less, op_rel_greater, op_rel_lessequal, op_rel_greaterequal, op_rel_equal, op_rel_notequal, op_and, op_or, parentheses2, brackets2, semicolon, comma} |
| ⟨Acceso-Variable-Encadenado⟩ ::= brackets1<br><ExpOr> brackets2<br><Acceso-Variable-Encadenado3> |                                                               |                                                                                                                                                                                                                 |
| <Acceso-Variable-Encadenado3> ::= <Encadenado>                                                   | {dot, λ}                                                      | {op_mult, op_div, op_mod, op_pdiv, op_sum, op_sub, op_rel_less, op_rel_greater, op_rel_lessequal, op_rel_greaterequal, op_rel_equal, op_rel_notequal, op_and, op_or, parentheses2, brackets2, semicolon, comma} |
| <Acceso-Variable-Encadenado3> ::= λ                                                              |                                                               |                                                                                                                                                                                                                 |
| <Sentencia> ::= semicolon                                                                        | {semicolon,pif, pwhile, id, pself, ret, parenthesis1,braces1} | {pelse, semicolon,pif, pwhile, id, pself, ret, parenthesis1,braces1, braces2}                                                                                                                                   |
| <Sentencia> ::= <Asignación> semicolon                                                           |                                                               |                                                                                                                                                                                                                 |
| <Sentencia> ::= <Sentencia-Simple> semicolon                                                     |                                                               |                                                                                                                                                                                                                 |
| <Sentencia> ::= pif parentheses1 <ExpOr> parentheses2 <Sentencia> <Sentencialf>                  |                                                               |                                                                                                                                                                                                                 |



|                                                                            |                                                                                                                                                                                              |                                                                                  |
|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <Sentencia> ::= pwhile<br>parentheses1 <ExpOr><br>parentheses2 <Sentencia> |                                                                                                                                                                                              |                                                                                  |
| <Sentencia> ::= <Bloque>                                                   |                                                                                                                                                                                              |                                                                                  |
| <Sentencia> ::= ret<br><SentenciaRet>                                      |                                                                                                                                                                                              |                                                                                  |
| <SentenciaRet> ::=<br>semicolon                                            | {semicolon, pnil, ptrue,<br>pfalse, const_int,<br>const_string,<br>const_double,<br>parentheses1, pself, id,<br>pnew, id_class, op_sum,<br>op_sub, op_not,<br>op_increment,<br>op_decrement} | {pelse, semicolon,pif, pwhile, id, pself, ret,<br>parenthesis1,braces1, braces2} |
| <SentenciaRet> ::=<br><ExpOr> semicolon                                    |                                                                                                                                                                                              |                                                                                  |
| <Sentencialf> ::= pelse<br><Sentencia>                                     | {pelse, $\lambda$ }                                                                                                                                                                          | {pelse, semicolon,pif, pwhile, id, pself, ret,<br>parenthesis1,braces1, braces2} |
| <Sentencialf> ::= $\lambda$                                                |                                                                                                                                                                                              |                                                                                  |
| <Bloque> ::= braces1<br><Sentencias>                                       | {braces1}                                                                                                                                                                                    | {pelse, semicolon,pif, pwhile, id, pself, ret,<br>parenthesis1,braces1, braces2} |
| <Asignación> ::=<br><AccesoVar-Simple><br>op_equal <ExpOr>                 | {id, pself}                                                                                                                                                                                  | {semicolon}                                                                      |
| <Asignación> ::=<br><AccesoSelf-Simple><br>op_equal <ExpOr>                |                                                                                                                                                                                              |                                                                                  |
| <AccesoVar-Simple> ::= id<br><AccesoVar-Simple2>                           | {id}                                                                                                                                                                                         | {op_equal}                                                                       |
| <AccesoVar-Simple2> ::=<br><Encadenados-Simples>                           | {brackets1, dot, $\lambda$ }                                                                                                                                                                 | {op_equal}                                                                       |
| <AccesoVar-Simple2> ::=<br>brackets1 <ExpOr><br>brackets2                  |                                                                                                                                                                                              |                                                                                  |

|                                                                           |                                                                                                                                                           |                                                    |
|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| <Encadenados-Simples><br>::= <Encadenado-Simple><br><Encadenados-Simples> | {λ, dot}                                                                                                                                                  | {op_equal}                                         |
| <Encadenados-Simples><br>::= λ                                            |                                                                                                                                                           |                                                    |
| <AccesoSelf-Simple> ::= pself<br><Encadenados-Simples>                    | {pself}                                                                                                                                                   | {op_equal}                                         |
| <Encadenado-Simple> ::= dot id                                            | {dot}                                                                                                                                                     | {op_equal, dot}                                    |
| <Sentencia-Simple> ::= parentheses1 <ExpOr><br>parentheses2               | {parentheses1}                                                                                                                                            | {semicolon}                                        |
| <ExpOr> ::= <ExpAnd><br><ExpOr2>                                          | {pnil, ptrue, pfalse, const_int, const_string, const_double, parentheses1, pself, id, pnew, id_class, op_sum, op_sub, op_not, op_increment, op_decrement} | {parentheses2, brackets2, semicolon, comma}        |
| <ExpOr2> ::= op_or<br><ExpAnd> <ExpOr2>                                   | {op_or, λ}                                                                                                                                                | {parentheses2, brackets2, semicolon, comma}        |
| <ExpOr2> ::= λ                                                            |                                                                                                                                                           |                                                    |
| <ExpAnd> ::= <ExpIguual><br><ExpAnd2>                                     | {pnil, ptrue, pfalse, const_int, const_string, const_double, parentheses1, pself, id, pnew, id_class, op_sum, op_sub, op_not, op_increment, op_decrement} | {op_or, parentheses2, brackets2, semicolon, comma} |
| <ExpAnd2> ::= op_and<br><ExpIguual> <ExpAnd2>                             | {op_and, λ}                                                                                                                                               | {op_or, parentheses2, brackets2, semicolon, comma} |
| <ExpAnd2> ::= λ                                                           |                                                                                                                                                           |                                                    |

|                                                                                                                                                |                                                                                                                                                                                |                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\langle \text{ExpIgu} \rangle ::=$<br>$\langle \text{ExpCompuesta} \rangle$<br>$\langle \text{ExpIgu2} \rangle$                               | {pnil, ptrue, pfalse,<br>const_int, const_string,<br>const_double,<br>parentheses1, pself, id,<br>pnew, id_class, op_sum,<br>op_sub, op_not,<br>op_increment,<br>op_decrement} | {op_and, op_or, parentheses2, brackets2,<br>semicolon, comma}                                                                                                             |
| $\langle \text{ExpIgu2} \rangle ::= \langle \text{OpIgu} \rangle$<br>$\langle \text{ExpCompuesta} \rangle$<br>$\langle \text{ExpIgu2} \rangle$ | {op_rel_equal,<br>op_rel_notequal, $\lambda$ }                                                                                                                                 | {op_and, op_or, parentheses2, brackets2,<br>semicolon, comma}                                                                                                             |
| $\langle \text{ExpIgu2} \rangle ::= \lambda$                                                                                                   |                                                                                                                                                                                |                                                                                                                                                                           |
| $\langle \text{ExpCompuesta} \rangle ::=$<br>$\langle \text{ExpAd} \rangle$<br>$\langle \text{ExpCompuesta2} \rangle$                          | {pnil, ptrue, pfalse,<br>const_int, const_string,<br>const_double,<br>parentheses1, pself, id,<br>pnew, id_class, op_sum,<br>op_sub, op_not,<br>op_increment,<br>op_decrement} | {op_rel_equal, op_rel_notequal, op_and,<br>op_or, parentheses2, brackets2, semicolon,<br>comma}                                                                           |
| $\langle \text{ExpCompuesta2} \rangle ::=$<br>$\langle \text{OpCompuesto} \rangle$<br>$\langle \text{ExpAd} \rangle$                           | {op_rel_less,<br>op_rel_greater,<br>op_rel_lessequal,<br>op_rel_greaterequal, $\lambda$ }                                                                                      | {op_rel_equal, op_rel_notequal, op_and,<br>op_or, parentheses2, brackets2, semicolon,<br>comma}                                                                           |
| $\langle \text{ExpCompuesta2} \rangle ::= \lambda$                                                                                             |                                                                                                                                                                                |                                                                                                                                                                           |
| $\langle \text{ExpAd} \rangle ::= \langle \text{ExpMul} \rangle$<br>$\langle \text{ExpAd2} \rangle$                                            | {pnil, ptrue, pfalse,<br>const_int, const_string,<br>const_double,<br>parentheses1, pself, id,<br>pnew, id_class, op_sum,<br>op_sub, op_not,<br>op_increment,<br>op_decrement} | {op_rel_less, op_rel_greater,<br>op_rel_lessequal, op_rel_greaterequal,<br>op_rel_equal, op_rel_notequal, op_and,<br>op_or, parentheses2, brackets2, semicolon,<br>comma} |
| $\langle \text{ExpAd2} \rangle ::= \langle \text{OpAd} \rangle$<br>$\langle \text{ExpMul} \rangle \langle \text{ExpAd2} \rangle$               | {op_sum, op_sub, $\lambda$ }                                                                                                                                                   | {op_rel_less, op_rel_greater,<br>op_rel_lessequal, op_rel_greaterequal,<br>op_rel_equal, op_rel_notequal, op_and,<br>op_or, parentheses2, brackets2, semicolon,<br>comma} |

|                                             |                                                                                                                                                                                      |                                                                                                                                                                                                                             |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ExpAd2> ::= λ                              |                                                                                                                                                                                      |                                                                                                                                                                                                                             |
| <ExpMul> ::= <ExpUn><br><ExpMul2>           | {pnil, ptrue, pfalse,<br>const_int, const_string,<br>const_double,<br>parentheses1, pself, id,<br>pnew, id_class, op_sum,<br>op_sub, op_not,<br>op_increment,<br>op_decrement}       | {op_sum, op_sub, op_rel_less,<br>op_rel_greater, op_rel_lessequal,<br>op_rel_greaterequal, op_rel_equal,<br>op_rel_notequal, op_and, op_or,<br>parentheses2, brackets2, semicolon,<br>comma}                                |
| <ExpMul2> ::= <OpMul><br><ExpUn> <ExpMul2>  | {op_mult, op_div, op_mod,<br>pdiv, λ}                                                                                                                                                | {op_sum, op_sub, op_rel_less,<br>op_rel_greater, op_rel_lessequal,<br>op_rel_greaterequal, op_rel_equal,<br>op_rel_notequal, op_and, op_or,<br>parentheses2, brackets2, semicolon,<br>comma}                                |
| <ExpMul2> ::= λ                             |                                                                                                                                                                                      |                                                                                                                                                                                                                             |
| <ExpUn> ::= <OpUnario><br><ExpUn>           | {pnil, ptrue, pfalse,<br>const_int, const_string,<br>const_double,<br>parentheses1, pself, id,<br>pnew, id_class, op_sum,<br>op_sub, op_not,<br>op_increment,<br>op_decrement}       | {op_mult, op_div, op_mod, pdiv, op_sum,<br>op_sub, op_rel_less, op_rel_greater,<br>op_rel_lessequal, op_rel_greaterequal,<br>op_rel_equal, op_rel_notequal, op_and,<br>op_or, parentheses2, brackets2, semicolon,<br>comma} |
| <ExpUn> ::= <Operando>                      |                                                                                                                                                                                      |                                                                                                                                                                                                                             |
| <ExpUn> ::= parenthesis1<br><ExpUnAux>      |                                                                                                                                                                                      |                                                                                                                                                                                                                             |
| <ExpUnAux> ::= pint<br>parentheses2 <ExpUn> | {pint, pnil, ptrue, pfalse,<br>const_int, const_string,<br>const_double,<br>parentheses1, pself, id,<br>pnew, id_class, op_sum,<br>op_sub, op_not,<br>op_increment,<br>op_decrement} | {op_mult, op_div, op_mod, pdiv, op_sum,<br>op_sub, op_rel_less, op_rel_greater,<br>op_rel_lessequal, op_rel_greaterequal,<br>op_rel_equal, op_rel_notequal, op_and,<br>op_or, parentheses2, brackets2, semicolon,<br>comma} |
| <ExpUnAux> ::=<br><ExpresionParentizada2>   |                                                                                                                                                                                      |                                                                                                                                                                                                                             |

|                                          |                                                                               |                                                                                                                                                                    |
|------------------------------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <OpIqual> ::=<br>op_rel_equal            | {op_rel_equal,<br>op_rel_notequal}                                            | {pnil, ptrue, pfalse, const_int, const_string,<br>const_double, parentheses1, pself, id, pnew,<br>id_class, op_sum, op_sub, op_not,<br>op_increment, op_decrement} |
| <OpIqual> ::=<br>op_rel_notequal         |                                                                               |                                                                                                                                                                    |
| <OpCompuesto> ::=<br>op_rel_less         | {op_rel_less,<br>op_rel_greater,<br>op_rel_lessequal,<br>op_rel_greaterequal} | {pnil, ptrue, pfalse, const_int, const_string,<br>const_double, parentheses1, pself, id, pnew,<br>id_class, op_sum, op_sub, op_not,<br>op_increment, op_decrement} |
| <OpCompuesto> ::=<br>op_rel_greater      |                                                                               |                                                                                                                                                                    |
| <OpCompuesto> ::=<br>op_rel_lessequal    |                                                                               |                                                                                                                                                                    |
| <OpCompuesto> ::=<br>op_rel_greaterequal |                                                                               |                                                                                                                                                                    |
| <OpAd> ::= op_sum                        | {op_sum, op_sub}                                                              | {pnil, ptrue, pfalse, const_int, const_string,<br>const_double, parentheses1, pself, id, pnew,<br>id_class, op_sum, op_sub, op_not,<br>op_increment, op_decrement} |
| <OpAd> ::= op_sub                        |                                                                               |                                                                                                                                                                    |
| <OpUnario> ::= op_sum                    | {op_sum, op_sub, op_not,<br>op_increment,<br>op_decrement}                    | {pnil, ptrue, pfalse, const_int, const_string,<br>const_double, parentheses1, pself, id, pnew,<br>id_class, op_sum, op_sub, op_not,<br>op_increment, op_decrement} |
| <OpUnario> ::= op_sub                    |                                                                               |                                                                                                                                                                    |
| <OpUnario> ::= op_not                    |                                                                               |                                                                                                                                                                    |
| <OpUnario> ::=<br>op_increment           |                                                                               |                                                                                                                                                                    |
| <OpUnario> ::=<br>op_decrement           |                                                                               |                                                                                                                                                                    |
| <OpMul> ::= op_mult                      | {op_mult, op_div, op_mod,<br>pdiv}                                            | {pnil, ptrue, pfalse, const_int, const_string,<br>const_double, parentheses1, pself, id, pnew,<br>id_class, op_sum, op_sub, op_not,<br>op_increment, op_decrement} |

|                                          |                                                                                         |                                                                                                                                                                                                                 |
|------------------------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <OpMul> ::= op_div                       |                                                                                         |                                                                                                                                                                                                                 |
| <OpMul> ::= op_mod                       |                                                                                         |                                                                                                                                                                                                                 |
| <OpMul> ::= pdiv                         |                                                                                         |                                                                                                                                                                                                                 |
| <Operando> ::= <Literal>                 | {pnil, ptrue, pfalse, const_int, const_string, const_double, pself, id, pnew, id_class} | {op_mult, op_div, op_mod, op_pdiv, op_sum, op_sub, op_rel_less, op_rel_greater, op_rel_lessequal, op_rel_greaterequal, op_rel_equal, op_rel_notequal, op_and, op_or, parentheses2, brackets2, semicolon, comma} |
| <Operando> ::= <Primario>                |                                                                                         |                                                                                                                                                                                                                 |
| <Literal> ::= pnil                       | {pnil, ptrue, pfalse, const_int, const_string, const_double}                            | {op_mult, op_div, op_mod, op_pdiv, op_sum, op_sub, op_rel_less, op_rel_greater, op_rel_lessequal, op_rel_greaterequal, op_rel_equal, op_rel_notequal, op_and, op_or, parentheses2, brackets2, semicolon, comma} |
| <Literal> ::= ptrue                      |                                                                                         |                                                                                                                                                                                                                 |
| <Literal> ::= pfalse                     |                                                                                         |                                                                                                                                                                                                                 |
| <Literal> ::= const_int                  |                                                                                         |                                                                                                                                                                                                                 |
| <Literal> ::= const_string               |                                                                                         |                                                                                                                                                                                                                 |
| <Literal> ::= const_double               |                                                                                         |                                                                                                                                                                                                                 |
| <Primario> ::= <AccesoSelf>              | {pself, id, pnew, id_class}                                                             | {op_mult, op_div, op_mod, op_pdiv, op_sum, op_sub, op_rel_less, op_rel_greater, op_rel_lessequal, op_rel_greaterequal, op_rel_equal, op_rel_notequal, op_and, op_or, parentheses2, brackets2, semicolon, comma} |
| <Primario> ::= id<br><Primario2>         |                                                                                         |                                                                                                                                                                                                                 |
| <Primario> ::= <Llamada-Método-Estático> |                                                                                         |                                                                                                                                                                                                                 |
| <Primario> ::= <Llamada-Conclassor>      |                                                                                         |                                                                                                                                                                                                                 |

|                                                         |                                      |                                                                                                                                                                                                                                |
|---------------------------------------------------------|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <Primario2> ::=<br><Argumentos-Actuales><br><Primario3> | {parenthesis1, dot, λ,<br>brackets1} | {op_mult, op_div, op_mod, op_pdiv, op_sum,<br>op_sub, op_rel_less, op_rel_greater,<br>op_rel_lessequal, op_rel_greaterequal,<br>op_rel_equal, op_rel_notequal, op_and,<br>op_or, parentheses2, brackets2, semicolon,<br>comma} |
| <Primario2> ::=<br><AccesoVar2>                         |                                      |                                                                                                                                                                                                                                |
| <Primario3> ::=<br><Encadenado>                         | {dot, λ}                             | {op_mult, op_div, op_mod, op_pdiv, op_sum,<br>op_sub, op_rel_less, op_rel_greater,<br>op_rel_lessequal, op_rel_greaterequal,<br>op_rel_equal, op_rel_notequal, op_and,<br>op_or, parentheses2, brackets2, semicolon,<br>comma} |
| <Primario3> ::= λ                                       |                                      |                                                                                                                                                                                                                                |

#### 2.4.7. Gramática LL(1)

A pesar de las transformaciones que se realizaron para quitar la ambigüedad de la gramática, el conflicto introducido por el if-else hace que la misma sea inherentemente ambigua, por lo tanto, no LL(1). Sin embargo, al haber resuelto ese conflicto eligiendo reducir en vez de desplazar, la misma se pudo analizar con un analizador sintáctico descendente predictivo recursivo.

#### 2.4.8. Excepciones

Las excepciones en esta etapa se producen cuando la estructura del programa no respeta la gramática del lenguaje TinyS, por lo que las mismas ocurren en ciertas partes del código en las que se esperaban ciertos símbolos, identificadores o palabras claves según la gramática, pero se encuentran otras cosas.

Las excepciones en todos los casos se reportan con el siguiente formato:

ERROR: SINTÁCTICO | NUMERO DE LINEA (NUMERO DE COLUMNA) |  
 DESCRIPCION: | LINEA X (COLUMNA Y) | mensaje |

#### 2.4.9. Casos de testeo

Tabla 3. Casos de testeo del análisis sintáctico.

| Escenario          | Nombre Test                      | Salida Esperada                                                                                                                                                                                                                                             |
|--------------------|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Doble start        | test1.s                          | ERROR:<br>SINTÁCTICO<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 5<br>(COLUMNA 1)   Se<br>esperaba '{', '(', '}',<br>'ret', 'self', 'while', 'if',<br>';' o identificador de<br>método o variable.<br>Se encontró: pstart |
| Clase no declarada | test2.s<br>Similares:<br>test5.s | ERROR:<br>SEMÁNTICO -<br>DECLARACIONES<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 2<br>(COLUMNA 1)   El<br>tipo 'Fibonacci' de la<br>variable fib no se<br>encuentra declarado.<br>                                      |



|                                |                                   |                                                                                                                                                                                                             |
|--------------------------------|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Programa sin método start      | test3.                            | ERROR:<br>SINTÁCTICO<br>  NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  <br>  LINEA 4 (COLUMNA 2)   Se esperaba 'class', 'impl' o 'start'. Se encontró: end_of_file                                   |
| Asignación dentro de una clase | test4.s                           | ERROR:<br>SINTÁCTICO<br>  NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  <br>  LÍNEA 4 (COLUMNA 1)   Se esperaba pub, str, bool, int, double, array, identificador de clase o '}'. Se encontró: id_obj |
| Fibonacci                      | test6.s                           | CORRECTO:<br>ANÁLISIS SINTÁCTICO                                                                                                                                                                            |
| Paréntesis sin cerrar          | test7.s<br>Similares:<br>test25.s | ERROR:<br>SINTÁCTICO<br>  NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:                                                                                                                                |

|                                     |          |                                                                                                                                                                                                                                                                                 |
|-------------------------------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                     |          | LÍNEA 21<br>(COLUMNA 24)   Se<br>esperaba<br>parentheses2. Se<br>encontró semicolon                                                                                                                                                                                             |
| Punto y coma<br>faltante            | test8.s  | ERROR:<br>SINTÁCTICO<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 5<br>(COLUMNA 1)   Se<br>esperaba '*', '/', '%',<br>'div', '+', '-', '<', '>',<br>'<=', '>=', '==', '!=',<br>'&&', '  ', ')', ']', ':', ',',<br>'.'. Se encontró:<br>braces2 |
| Acceso encadenado<br>a un método    | test9.s  | CORRECTO:<br>ANÁLISIS<br>SINTÁCTICO                                                                                                                                                                                                                                             |
| Acceso encadenado<br>a una variable | test10.s | CORRECTO:<br>ANÁLISIS<br>SINTÁCTICO                                                                                                                                                                                                                                             |
| Acceso a una clase                  | test11.s | ERROR:<br>SINTÁCTICO<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 4<br>(COLUMNA 61)   Se                                                                                                                                                       |

|                                                |                                    |                                                                                                                                                                                                                                        |
|------------------------------------------------|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                |                                    | esperaba<br>identificador de<br>método o variable.<br>Se encontró:<br>id_class                                                                                                                                                         |
| Identificador de clase<br>incorrecto           | test12.s                           | ERROR:<br>SINTÁCTICO<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LINEA 1<br>(COLUMNA 6)   Se<br>esperaba<br>identificador, IO,<br>object. Se encontró:<br>const_int                                        |
| Llave de cierre<br>faltante                    | test15.s                           | ERROR:<br>SINTÁCTICO<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LINEA 3<br>(COLUMNA 0)   Se<br>esperaba pub, str,<br>bool, int, double,<br>array, identificador<br>de clase o '}'. Se<br>encontró: pstart |
| Declaración de<br>variables en método<br>start | test16.s<br>Similares:<br>test18.s | CORRECTO:<br>ANÁLISIS<br>SINTÁCTICO                                                                                                                                                                                                    |

|                                    |          |                                                                                                                                                                                                                                                                                                 |
|------------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Clase en lugar de objeto           | test17.s | ERROR:<br>SINTÁCTICO<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 2<br>(COLUMNA 8)   Se<br>esperaba id_obj. Se<br>encontró id_class                                                                                                                            |
| Creación de array<br>sin corchetes | test19.s | ERROR:<br>SINTÁCTICO<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 3<br>(COLUMNA 16)   Se<br>esperaba brackets1.<br>Se encontró<br>const_int                                                                                                                    |
| Operador ++ del lado<br>derecho    | test20.s | ERROR:<br>SINTÁCTICO<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 5<br>(COLUMNA 10)   Se<br>esperaba '*', '/', '%',<br>'div', '+', '-', '<', '>',<br>'<=', '>=', '==', '!=',<br>'&&', '  ', ')', ']', ';', ',',<br>'.', '[', '('. Se encontró:<br>op_increment |

|                                                        |          |                                                                                                                                                                                                                                                           |
|--------------------------------------------------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Operación matemática                                   | test23.s | CORRECTO:<br>ANÁLISIS<br>SINTÁCTICO                                                                                                                                                                                                                       |
| Operación booleana                                     | test24.s | CORRECTO:<br>ANÁLISIS<br>SINTÁCTICO                                                                                                                                                                                                                       |
| Comentarios simples y multilínea                       | test26.s | CORRECTO:<br>ANÁLISIS<br>SINTÁCTICO                                                                                                                                                                                                                       |
| Declaración de método                                  | test27.s | CORRECTO:<br>ANÁLISIS<br>SINTÁCTICO                                                                                                                                                                                                                       |
| While posee un () adicional después de la expresión.   | test35.s | ERROR:<br>SINTÁCTICO<br>  NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  <br>  LINEA 4 (COLUMNA 18)   Se esperaba 'nil', 'true', 'false', '(', 'self', 'new', '+', '-', '!', '++', '--', una constante o un identificador. Se encontró: parentheses2 |
| Encadenado de métodos y variables, array y asignación. | test44.s | CORRECTO:<br>ANÁLISIS<br>SINTÁCTICO                                                                                                                                                                                                                       |
| Herencia.                                              | test46.s | CORRECTO:                                                                                                                                                                                                                                                 |

|                                 |                                                             |                                                                                                                                                                                                                   |
|---------------------------------|-------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                 | Similares:<br>test14.s<br>test52.s<br>test13.s (incorrecto) | ANÁLISIS<br>SINTÁCTICO                                                                                                                                                                                            |
| Falta clase de la que hereda A. | test47.s                                                    | ERROR:<br>SINTÁCTICO<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LINEA 1<br>(COLUMNA 11)   Se<br>esperaba array,<br>identificador, str,<br>bool, int, double. Se<br>encontró: braces1 |
| Clases, ret, herencia, métodos  | test48.s                                                    | CORRECTO:<br>ANÁLISIS<br>SINTÁCTICO                                                                                                                                                                               |
| Start al principio.             | test49.s                                                    | ERROR:<br>SINTÁCTICO<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 5<br>(COLUMNA 0)   Se<br>esperaba<br>end_of_file. Se<br>encontró pclass                                        |
| Creación de arrays              | test50.s                                                    | CORRECTO:                                                                                                                                                                                                         |

|                                                          |                                                |                                                                                                                                                         |
|----------------------------------------------------------|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                          |                                                | ANÁLISIS SINTÁCTICO                                                                                                                                     |
| Casteo en new Int                                        | test53.s                                       | CORRECTO:<br>ANÁLISIS SINTÁCTICO                                                                                                                        |
| Clases, llamadas estáticas, constructores, self, arrays. | test54.s<br>Similares:<br>test51.s<br>test55.s | CORRECTO:<br>ANÁLISIS SINTÁCTICO.                                                                                                                       |
| Llamada a constructor sin ().                            | test56.s                                       | ERROR:<br>SINTÁCTICO<br>  NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  <br>  LÍNEA 10 (COLUMNA 13)   Se esperaba '('. Se encontró: semicolon<br> |
| Start con argumentos.                                    | test57.s                                       | ERROR:<br>SINTÁCTICO<br>  NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  <br>  LÍNEA 1 (COLUMNA 5)   Se esperaba '{'. Se encontró:<br>parentheses1 |

|                                              |                                                             |                                     |
|----------------------------------------------|-------------------------------------------------------------|-------------------------------------|
| If-else anidado sin llaves.                  | test58.s<br>Similares:<br>test21.s<br>test22.s (incorrecto) | CORRECTO:<br>ANÁLISIS<br>SINTÁCTICO |
| Clases, ret, expresión, encadenado y casteo. | test59.s                                                    | CORRECTO:<br>ANÁLISIS<br>SINTÁCTICO |

## 2.5. Análisis Semántico - Chequeo de declaraciones

### 2.5.1. Estructura y diagrama de clases de la tabla de símbolos

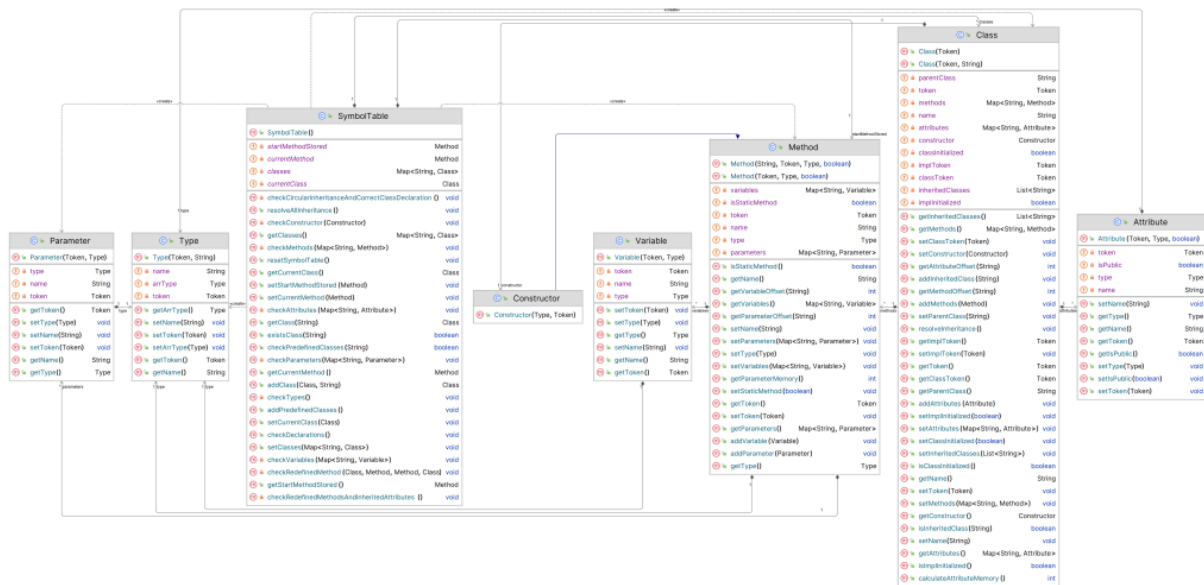


Figura 3. Diagrama de clases de la tabla de símbolos.

En el siguiente enlace se encuentra la misma Figura 3 con mayor resolución o el mismo se puede observar en el anexo 1.

<https://drive.google.com/file/d/19XdWEdkE0WNgetJmiufhEPZzBvS6xqx5/view?usp=sharing>

### 2.5.2. Esquema de traducción

⟨Program⟩ ::= ⟨Lista-Definiciones⟩ ⟨Start⟩



$\langle \text{Start} \rangle ::= \{ \text{Method method} = \text{new Method}(\text{lookahead}, \text{null}, \text{false});$   
 $\text{SymbolTable.setCurrentMethod}(\text{method}); \}$  pstart  $\langle \text{Bloque-Metodo} \rangle$

$\langle \text{Lista-Definiciones} \rangle ::= \langle \text{Class} \rangle \langle \text{Lista-Definiciones} \rangle$   
 $\langle \text{Lista-Definiciones} \rangle ::= \langle \text{Impl} \rangle \langle \text{Lista-Definiciones} \rangle$   
 $\langle \text{Lista-Definiciones} \rangle ::= \lambda$

$\langle \text{Class} \rangle ::= \text{pclass } \{ \text{Class class1} = \text{new Class}(\text{lookahead});$   
 $\text{SymbolTable.addClass}(\text{class1}, \text{"class"});$   
 $\text{SymbolTable.setCurrentClass}(\text{class1}); \}$  id\_class  $\langle \text{Class2} \rangle$

$\langle \text{Class2} \rangle ::= \langle \text{Herencia} \rangle \text{braces1 } \langle \text{Atributos} \rangle \text{braces2}$   
 $\langle \text{Class2} \rangle ::= \text{braces1 } \langle \text{Atributos} \rangle \text{braces2}$

$\langle \text{Atributos} \rangle ::= \langle \text{Atributo} \rangle \langle \text{Atributos} \rangle$   
 $\langle \text{Atributos} \rangle ::= \lambda$

$\langle \text{Impl} \rangle ::= \text{pimpl } \{ \text{Class class1} = \text{new Class}(\text{lookahead});$   
 $\text{SymbolTable.addClass}(\text{class1}, \text{"impl"});$   
 $\text{SymbolTable.setCurrentClass}(\text{class1}); \}$  id\_class braces1  $\langle \text{Miembro} \rangle$

$\langle \text{Miembros} \rangle \text{braces2}$   
 $\langle \text{Miembros} \rangle ::= \langle \text{Miembro} \rangle \langle \text{Miembros} \rangle$   
 $\langle \text{Miembros} \rangle ::= \lambda$

$\langle \text{Herencia} \rangle ::= \text{colon}$   
 $\{ \text{SymbolTable.currentClass.setParentClass}(\text{lookahead.getLexeme}()); \}$   $\langle \text{Tipo} \rangle$

$\langle \text{Miembro} \rangle ::= \langle \text{Metodo} \rangle$   
 $\langle \text{Miembro} \rangle ::= \langle \text{Constructor} \rangle$

$\langle \text{Constructor} \rangle ::= \{ \text{Type type} = \text{new}$   
 $\text{Type}(\text{SymbolTable.getCurrentClass.getToken}(), \text{"class"});$   
 $\text{Constructor constructor} = \text{new Constructor}(\text{type}, \text{lookahead});$   
 $\text{SymbolTable.setCurrentMethod}(\text{constructor});$

`SymbolTable.getCurrentClass.setConstructor(constructor);}` dot

⟨Argumentos-Formales⟩ ⟨Bloque-Metodo⟩

⟨Atributo⟩ ::= ⟨Visibilidad⟩ ⟨Tipo⟩ `{Type type = tipo();`

`listaDeclaracionVariables(type, true, "attribute")}`

⟨Lista-Declaracion-Variables⟩ `}` semicolon

⟨Atributo⟩ ::= ⟨Tipo⟩ `{Type type = tipo(); listaDeclaracionVariables(type, false,`  
`"attribute")}` ⟨Lista-Declaracion-Variables⟩ semicolon

⟨Metodo⟩ ::= ⟨Forma-Metodo⟩ pfn `{metodo2(true);}` <Metodo2>

⟨Metodo⟩ ::= pfn `{metodo2(false);}` <Metodo2>

<Metodo2> ::= ⟨Tipo-Metodo⟩ `{Type type = tipoMetodo();`

`Method method = new Method(lookahead, type, isStatic);`

`SymbolTable.setCurrentMethod(method);`

`SymbolTable.getCurrentClass().addMethods(method); }` id\_obj

⟨Argumentos-Formales⟩ ⟨Bloque-Metodo⟩

<Metodo2> ::= `{Type type = new Type(lookahead, "void");`

`Method method = new Method(lookahead, type, isStatic);`

`SymbolTable.setCurrentMethod(method);`

`SymbolTable.getCurrentClass().addMethods(method); }` id\_obj

⟨Argumentos-Formales⟩ ⟨Bloque-Metodo⟩

⟨Visibilidad⟩ ::= ppub

⟨Forma-Metodo⟩ ::= pst

⟨Bloque-Metodo⟩ ::= braces1 ⟨Mas-Decl-Var-Locales⟩ ⟨Sentencias⟩

<Sentencias> ::= <Sentencia> <Sentencias>

<Sentencias> ::= braces2

⟨Mas-Decl-Var-Locales⟩ ::= ⟨Decl-Var-Locales⟩ ⟨Mas-Decl-Var-Locales⟩

⟨Mas-Decl-Var-Locales⟩ ::=  $\lambda$

```

<Decl-Var-Locales> ::= <Tipo> {Type type = tipo();
listaDeclaracionVariables(type, false, "variable");}
<Lista-Declaracion-Variables> semicolon

```

```

<Lista-Declaracion-Variables> ::= {if (option.equals("attribute") {Attribute
attribute = new Attribute(lookahead, type, visibilidad);
SymbolTable.getCurrentClass().addAttributes(attribute)}
else {
Variable variable = new Variable(lookahead, type);
SymbolTable.getCurrentMethod().addVariable(variable) }
id_obj <Lista-Declaracion-Variables2>

```

```

<Lista-Declaracion-Variables2> ::= comma <Lista-Declaracion-Variables>
<Lista-Declaracion-Variables2> ::= λ

```

```

<Argumentos-Formales> ::= parentheses1 <Argumentos-Formales2>

```

```

<Argumentos-Formales2> ::= <Lista-Argumentos-Formales> parentheses2
<Argumentos-Formales2> ::= parentheses2

```

```

<Lista-Argumentos-Formales> ::= <Argumento-Formal>
<Lista-Argumentos-Formales2>
<Lista-Argumentos-Formales2> ::= comma <Lista-Argumentos-Formales>
<Lista-Argumentos-Formales2> ::= λ

```

```

<Argumento-Formal> ::= <Tipo> {Type type = tipo();
Parameter parameter = new Parameter(lookahead, type);
SymbolTable.getCurrentMethod().addParameter(parameter);} id_obj

```

```

<Tipo-Metodo> ::= <Tipo> {return tipo();}
<Tipo-Metodo> ::= pvoid {return new Type(lookaheadAux, "void");}

```

```

<Tipo> ::= <Tipo-Primitivo> {return tipoPrimitivo();}
<Tipo> ::= {Type type = new Type(lookahead, "Array");} parray <Tipo-Primitivo>
{Type tipoPrimitivoType = tipoPrimitivo();
type.setArrayType(tipoPrimitivoType);}

```

```

<Tipo> ::= id_class {return new Type(lookaheadAux, "class");}

<Tipo-Primitivo> ::= pstr {return new Type(lookaheadAux, "Str");}
<Tipo-Primitivo> ::= pbool {return new Type(lookaheadAux, "Bool");}
<Tipo-Primitivo> ::= pint {return new Type(lookaheadAux, "Int");}
<Tipo-Primitivo> ::= pdouble {return new Type(lookaheadAux, "Double");}

<ExpresionParentizada2> ::= <ExpOr> parentheses2
<ExpresionParentizada3>
<ExpresionParentizada3> ::= <Encadenado>
<ExpresionParentizada3> ::= λ

<AccesoSelf> ::= pself <AccesoSelf2>
<AccesoSelf2> ::= <Encadenado>
<AccesoSelf2> ::= λ
<AccesoVar2> ::= <AccesoVar3>
<AccesoVar2> ::= brackets1 <ExpOr> brackets2 <AccesoVar3>
<AccesoVar3> ::= <Encadenado>
<AccesoVar3> ::= λ

<Llamada-Metodo> ::= id <Argumentos-Actuales> <Llamada-Metodo2>
<Llamada-Metodo2> ::= <Encadenado>
<Llamada-Metodo2> ::= λ
<Llamada-Metodo-Estatico> ::= id_class dot <Llamada-Metodo>

<Llamada-Conclassor> ::= pnw <Llamada-Conclassor2>

<Llamada-Conclassor2> ::= id_class <Argumentos-Actuales>
<Llamada-Conclassor3>

<Llamada-Conclassor2> ::= <Tipo-Primitivo> brackets1 <ExpOr> brackets2
<Llamada-Conclassor3>

<Llamada-Conclassor3> ::= <Encadenado>
<Llamada-Conclassor3> ::= λ

<Argumentos-Actuales> ::= parentheses1 <Argumentos-Actuales2>

```

<Argumentos-Actuales2> ::= <Lista-Expresiones> parentheses2

<Argumentos-Actuales2> ::= parentheses2

<Lista-Expresiones> ::= <ExpOr> <Lista-Expresiones2>

<Lista-Expresiones2> ::= comma <Lista-Expresiones>

<Lista-Expresiones2> ::=  $\lambda$

<Encadenado> ::= dot id <Encadenado2>

<Encadenado2> ::= <Llamada-Metodo-Encadenado>

<Encadenado2> ::= <Acceso-Variable-Encadenado>

<Llamada-Metodo-Encadenado> ::= <Argumentos-Actuales>

<Llamada-Metodo-Encadenado2>

<Llamada-Metodo-Encadenado2> ::= <Encadenado>

<Llamada-Metodo-Encadenado2> ::=  $\lambda$

<Acceso-Variable-Encadenado> ::= <Acceso-Variable-Encadenado3>

<Acceso-Variable-Encadenado> ::= brackets1 <ExpOr> brackets2

<Acceso-Variable-Encadenado3>

<Acceso-Variable-Encadenado3> ::= <Encadenado>

<Acceso-Variable-Encadenado3> ::=  $\lambda$

<Sentencia> ::= semicolon

<Sentencia> ::= <Asignación> semicolon

<Sentencia> ::= <Sentencia-Simple> semicolon

<Sentencia> ::= pif parentheses1 <ExpOr> parentheses2 <Sentencia>

<Sentencialf>

<Sentencia> ::= pwhile parentheses1 <ExpOr> parentheses2 <Sentencia>

<Sentencia> ::= <Bloque>

<Sentencia> ::= ret <SentenciaRet>

<SentenciaRet> ::= semicolon

<SentenciaRet> ::= <ExpOr> semicolon

<Sentencialf> ::= pelse <Sentencia>

<Sentencialf> ::=  $\lambda$

<Bloque> ::= braces1 <Sentencias> pif

<Asignación> ::= <AccesoVar-Simple> op\_equal <ExpOr>

<Asignación> ::= <AccesoSelf-Simple> op\_equal <ExpOr>

<AccesoVar-Simple> ::= id <AccesoVar-Simple2>

<AccesoVar-Simple2> ::= <Encadenados-Simples>

<AccesoVar-Simple2> ::= brackets1 <ExpOr> brackets2

<Encadenados-Simples> ::= <Encadenado-Simple> <Encadenados-Simples>

<Encadenados-Simples> ::=  $\lambda$

<AccesoSelf-Simple> ::= pself <Encadenados-Simples>

<Encadenado-Simple> ::= dot id

<Sentencia-Simple> ::= parentheses1 <ExpOr> parentheses2

<ExpOr> ::= <ExpAnd> <ExpOr2>

<ExpOr2> ::= op\_

or <ExpAnd> <ExpOr2>

<ExpOr2> ::=  $\lambda$

<ExpAnd> ::= <Explgual> <ExpAnd2>

<ExpAnd2> ::= op\_and <Explgual> <ExpAnd2>

<ExpAnd2> ::=  $\lambda$

<Explgual> ::= <ExpCompuesta> <Explgual2>

<Explgual2> ::= <Oplgual> <ExpCompuesta> <Explgual2>

<Explgual2> ::=  $\lambda$

$\langle \text{ExpCompuesta} \rangle ::= \langle \text{ExpAd} \rangle \langle \text{ExpCompuesta2} \rangle$

$\langle \text{ExpCompuesta2} \rangle ::= \langle \text{OpCompuesto} \rangle \langle \text{ExpAd} \rangle$

$\langle \text{ExpCompuesta2} \rangle ::= \lambda$

$\langle \text{ExpAd} \rangle ::= \langle \text{ExpMul} \rangle \langle \text{ExpAd2} \rangle$

$\langle \text{ExpAd2} \rangle ::= \langle \text{OpAd} \rangle \langle \text{ExpMul} \rangle \langle \text{ExpAd2} \rangle$

$\langle \text{ExpAd2} \rangle ::= \lambda$

$\langle \text{ExpMul} \rangle ::= \langle \text{ExpUn} \rangle \langle \text{ExpMul2} \rangle$

$\langle \text{ExpMul2} \rangle ::= \langle \text{OpMul} \rangle \langle \text{ExpUn} \rangle \langle \text{ExpMul2} \rangle$

$\langle \text{ExpMul2} \rangle ::= \lambda$

$\langle \text{ExpUn} \rangle ::= \langle \text{OpUnario} \rangle \langle \text{ExpUn} \rangle$

$\langle \text{ExpUn} \rangle ::= \langle \text{Operando} \rangle$

$\langle \text{ExpUn} \rangle ::= \text{parenthesis1} \langle \text{ExpUnAux} \rangle$

$\langle \text{ExpUnAux} \rangle ::= \text{pint parentheses2} \langle \text{ExpUn} \rangle$

$\langle \text{ExpUnAux} \rangle ::= \langle \text{ExpresionParentizada2} \rangle$

$\langle \text{OpIguar} \rangle ::= \text{op\_rel\_equal}$

$\langle \text{OpIguar} \rangle ::= \text{op\_rel\_notequal}$

$\langle \text{OpCompuesto} \rangle ::= \text{op\_rel\_less}$

$\langle \text{OpCompuesto} \rangle ::= \text{op\_rel\_greater}$

$\langle \text{OpCompuesto} \rangle ::= \text{op\_rel\_lessequal}$

$\langle \text{OpCompuesto} \rangle ::= \text{op\_rel\_greaterequal}$

$\langle \text{OpAd} \rangle ::= \text{op\_sum}$

$\langle \text{OpAd} \rangle ::= \text{op\_sub}$

$\langle \text{OpUnario} \rangle ::= \text{op\_sum}$

$\langle \text{OpUnario} \rangle ::= \text{op\_sub}$

$\langle \text{OpUnario} \rangle ::= \text{op\_not}$

```

<OpUnario> ::= op_increment
<OpUnario> ::= op_decrement

<OpMul> ::= op_mult
<OpMul> ::= op_div
<OpMul> ::= op_mod
<OpMul> ::= pdiv

<Operando> ::= <Literal>
<Operando> ::= <Primario>

<Literal> ::= pnil
<Literal> ::= ptrue
<Literal> ::= pfalse
<Literal> ::= const_int
<Literal> ::= const_string
<Literal> ::= const_double

<Primario> ::= <AccesoSelf>
<Primario> ::= id <Primario2>
<Primario> ::= <Llamada-Método-Estático>
<Primario> ::= <Llamada-Conclassor>

<Primario2> ::= <Argumentos-Actuales> <Primario3>
<Primario2> ::= <AccesoVar2>

<Primario3> ::= <Encadenado>
<Primario3> ::= λ

```

### 2.5.3. Ejemplo de JSON

Se elaboró un ejemplo de la tabla de símbolos en formato JSON para el siguiente código (Figura 4). El mismo posee una lista de clases identificadas con su nombre, cada clase almacena su información semántica, como el nombre, la clase padre, las clases heredadas, los atributos, los métodos y el constructor, junto con los tokens asociados a su definición. A su vez, cada método contiene su



tipo de retorno, los parámetros formales y las variables locales, mientras que los atributos, parámetros y variables detallan su nombre, tipo y token correspondiente.

```
1 //Imprime 9.4, 9.4 y 3.8
2 class A {
3     pub Int a;
4     pub Double b;
5 }
6 class B : A {
7     pub Double c;
8 }
9
10 impl A {
11     .() {}
12 }
13 impl B {
14     .(Double c, Double b) {
15         self.c = c;
16         self.b = b;
17     }
18
19     fn hola(Double c) {
20         (IO.out_double(c));
21     }
22 }
23
24 start {
25     B b;
26     Double d;
27     b = new B(9.4, 3.8);
28     b.a = 2;
29     d = b.c - b.b;
30     (b.hola(9.4));
31     (IO.out_str("\n"));
32     (IO.out_double(b.c));
33     (IO.out_str("\n"));
34     (IO.out_double(b.b));
35 }
```

Figura 4. Ejemplo de código en TinyS.

El JSON resultante tiene la siguiente estructura:

```

{
  "classes": {
    "Str": {
      "name": "Str",
      "token": {
        "name": "pstr",
        "lexeme": "Str",
        "valor": null,
        "column": 0,
        "row": 0
      },
      "parentClass": "Object",
      "inheritedClasses": ["Object"],
      "attributes": {},
      "methods": {
        "length": {
          "name": "length",
          "type": {
            "name": "Int",
            "arrType": null,
            "token": {
              "name": "id_obj",
              "lexeme": "length",
              "valor": null,
              "column": 0,
              "row": 0
            }
          },
          "token": {
            "name": "id_obj",
            "lexeme": "length",
            "valor": null,
            "column": 0,
            "row": 0
          },
          "isStaticMethod": false,
          "variables": {},
          "parameters": {}
        },
        "concat": {
          "name": "concat",
          "type": {
            "name": "Str",
            "arrType": null,
            "token": {
              "name": "id_obj",
              "lexeme": "concat",

```

Figura 5. JSON de la tabla de símbolos.

El JSON completo se puede observar en el siguiente enlace o en el anexo 2:

[https://drive.google.com/file/d/1MzAUzz7xRf55Na\\_TeYfoD\\_OjW5Qxml/TU/view?usp=sharing](https://drive.google.com/file/d/1MzAUzz7xRf55Na_TeYfoD_OjW5Qxml/TU/view?usp=sharing)

#### 2.5.4. Errores semánticos detectados

El chequeo de declaraciones se realizó en distintas etapas:

En primer lugar, se chequeó la herencia circular, y junto a ello la correcta declaración de las clases. En este chequeo, se recorrieron las clases declaradas y para cada una de ellas se verificó que contenga implementación, definición y constructor declarados. Además, se verifica que no hereden de tipos primitivos. Luego, se itera sobre los ancestros de cada clase, verificando la existencia de los mismos y la herencia circular. Los errores encontrados en esta etapa son:

- Clase no declarada.
- Implementación no declarada.
- Clase sin constructor.
- Herencia circular.
- Herencia de clase no declarada.

Luego, se verifican los métodos redefinidos y los atributos heredados. Para ello, se recorren los ancestros de cada clase, verificando que si un método es redefinido, no sea estático, posea la misma cantidad y tipo de parámetros, y que el tipo de retorno coincida. Además, se verifica que los atributos no sean redefinidos. Los errores detectados en esta etapa son:

- Atributo redefinido.
- Redefinición de método estático.
- Redefinición incorrecta de método (Signatura o tipo de retorno diferente).

Posteriormente, se corroboran los tipos de los atributos, métodos, constructores, parámetros y variables. Es decir, se comprueba que los mismos hayan sido declarados. En este paso se encuentran las siguientes errores:

- Tipo de atributo no declarado.
- Tipo de retorno no declarado.
- Tipo de parámetro no declarado.
- Tipo de variable no declarado.

Por último, al momento de agregar una clase, un método, un atributo, un constructor, un parámetro o una variable se verifica que no ocurran las siguientes inconsistencias:

- Clase ya declarada.
- Implementación de la clase ya declarada.
- Método ya declarado en el ámbito.
- Atributo ya declarado en el ámbito.
- Constructor ya declarado.
- Parámetro ya declarado.
- Variable ya declarado.
- Variable posee el mismo identificador que un parámetro del método.

### 2.5.5. Casos de testeo

Tabla 4. Casos de testeo de chequeo de declaraciones.

| Escenario         | Nombre Test                                                          | Salida Esperada                                                                                                                                                                          |
|-------------------|----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Herencia circular | circularInheritance2.s<br><br>Similares<br><br>circularInheritance.s | ERROR:<br>SEMÁNTICO-<br>DECLARACIONES<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 2<br>(COLUMNA 6)  <br>Herencia circular<br>detectada. Ciclo en:<br>B |
| Herencia circular | circularInheritance.s                                                | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO-<br>DECLARACIONES                                                                                                                                     |

|                                   |                                                                                                                       |                                                                                                                                                                                              |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tipos de los atributos            | <p>attributesWithTheSameName.s</p> <p>Similares:</p> <p>attributeTypeNotdefined.s</p> <p>attributeTypeNotExists.s</p> | <p>ERROR:</p> <p>SEMÁNTICO - DECLARACIONES   NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  </p> <p>  LÍNEA 3 (COLUMNA 8)   El atributo de clase x ya fue definido en este ámbito  </p> |
| Implementaciones erróneas         | <p>implNotDefined.s</p> <p>Similares:</p> <p>implRedefinition.s</p>                                                   | <p>ERROR:</p> <p>SEMÁNTICO-DECLARACIONES   NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  </p> <p>  LÍNEA 1 (COLUMNA 6)   La implementación de la clase A no está definida.  </p>       |
| Clases no definidas o redefinidas | <p>classNotDefined.s</p> <p>Similares:</p> <p>classRedefinition.s</p>                                                 | <p>ERROR:</p> <p>SEMÁNTICO - DECLARACIONES   NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  </p> <p>  LÍNEA 2 (COLUMNA 5)   La declaración de la clase A no está</p>                    |

|                                                |                               |                                                                                                                                                                                                                                  |
|------------------------------------------------|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                |                               | definida.                                                                                                                                                                                                                        |
| Atributo redefinido                            | inheritedAttributeRedefined.s | ERROR:<br>SEMÁNTICO-<br>DECLARACIONES<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LINEA 13<br>(COLUMNA 11)   La<br>clase: B posee el<br>atributo: x que ya fue<br>definido en una clase<br>ancestro. |
| Métodos con el mismo nombre en el mismo ámbito | methodsWithTheSameName.s      | ERROR:<br>SEMÁNTICO-<br>DECLARACIONES<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 9<br>(COLUMNA 11)   El<br>método hola ya fue<br>definido en este<br>ámbito                                   |
| Tipo de retorno no existente                   | nonExistingReturnType.s       | ERROR:<br>SEMÁNTICO-<br>DECLARACIONES<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 7                                                                                                            |

|                                                  |                                                                                                           |                                                                                                                                                                                                                               |
|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                  |                                                                                                           | (COLUMNA 7)   El tipo de retorno 'Pepito' del método f no se encuentra declarado.                                                                                                                                             |
| Método estático redefinido                       | redefinedStaticMethod.s<br><br>Similares:<br><br>redefinedStaticMethod2.s<br><br>redefinedStaticMethod3.s | ERROR:<br>SEMÁNTICO-DECLARACIONES<br>  NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  <br>  LÍNEA 12 (COLUMNA 15)   La clase A intenta redefinir el método de clase (static) f de la clase B, lo cual no está permitido. |
| Ancestro no definido                             | undefinedAncestor.s                                                                                       | ERROR:<br>SEMÁNTICO-DECLARACIONES<br>  NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  <br>  LÍNEA 2 (COLUMNA 6)   La clase: B hereda de una clase 'A' que no fue declarada.                                              |
| Variables con el mismo nombre en el mismo ámbito | variablesWithTheSameName.s                                                                                | ERROR:<br>SEMÁNTICO-DECLARACIONES                                                                                                                                                                                             |

|                                              |                                                                                                                                 |                                                                                                                                                                                          |
|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                              | <p>Similares:</p> <p>redefinedVariables.s</p>                                                                                   | <p>  NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  </p> <p>  LÍNEA 8 (COLUMNA 12)   La variable x ya fue definida en este ámbito  </p>                                             |
| Constructor                                  | <p>constructor.s</p> <p>Similares:</p> <p>constructor2.s</p> <p>constructor3.s</p> <p>constructor4.s</p> <p>noConstructor.s</p> | <p>ERROR: SEMÁNTICO - DECLARACIONES   NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  </p> <p>  LÍNEA 4 (COLUMNA 6)   El tipo Pepito del parámetro x no se encuentra definido.  </p> |
| Constructor                                  | <p>constructor.s</p> <p>constructor2.s</p> <p>constructor3.s</p>                                                                | <p>CORRECTO: ANÁLISIS SEMÁNTICO-DECLARACIONES</p>                                                                                                                                        |
| Programa completo con varias funcionalidades | <p>correctProgram.s</p>                                                                                                         | <p>CORRECTO: ANÁLISIS SEMÁNTICO-DECLARACIONES</p>                                                                                                                                        |
| Múltiples herencias                          | <p>inheritance.s</p> <p>Similares:</p>                                                                                          | <p>CORRECTO: ANÁLISIS SEMÁNTICO-</p>                                                                                                                                                     |



|                                                                     |                                                                                                               |                                                                                                                                                                                         |
|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                     | <p>inheritedAttributeNotRedefined.s</p> <p>IOInheritance.s</p>                                                | DECLARACIONES                                                                                                                                                                           |
| Redefinición de método                                              | <p>redefinedMethod.s</p> <p>Similares:</p> <p>redefinedMethod2.s</p> <p>redefinedMethod3.s</p>                | CORRECTO:<br>ANÁLISIS SEMÁNTICO-DECLARACIONES                                                                                                                                           |
| Existe una variable con el mismo nombre que un parámetro que recibe | <p>variableSameNameAsParameter.s</p> <p>Similares:</p> <p>variablesWithTheSameName.s</p>                      | ERROR:<br>SEMÁNTICO-DECLARACIONES   NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:   LÍNEA 16 (COLUMNA 15)   La variable x posee el mismo identificador que un parametro del metodo |
| Se declara una variable de un tipo no definido                      | <p>variableTypeNotDefined.s</p> <p>Similares:</p> <p>localVarTypeNotExists.s</p> <p>undefinedVariableType</p> | ERROR:<br>SEMÁNTICO-DECLARACIONES   NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:   LÍNEA 21 (COLUMNA 4)   El                                                                      |

|                                                      |                                                                       |                                                                                                                                                                                                  |
|------------------------------------------------------|-----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                      | pe.s                                                                  | tipo 'C' de la variable<br>c no se encuentra<br>declarado.                                                                                                                                       |
| Métodos con el<br>mismo nombre en el<br>mismo ámbito | sameNameMethods.<br>s<br><br>Similares:<br><br>sameNameMethods<br>2.s | ERROR:<br>SEMÁNTICO -<br>DECLARACIONES<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LINEA 15<br>(COLUMNA 11)   El<br>método getC ya fue<br>definido en este<br>ámbito |
| Se declaran dos<br>constructores de la<br>clase      | twoConstructors.s                                                     | ERROR:<br>SEMÁNTICO-<br>DECLARACIONES<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 9<br>(COLUMNA 4)   Una<br>clase no puede tener<br>más de un<br>constructor   |
| Clase hereda de una<br>clase no declarada            | undefinedAncestor.s                                                   | ERROR:<br>SEMÁNTICO-<br>DECLARACIONES<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:                                                                                           |

|                                                                                                               |                                                                                    |                                                                                                                                                                                                                                                          |
|---------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                               |                                                                                    | LÍNEA 2<br>(COLUMNA 6)   La<br>clase: B hereda de<br>una clase 'A' que no<br>fue declarada.                                                                                                                                                              |
| El tipo del parámetro<br>no está declarado.                                                                   | parameterUndefined<br>Type.s<br><br>Similares:<br><br>parameterTypeNotEx<br>ists.s | ERROR:<br>SEMÁNTICO -<br>DECLARACIONES<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 15<br>(COLUMNA 29)   El<br>tipo C del parámetro<br>c no se encuentra<br>definido.                                                   |
| La redefinición del<br>método no es<br>correcta debido a<br>que posee diferente<br>cantidad de<br>parámetros. | redefinedMethodArg<br>uments.s                                                     | ERROR:<br>SEMÁNTICO -<br>DECLARACIONES<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LINEA 12<br>(COLUMNA 12)   La<br>clase: B redefine el<br>método: f<br>incorrectamente. La<br>cantidad de<br>parámetros no es la<br>misma. |
| La redefinición del<br>método no es                                                                           | redefinedMethodArg<br>umentsTypes.s                                                | ERROR:<br>SEMÁNTICO -                                                                                                                                                                                                                                    |

|                                                           |                                                                                                                                                                                         |                                                                                                                                                                                                              |
|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| correcta debido a que posee diferente tipo de parámetros. | <p>Similares:</p> <p>redefinesMethodType.s</p> <p>redefinedMethodWithInvalidSignature.s</p> <p>redefinedMethodWithInvalidSignature2.s</p> <p>redefinedMethodWithInvalidSignature3.s</p> | <p>DECLARACIONES</p> <p>  NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  </p> <p>  LÍNEA 12 (COLUMNA 12)   La clase: B redefine el método: f incorrectamente. Los parámetros son de distinto tipo  </p> |
|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|



no contienen todos sus atributos, sólo los correspondientes a otros nodos y a operadores. Las flechas representan herencias.

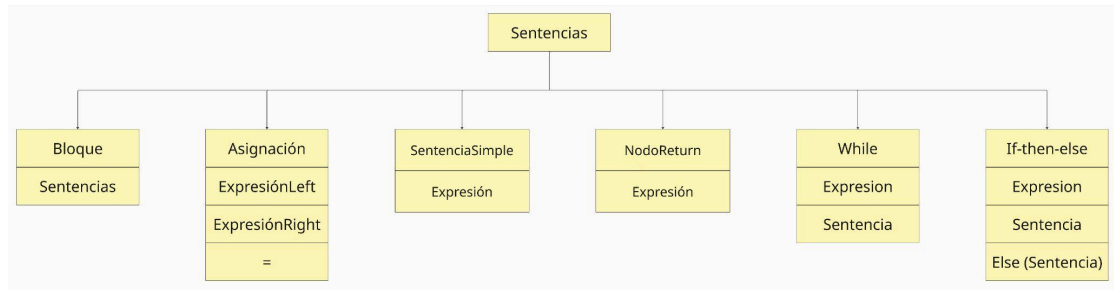


Figura 7. Nodos de tipo sentencia.

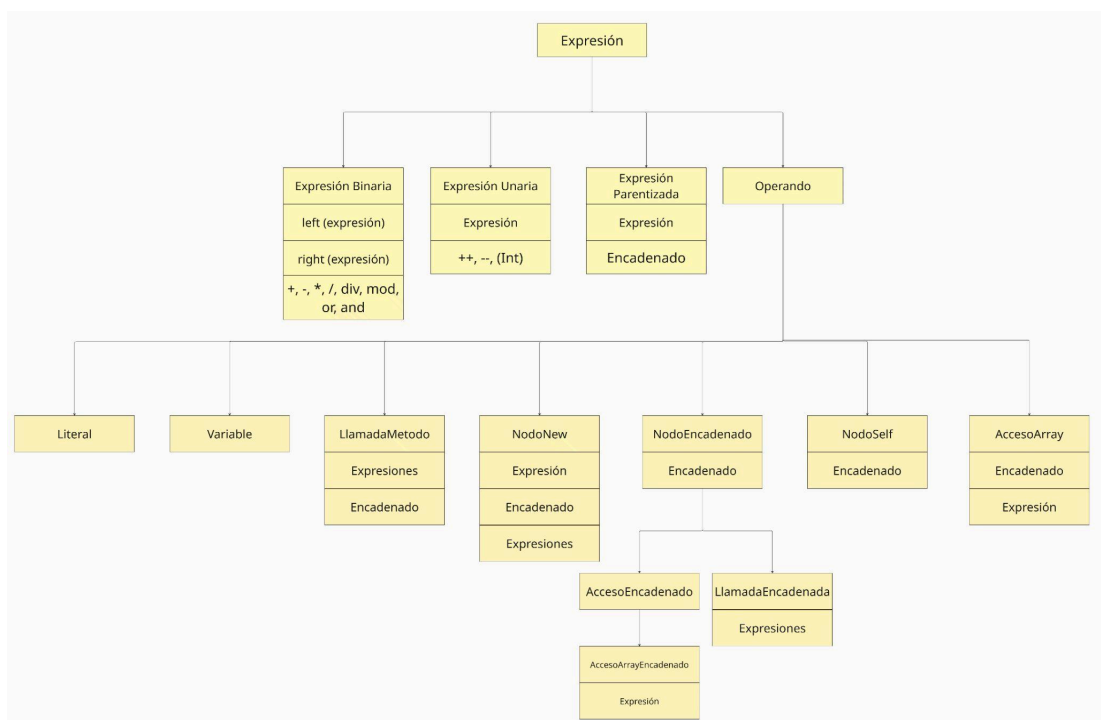


Figura 8. Nodos de tipo expresión.

## 2.6.2. Esquema de traducción

$\langle \text{Program} \rangle ::= \langle \text{Lista-Definiciones} \rangle \langle \text{Start} \rangle$

$\langle \text{Start} \rangle ::= \{ \text{AST.setCurrentClass(null);}$   
 $\text{AST.setCurrentMethod(lookahead);} \}$  pstart  $\langle \text{Bloque-Metodo} \rangle$

$\langle \text{Lista-Definiciones} \rangle ::= \langle \text{Class} \rangle \langle \text{Lista-Definiciones} \rangle$

$\langle \text{Lista-Definiciones} \rangle ::= \langle \text{Impl} \rangle \langle \text{Lista-Definiciones} \rangle$

<Lista-Definiciones> ::= λ

⟨Class⟩ ::= pclass id\_class <Class2>

<Class2> ::= ⟨Herencia⟩ braces1 ⟨Atributos⟩ braces2

<Class2> ::= braces1 ⟨Atributos⟩ braces2

<Atributos> ::= <Atributo> <Atributos>

<Atributos> ::= λ

⟨Impl⟩ ::= pimpl {AST.setCurrentClass(lookahead.getLexeme());} id\_class  
braces1 ⟨Miembro⟩ <Miembros> braces2

<Miembros> ::= <Miembro> <Miembros>

<Miembros> ::= λ

⟨Herencia⟩ ::= colon ⟨Tipo⟩

⟨Miembro⟩ ::= ⟨Metodo⟩

⟨Miembro⟩ ::= ⟨Constructor⟩

⟨Constructor⟩ ::= {AST.setCurrentMethod(null);} dot ⟨Argumentos-Formales⟩  
⟨Bloque-Metodo⟩

⟨Atributo⟩ ::= ⟨Visibilidad⟩ ⟨Tipo⟩ ⟨Lista-Declaracion-Variables⟩ } semicolon

⟨Atributo⟩ ::= ⟨Tipo⟩ ⟨Lista-Declaracion-Variables⟩ semicolon

⟨Metodo⟩ ::= ⟨Forma-Metodo⟩ pfn <Metodo2>

⟨Metodo⟩ ::= pfn <Metodo2>

<Metodo2> ::= ⟨Tipo-Metodo⟩

{AST.setCurrentMethod(lookahead.getLexeme()); } id\_obj

⟨Argumentos-Formales⟩ ⟨Bloque-Metodo⟩

<Metodo2> ::= {AST.setCurrentMethod(lookahead.getLexeme());} id\_obj  
<Argumentos-Formales> <Bloque-Metodo>

<Visibilidad> ::= ppub

<Forma-Metodo> ::= pst

<Bloque-Metodo> ::= braces1 <Mas-Decl-Var-Locales> {BlockNode blockNode  
= new BlockNode(AST.getCurrentClass(), AST.getCurrentMethod(), true);  
AST.addBlockNode(blockNode); sentencias(blockNode)} <Sentencias>

<Sentencias> ::= <Sentencia> {blockNode.addSentence(sentencia());  
sentencias(blockNode);} <Sentencias>  
<Sentencias> ::= braces2

<Mas-Decl-Var-Locales> ::= <Decl-Var-Locales> <Mas-Decl-Var-Locales>  
<Mas-Decl-Var-Locales> ::=  $\lambda$

<Decl-Var-Locales> ::= <Tipo> <Lista-Declaracion-Variables> semicolon

<Lista-Declaracion-Variables> ::= id\_obj <Lista-Declaracion-Variables2>

<Lista-Declaracion-Variables2> ::= comma <Lista-Declaracion-Variables>  
<Lista-Declaracion-Variables2> ::=  $\lambda$

<Argumentos-Formales> ::= parentheses1 <Argumentos-Formales2>

<Argumentos-Formales2> ::= <Lista-Argumentos-Formales> parentheses2  
<Argumentos-Formales2> ::= parentheses2

<Lista-Argumentos-Formales> ::= <Argumento-Formal>  
<Lista-Argumentos-Formales2>

<Lista-Argumentos-Formales2> ::= comma <Lista-Argumentos-Formales>  
<Lista-Argumentos-Formales2> ::=  $\lambda$

<Argumento-Formal> ::= <Tipo> id\_obj



⟨Tipo-Metodo⟩ ::= ⟨Tipo⟩

⟨Tipo-Metodo⟩ ::= pvoid

⟨Tipo⟩ ::= ⟨Tipo-Primitivo⟩

⟨Tipo⟩ ::= parray ⟨Tipo-Primitivo⟩

⟨Tipo⟩ ::= id\_class

⟨Tipo-Primitivo⟩ ::= pstr

⟨Tipo-Primitivo⟩ ::= pbool

⟨Tipo-Primitivo⟩ ::= pint

⟨Tipo-Primitivo⟩ ::= pdouble

⟨ExpresionParentizada2⟩ ::= {ParenthesizedExpressionNode  
parenthesizedExpressionNode = new ParenthesizedExpressionNode();}

<ExpOr> {ExpressionNode expressionNode = expOr();  
parenthesizedExpressionNode.setExpression(expressionNode);}  
parentheses2 <ExpresionParentizada3> { ChainedNode chainedNode =  
expresionParentizada3();  
parenthesizedExpressionNode.setChainedNode(chainedNode);  
return parenthesizedExpressionNode; }

<ExpresionParentizada3> ::= <Encadenado> {return encadenado();}

<ExpresionParentizada3> ::= λ

⟨AccesoSelf⟩ ::= { SelfNode selfNode = new SelfNode(lookahead,  
AST.getCurrentClass(), AST.getCurrentMethod()); } pself

⟨AccesoSelf2⟩ {ChainedNode chainedNode = accesoSelf2();  
selfNode.setChainedNode(chainedNode);  
return selfNode;}

⟨AccesoSelf2⟩ ::= <Encadenado> {return encadenado();}

⟨AccesoSelf2⟩ ::= λ

⟨AccesoVar2⟩ ::= <AccesoVar3> {ChainedNode chainedNode = accesoVar3();  
if (chainedNode == null) {

```

        VariableNode variableNode = new VariableNode(token,
AST.getCurrentClass(), AST.getCurrentMethod());
        return variableNode;
    } else {
        ChainedAccessNode chainedAccessNode = new
ChainedAccessNode(token, AST.getCurrentClass(), AST.getCurrentMethod());
        chainedAccessNode.setChainedNode(chainedNode);
        return chainedAccessNode;
    }
}

<AccesoVar2> ::= {ArrayAccessNode arrayAccessNode = new
ArrayAccessNode(token);} brackets1 <ExpOr> { ExpressionNode
expressionNode = expOr();
arrayAccessNode.setExpression(expressionNode);} brackets2 <AccesoVar3>
{ChainedNode chainedNode = accesoVar3();
arrayAccessNode.setChainedNode(chainedNode);}

<AccesoVar3> ::= <Encadenado> {return encadenado();}
<AccesoVar3> ::= λ

<Llamada-Metodo> ::= {MethodCallNode methodCallNode = new
MethodCallNode(class1, AST.getCurrentClass(), AST.getCurrentMethod(),
lookahead, isStatic);} id <Argumentos-Actuales> {List<ExpressionNode>
expressionList = argumentosActuales();
methodCallNode.setParameterList(expressionList);}
<Llamada-Metodo2> {ChainedNode chainedNode = llamadaMetodo2();
methodCallNode.setChainedNode(chainedNode);}

<Llamada-Metodo2> ::= <Encadenado> {return encadenado();}
<Llamada-Metodo2> ::= λ

<Llamada-Metodo-Estatico> ::= {String class1 = lookahead.getLexeme();;}
id_class dot {boolean isStatic = true;} <Llamada-Metodo> {return
llamadaMetodo(class1, isStatic);}

<Llamada-Conclassor> ::= pnw <Llamada-Conclassor2> {return
llamadaConclassor2();}

```

```

<Llamada-Conclassor2> ::= {NewNode newNode = new NewNode(lookahead,
"class");} id_class <Argumentos-Actuales> {List<ExpressionNode>
parameterList = argumentosActuales();
newNode.setParameterList(parameterList);} <Llamada-Conclassor3>
{ChainedNode chainedNode = llamadaConclassor3();
newNode.setChainedNode(chainedNode);
return newNode;}

<Llamada-Conclassor2> ::= {NewNode newNode = new NewNode(lookahead,
"array");} <Tipo-Primitivo> brackets1 <ExpOr> {ExpressionNode
expressionNode = expOr();
newNode.setExpression(expressionNode);} brackets2
<Llamada-Conclassor3> {ChainedNode chainedNode =
llamadaConclassor3();
newNode.setChainedNode(chainedNode);
return newNode;}

<Llamada-Conclassor3> ::= <Encadenado> {return encadenado();}
<Llamada-Conclassor3> ::= λ

<Argumentos-Actuales> ::= parentheses1 <Argumentos-Actuales2> { return
argumentosActuales2(); }

<Argumentos-Actuales2> ::= {List<ExpressionNode> expressionList = new
ArrayList<>();
listaExpresiones(expressionList);} <Lista-Expresiones> parentheses2 { return
expressionList; }
<Argumentos-Actuales2> ::= parentheses2 {return new ArrayList<>();}

<Lista-Expresiones> ::= <ExpOr> { ExpressionNode expressionNode =
expOr();
expressionList.add(expressionNode);
listaExpresiones2(expressionList); } <Lista-Expresiones2>

<Lista-Expresiones2> ::= comma
{ listaExpresiones(expressionList);} <Lista-Expresiones>
<Lista-Expresiones2> ::= λ

```

```

<Encadenado> ::= dot { Token token = lookahead; } id
               <Encadenado2> {return encadenado2(token); }

<Encadenado2> ::= <Llamada-Metodo-Encadenado>
               { return llamadaMetodoEncadenado(token); }
<Encadenado2> ::= <Acceso-Variable-Encadenado>
               { return accesoVariableEncadenado(token); }

<Llamada-Metodo-Encadenado> ::= {ChainedCallNode chainedCallNode =
new ChainedCallNode(token);} <Argumentos-Actuales>
{List<ExpressionNode> expressionList = argumentosActuales();
chainedCallNode.setParameterList(expressionList);}
<Llamada-Metodo-Encadenado2> {ChainedNode chainedNode =
llamadaMetodoEncadenado2();
chainedCallNode.setChainedNode(chainedNode);
return chainedCallNode;}

<Llamada-Metodo-Encadenado2> ::= <Encadenado> {return encadenado();}
<Llamada-Metodo-Encadenado2> ::= λ

<Acceso-Variable-Encadenado> ::= <Acceso-Variable-Encadenado3> { return
accesoVariableEncadenado3(token); }
<Acceso-Variable-Encadenado> ::= {ChainedArrayAccessNode
chainedArrayAccessNode = new ChainedArrayAccessNode(token,
AST.getCurrentClass(), AST.getCurrentMethod()); } brackets1 <ExpOr>
{ExpressionNode expressionNode = expOr();
chainedArrayAccessNode.setExpression(expressionNode);} brackets2
<Acceso-Variable-Encadenado3> {ChainedNode chainedNode =
accesoVariableEncadenado3();
chainAccessNode.setChainedNode(chainedNode);
return chainedAccessNode;}

<Acceso-Variable-Encadenado3> ::= <Encadenado> {ChainedNode chained
= encadenado();
if (token != null) {

```

```

ChainedAccessNode chainedAccessNode = new ChainedAccessNode(token,
AST.getCurrentClass(), AST.getCurrentMethod();
chainedAccessNode.setChainedNode(chained); }
else {
return chained;}}
<Acceso-Variable-Encadenado3> ::= λ {if (token != null) {return new
ChainedAccessNode(token, AST.getCurrentClass(), AST.getCurrentMethod();}
else {
return null;}}

<Sentencia> ::= semicolon
<Sentencia> ::= <Asignación> {AssignmentNode assignmentNode =
asignacion();} semicolon {return assignmentNode; }
<Sentencia> ::= <Sentencia-Simple> {SimpleSentenceNode
simpleSentenceNode = sentenciaSimple();} semicolon {return
simpleSentenceNode;}
<Sentencia> ::= {Token token = lookahead;} pif parentheses1
    <ExpOr> {ExpressionNode expressionNode = expOr();}
    parentheses2 <Sentencia> {SentenceNode sentenceNode =
    sentencia();}
    <Sentencialf> {SentenceNode elseSentenceNode = sentencialf();
    return new IfThenElseNode(expressionNode, sentenceNode,
    elseSentenceNode, AST.getCurrentClass(), AST.getCurrentMethod(),
    token);}
<Sentencia> ::= {Token token = lookahead;} pwhile parentheses1
    <ExpOr> {ExpressionNode expressionNode = expOr();}
    parentheses2 <Sentencia> {SentenceNode sentenceNode =
    sentencia();
    return new WhileNode(expressionNode, sentenceNode,
    AST.getCurrentClass(), AST.getCurrentMethod(), token);}
<Sentencia> ::= <Bloque> {return bloque();}
<Sentencia> ::= ret <SentenciaRet> {ExpressionNode expressionNode =
sentenciaRet(); return new ReturnNode(expressionNode);}

<SentenciaRet> ::= semicolon
<SentenciaRet> ::= <ExpOr> {ExpressionNode expressionNode = expOr();}
    semicolon {return expressionNode;}

```

```

<Sentencialf> ::= pelse <Sentencia> {return sentencia();}
<Sentencialf> ::= λ {return null;}

<Bloque> ::= braces1 {BlockNode blockNode = new
BlockNode(AST.getCurrentClass(), AST.getCurrentMethod(), false);
sentencias(blockNode);} <Sentencias> {return blockNode;}

<Asignación> ::= <AccesoVar-Simple> {ExpressionNode leftNode =
accesoVarSimple(); }
    op_equal <ExpOr> {ExpressionNode rightNode = expOr();}
    {return new AssignmentNode(leftNode, rightNode);}
<Asignación> ::= <AccesoSelf-Simple> {ExpressionNode leftNode =
accesoSelfSimple();}
    op_equal <ExpOr> {ExpressionNode rightNode = expOr();}
    {return new AssignmentNode(leftNode, rightNode);}

<AccesoVar-Simple> ::= {Token token = lookahead;} id <AccesoVar-Simple2>
{return accesoVarSimple2(token);}

<AccesoVar-Simple2> ::= <Encadenados-Simples> {ChainedNode
chainedNode = encadenadosSimples();
if (chainedNode == null) {
    VariableNode variableNode = new VariableNode(token,
AST.getCurrentClass(), AST.getCurrentMethod());
    return variableNode;
} else {
    ChainedAccessNode chainedAccessNode = new
ChainedAccessNode(token, AST.getCurrentClass(), AST.getCurrentMethod()
);
    chainedAccessNode.setChainedNode(chainedNode);
    return chainedAccessNode;
}
}

<AccesoVar-Simple2> ::=
    {ArrayAccessNode arrayAccessNode = new
    ArrayAccessNode(token);} brackets1
    <ExpOr> {ExpressionNode expressionNode = expOr();}

```

```

    arrayAccessNode.setExpression(expressionNode);
    return arrayAccessNode;} brackets2

```

```

<Encadenados-Simples> ::= <Encadenado-Simple> {ChainedAccessNode
    chainedAccessNode = encadenadoSimple();} <Encadenados-Simples>
    {ChainedAccessNode chainAccessNode2 = encadenadosSimples();
    chainedAccessNode.setChainedNode(chainAccessNode2);
    return chainedAccessNode;}
<Encadenados-Simples> ::= λ

```

```

<AccesoSelf-Simple> ::= {Token token = lookahead;} pself
<Encadenados-Simples> {ChainedAccessNode chainedAccessNode =
    encadenadosSimples();
    SelfNode selfNode = new SelfNode(token, AST.getCurrentClass(),
    AST.getCurrentMethod());
    selfNode.setChainedNode(chainedAccessNode);
    return selfNode;}

```

```

<Encadenado-Simple> ::= dot {Token token = lookahead;} id {return new
    ChainedAccessNode(token, AST.getCurrentClass(),
    AST.getCurrentMethod());}

```

```

<Sentencia-Simple> ::= parentheses1 <ExpOr> {ExpressionNode
    expressionNode = expOr();} parentheses2 {return new
    SimpleSentenceNode(expressionNode);}

```

```

<ExpOr> ::= <ExpAnd> { ExpressionNode leftNode = expAnd(); }
    <ExpOr2> { return expOr2(leftNode); }

```

```

<ExpOr2> ::= { Token token = lookahead; } op_or <ExpAnd> {
    ExpressionNode rightNode = expAnd() }
    { ExpressionNode newLeftNode = new
    BinaryExpressionNode(leftNode, rightNode, token) }
    <ExpOr2> { return expOr2(newLeftNode); }
<ExpOr2> ::= λ { return leftNode; }

```

```

<ExpAnd> ::= <ExpIgual> { ExpressionNode leftNode = expIgual(); }

```

```

<ExpAnd2> { return expAnd2(leftNode); }

<ExpAnd2> ::= { Token token = lookahead; } op_and <Explgual> {
    expressionNode rightNode = explgual(); }
    { ExpressionNode newLeftNode = new
    BinaryExpressionNode(leftNode, rightNode, token); }
    <ExpAnd2> { return expAnd2(newLeftNode); }
<ExpAnd2> ::= λ { return leftNode; }

<Explgual> ::= <ExpCompuesta> { ExpressionNode leftNode =
    expCompuesta(); }
    <Explgual2> { return explgual2(leftNode); }

<Explgual2> ::= <Oplgual> { Token token = oplgual(); } <ExpCompuesta> {
    ExpressionNode rightNode= expCompuesta();
    ExpressionNode newLeftNode = new BinaryExpressionNode(leftNode,
    rightNode, token); }
    <Explgual2> { return explgual2(newLeftNode ); }
<Explgual2> ::= λ { return leftNode; }

<ExpCompuesta> ::= <ExpAd> { ExpressionNode leftNode = expAd(); }
    <ExpCompuesta2> { return expCompuesta2(leftNode); }

<ExpCompuesta2> ::= <OpCompuesto> { Token token = opCompuesto(); }
    <ExpAd> { ExpressionNode rightNode = expAd(); }
    { ExpressionNode expressionNode = BinaryExpressionNode (leftNode,
    rightNode, token); }
<ExpCompuesta2> ::= λ { return leftNode; }

<ExpAd> ::= <ExpMul> { ExpressionNode leftNode = expMul(); }
    <ExpAd2> { return expAd2(leftNode); }

<ExpAd2> ::= <OpAd> { Token token = opAd(); }
    <ExpMul> { ExpressionNode rightNode = expMul(); }

```



```

        { ExpressionNode newLeftNode= new
BinaryExpressionNode(leftNode, rightNode, token); }
        <ExpAd2> { return expAd2(newLeftNode); }
<ExpAd2> ::= λ { return leftNode; }

<ExpMul> ::= <ExpUn> { ExpressionNode leftNode = expUn(); }
        <ExpMul2> { return expMul2(leftNode); }

<ExpMul2> ::= <OpMul> { Token token = opMul(); }
        <ExpUn> { ExpressionNode rightNode = expUn(); }
        { ExpressionNode newLeftNode = new
BinaryExpressionNode(leftNode, rightNode, token); }
        <ExpMul2> { return expMul2(newLeftNode); }
<ExpMul2> ::= λ { return leftNode; }

<ExpUn> ::= <OpUnario> { Token token = opUnario(); }
        <ExpUn> { ExpressionNode expressionNode = expUn(); }
        { return new UnaryNode(expressionNode, token); }
<ExpUn> ::= <Operando> { return operando(); }
<ExpUn> ::= parenthesis1 <ExpUnAux> { return expUnAux(); }

<ExpUnAux> ::= { Token token = lookahead; } pint parentheses2
        <ExpUn> { ExpressionNode expressionNode = expUn(); }
        { return new UnaryNode(expressionNode, token); }
<ExpUnAux> ::= <ExpresionParentizada2> { return expresionParentizada2(); }

<OpIguar> ::= {Token token = lookahead;} op_rel_equal {return token;}
<OpIguar> ::= {Token token = lookahead;} op_rel_notequal {return token;}

<OpCompuesto> ::= {Token token = lookahead;} op_rel_less {return token;}
<OpCompuesto> ::= {Token token = lookahead;} op_rel_greater {return
token;}
<OpCompuesto> ::= {Token token = lookahead;} op_rel_lessequal {return
token;}
<OpCompuesto> ::= {Token token = lookahead;} op_rel_greaterequal {return
token;}

```

```

<OpAd> ::= {Token token = lookahead;} op_sum {return token;}
<OpAd> ::= {Token token = lookahead;} op_sub {return token;}

<OpUnario> ::= {Token token = lookahead;} op_sum {return token;}
<OpUnario> ::= {Token token = lookahead;} op_sub {return token;}
<OpUnario> ::= {Token token = lookahead;} op_not {return token;}
<OpUnario> ::= {Token token = lookahead;} op_increment {return token;}
<OpUnario> ::= {Token token = lookahead;} op_decrement {return token;}

<OpMul> ::= {Token token = lookahead;} op_mult {return token;}
<OpMul> ::= {Token token = lookahead;} op_div {return token;}
<OpMul> ::= {Token token = lookahead;} op_mod {return token;}
<OpMul> ::= {Token token = lookahead;} pdiv {return token;}

<Operando> ::= <Literal> {return literal();}
<Operando> ::= <Primario> { return primario(); }

<Literal> ::= {Token token = lookahead;} pnul {return new LiteralNode(token,
"nul");}
<Literal> ::= {Token token = lookahead;} ptrue {return new LiteralNode(token,
"true");}
<Literal> ::= {Token token = lookahead;} pfalse {return new LiteralNode(token,
"false");}
<Literal> ::= {Token token = lookahead;} const_int {return new
LiteralNode(token, "const_int");}
<Literal> ::= {Token token = lookahead;} const_string {return new
LiteralNode(token, "const_string");}
<Literal> ::= {Token token = lookahead;} const_double {return new
LiteralNode(token, "const_double");}

<Primario> ::= <AccesoSelf> { return accesoSelf(); }
<Primario> ::= {Token token = lookahead;} id <Primario2> {return
primario2(token);}
<Primario> ::= <Llamada-Método-Estático> { return llamadaMetodoEstatico();}
<Primario> ::= <Llamada-Conclassor> {return llamadaConclassor(); }

<Primario2> ::= {bool isStatic = false;

```

```

MethodCallNode methodCallNode = new
MethodCallNode(AST.getCurrentClass(), AST.getCurrentClass(),
AST.getCurrentMethod(), token, isStatic); }
<Argumentos-Actuales> { List<ExpressionNode> parameterList=
argumentosActuales();
methodCallNode.setParameterList(parameterList);}
<Primario3> {ChainedNode chainedNode = primario3();
methodCallNode.setChainedNode(chainedNode);
return methodCallNode;}
<Primario2> ::= <AccesoVar2> { return accesoVar2(token); }

<Primario3> ::= <Encadenado> { return encadenado(); }
<Primario3> ::= λ

```

### 2.6.3. Ejemplo de JSON

Para el mismo código que en la Figura 4, se generó el siguiente JSON que contiene la estructura del árbol sintáctico abstracto. El mismo contiene una lista de nodos bloque, donde cada uno corresponde al bloque start o el cuerpo de un método. Cada bloque incluye información contextual (nombre de clase, nombre de método, entre otros) y una lista de sentencias. Las sentencias y expresiones se representan de forma jerárquica y recursiva, identificadas mediante un campo type que indica el tipo de nodo del AST.

```

{
  "blockNodes": [
    {
      "type": "BlockNode",
      "className": "A",
      "methodName": null,
      "methodBlock": true,
      "sentences": []
    },
    {
      "type": "BlockNode",
      "className": "B",
      "methodName": null,
      "methodBlock": true,
      "sentences": [
        {
          "type": "AssignmentNode",
          "leftNode": {
            "type": "SelfNode",
            "token": {
              "name": "pself",
              "lexeme": "self",
              "valor": null,
              "column": 4,
              "row": 14
            }
          },
          "className": "B",
          "chainedNode": {
            "type": "ChainedAccessNode",
            "name": {
              "name": "id_obj",
              "lexeme": "c",
              "valor": null,
              "column": 9,
              "row": 14
            },
            "className": "B",
            "methodName": null,
            "chainedNode": null
          }
        },
        {
          "type": "VariableNode",
          "token": {
            "name": "id_obj",
            "lexeme": "c",
            "valor": null
          }
        }
      ]
    }
  ]
}

```

Figura 9. JSON del AST.

El JSON completo se puede observar en el siguiente enlace o en el anexo 4:

[https://drive.google.com/file/d/1WhH9gjXdrki5OSnhN\\_pMA45Fn5rvUBJ-/view?usp=sharing](https://drive.google.com/file/d/1WhH9gjXdrki5OSnhN_pMA45Fn5rvUBJ-/view?usp=sharing)

#### 2.6.4. Errores semánticos detectados

En esta etapa, los errores semánticos se detectan en cada nodo del AST. A continuación, se indican los errores encontrados en cada nodo:

##### **ArrayAccessNode:**

En este nodo, primero se verifica que el acceso al array ocurra dentro de un contexto válido, es decir, dentro de un método o constructor perteneciente a una clase existente. Si esto no se cumple, se detecta un error. Luego, se busca el identificador en el contexto actual. Esto se realiza primero en los parámetros del método, luego en las variables locales y finalmente en los atributos de la clase. Si el identificador no se encuentra en ninguno de estos ámbitos, se detecta un error por el uso de un identificador no declarado. Como siguiente paso, se corrobora que su tipo corresponda a un array y, si no es así, se detecta un error de tipo.

También se verifica que el índice sea una expresión válida, especialmente de tipo Int. Esto nos permite detectar índices inválidos. Por otro lado, se comprueba que el array posea un tipo interno correctamente definido y válido, por lo que en caso contrario se encuentra un error de tipo.

Finalmente, se verifica que el acceso al array no posea encadenamiento, ya que estos no admiten encadenamientos.

Por lo tanto, los errores encontrados en este nodo son:

- Método y clase actual nulos
- Identificador no declarado en clase actual
- Se intentó indexar una variable que no es de tipo Array
- Índice nulo
- El índice no es de tipo Int
- El array no tiene subtipo
- Array con encadenamiento

De manera similar, en los nodos restantes se realizan chequeos obteniendo los tipos y verificando la compatibilidad, coherencia y rectitud entre ellos. Por ejemplo, en una asignación se verifica que el tipo del nodo izquierdo y el del derecho sean compatibles entre sí. Además, en estos chequeos se verifica que las entidades utilizadas hayan sido correctamente definidas en el ámbito en el que son accedidas. Esto se realiza en conjunto con la tabla de símbolos. También, se hacen otro tipo de verificaciones como: la presencia y ausencia del retorno en un método si corresponde; si los accesos a ciertas variables o atributos corresponden dependiendo de su visibilidad; la correcta cantidad de parámetros de un método en una invocación con un tipo adecuado; el correcto uso de self y contextos estáticos, entre otros.

**AssignmentNode:**

- Se intentó asignar self
- Tipo asignado incorrecto
- Se intentó asignar nil a un tipo primitivo

**BinaryExpressionNode:**

- Se encontró un tipo distinto al que se esperaba
- Se esperaba que ambos tipos fueran iguales
- Se esperaban tipos Double o Int

En este nodo, los errores son de tipo. En general, estos son similares y se repiten muchas veces, pero cambia el tipo que se esperaba dependiendo el caso (si es una operación en la que se espera Double, Int, Bool, etc.).

**BlockNode:**

- Se esperaba un retorno ya que el tipo del método no es void

**ChainedAccessNode:**

- No se permite que un Array tenga encadenamiento
- Tipo void no está permitido en un encadenamiento
- Clase no declarada en el contexto actual
- Atributo no declarado en la clase actual

- No se puede acceder a un atributo privado desde start o desde otra clase
- Tipo de clase inexistente
- Método y clase actual nulos
- Variable no inicializada debidamente en el ámbito en el que se encuentra

#### **ChainedArrayAccessNode:**

- Un array no puede tener encadenamiento
- El tipo void no puede estar en un encadenamiento
- No existe un atributo de tipo Array en la clase actual
- No se puede acceder a un atributo privado desde start u otra clase
- Índice nulo
- El índice del array debe ser de tipo Int
- Tipo del array es nulo
- Clase y método actuales son nulos.
- Variable inicializada incorrectamente en el ámbito actual

#### **ChainedCallNode:**

- El tipo void no puede estar en un encadenamiento
- Clase no declarada en el contexto actual
- Método no declarado en la clase actual
- La cantidad de parámetros es incorrecta
- Tipo de parámetro incorrecto
- El tipo no existe

#### **IfThenElseNode**

- La condición del if no devuelve un booleano (true o false)

#### **LiteralNode:**

- El tipo no es Int, Double, Str, Bool o nil

#### **MethodCallNode:**

- Se intenta llamar un método de instancia sin utilizar la instancia correspondiente
- El método no se encuentra declarado en el ámbito actual

- El método invocado no es estático
- Se intenta llamar a un método de instancia desde un contexto estático
- La cantidad de parámetros es incorrecta
- El tipo del parámetro es incorrecto

#### **SingleNode:**

- La clase referenciada no está declarada
- Un Array no puede tener encadenamiento
- La cantidad de parámetros es incorrecta
- El tipo del parámetro es incorrecto

#### **ReturnNode:**

- Un constructor no puede tener retorno
- El bloque start no puede tener retorno
- Un método void no puede contener retorno
- El tipo retornado es incorrecto

#### **SelfNode:**

- Self no puede referenciarse dentro de start
- Self no puede referenciarse en un contexto estático

#### **UnaryExpressionNode:**

- Se esperaba un tipo y se encontró otro
- Operador incorrecto

#### **VariableNode:**

- La clase y el método actual son nulos
- La clase es nula
- Un atributo de instancia es accedido desde un contexto estático
- Variable declarada incorrectamente en el contexto actual

#### **WhileNode:**

- La condición del while debe devolver un booleano (true o false)



### 2.6.5. Casos de testeo

Tabla 5. Casos de testeo de chequeo de sentencias.

| Escenario                                                   | Nombre Test                                                           | Salida Esperada                                    |
|-------------------------------------------------------------|-----------------------------------------------------------------------|----------------------------------------------------|
| Expresiones binarias con arrays                             | arrayAccessMul.s<br>arraysComparison.s<br>arraysComparison2.s         | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |
| Operaciones binarias con distintos operadores               | boolComparison.s<br>divOperator.s<br>pdivOperator.s<br>relationalOp.s | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |
| Suma, resta, incremento y decremento en expresiones unarias | increment.s<br>incrementAndDecrement.s<br>subAndSum.s<br>subAndSum2.s | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |
| Casteo                                                      | castOperator.s                                                        | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |
| Incremento en int de array                                  | arrayIncrement.s                                                      | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |

|                                                                        |                                                                                                                                    |                                                    |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| Operador de negación                                                   | notOperator.s                                                                                                                      | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |
| Llamada a método<br>length de un array                                 | arrayCallMethodTest.<br>s                                                                                                          | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |
| Asignaciones<br>usando self                                            | assignmentUsingSelf<br>.s<br><br>attributeAssignedWithSelf.s                                                                       | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |
| Expresiones<br>booleanas en<br>bloques if y while                      | booleanExpressionInIf.s<br><br>booleanExpressionInWhile.s                                                                          | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |
| Encadenados en<br>constructor, usando<br>self, con array y sin<br>void | chainedConstructor.s<br><br>chainedSelf.s<br><br>chainingWithoutVoid.s<br><br>chainingWithoutVoid2.s<br><br>chainingWithoutVoid3.s | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |

|                                                          |                                                                                                                        |                                                    |
|----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
|                                                          | correctChainedArray<br>Access.s                                                                                        |                                                    |
| Fibonacci                                                | completeTest.s                                                                                                         | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |
| Asignación normal y<br>con nil                           | correctAssignment.s<br><br>nilInAssignment.s                                                                           | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |
| Constructor con<br>parámetros                            | correctConstructorWi<br>thParameters.s                                                                                 | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |
| Polimorfismo en<br>distintas situaciones                 | correctPolymorphism<br>.s<br><br>polymorphism.s<br><br>polymorphismInPara<br>meters.s<br><br>polymorphismMetho<br>ds.s | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |
| Declaraciones de<br>atributos, variables y<br>parámetros | declaredAttribute.s<br><br>declaredVariable.s<br><br>declaredParameter.s                                               | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |
| Llamada a métodos<br>con self y con                      | methodCalledWithSe<br>lf.s                                                                                             | CORRECTO:<br>ANÁLISIS                              |

|                                                        |                                                                                   |                                                    |
|--------------------------------------------------------|-----------------------------------------------------------------------------------|----------------------------------------------------|
| parámetros correctos                                   | methodWithRightType.s<br><br>methodWithRightType2.s                               | SEMÁNTICO -<br>SENTENCIAS                          |
| Operación booleana con nil                             | nilBooleanOperation.s                                                             | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |
| Retornos en métodos                                    | retInMethod.s<br>retWithNil.s                                                     | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |
| Accesos desde método start                             | startAccessingInstanceAttributes.s<br><br>startAccessingInstanceMethods.s         | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |
| Llamadas a métodos estáticos                           | staticMethodCall.s<br><br>staticMethodCall2.s<br><br>staticMethodFromAnInstance.s | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |
| Creación de variables de tipos existentes              | variableTypeExists.s                                                              | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -<br>SENTENCIAS |
| Encadenado de método (length) con un objeto Str que se | strArrayWithChainedNode.s                                                         | CORRECTO:<br>ANÁLISIS<br>SEMÁNTICO -               |

|                                                         |                                     |                                                                                                                                                                                                                         |
|---------------------------------------------------------|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| encuentra dentro de un array                            | strArrayWithChainedNode2.s          | SENTENCIAS                                                                                                                                                                                                              |
| Operador and (&&) en expresión binaria                  | andOperator.s<br>andOperator2.s     | ERROR:<br>SEMÁNTICO - SENTENCIAS   NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  <br>  LÍNEA 4 (COLUMNA 10)   Se esperaba que ambos operadores fueran Bool. Se encontró: Int y Int                                |
| Operador de comparación igual (==) en expresión binaria | eqComparation.s<br>eqComparation2.s | ERROR:<br>SEMÁNTICO - SENTENCIAS   NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  <br>  LINEA 11 (COLUMNA 15)   Se esperaba que ambos operadores fueran del mismo tipo o tipo Int y Double. Se encontró: Int y Str |
| Se intenta acceder a un array incorrectamente           | incorrectArrayAccess.s              | ERROR:<br>SEMÁNTICO - SENTENCIAS   NÚMERO DE LÍNEA (NÚMERO                                                                                                                                                              |

|                                                  |                                                           |                                                                                                                                                                                                                        |
|--------------------------------------------------|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                  |                                                           | DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 12<br>(COLUMNA 12)   No<br>se permite que una<br>variable de tipo Array<br>tenga<br>encadenamiento                                                                          |
| Operación binaria<br>con tipo incorrecto         | incorrectDivType.s<br><br>incorrectPdivOperato<br>r.s     | ERROR:<br>SEMÁNTICO -<br>SENTENCIAS<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 4<br>(COLUMNA 10)   El<br>tipo asignado es<br>incorrecto. Se<br>esperaba: Int. Se<br>obtuvo: Double. |
| Tipos incorrectos en<br>una suma (Str y<br>Bool) | incorrectSumWithStr.<br>s<br><br>incorrectTypeInSum.<br>s | ERROR:<br>SEMÁNTICO -<br>SENTENCIAS<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 9<br>(COLUMNA 15)   Se<br>esperaba Int o<br>Double. Se encontró:<br>Str y Int                        |
| Operador módulo                                  | modOperator.s                                             | ERROR:                                                                                                                                                                                                                 |

|                                          |                      |                                                                                                                                                                                               |
|------------------------------------------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                          | modOperator2.s       | SEMÁNTICO -<br>SENTENCIAS<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LINEA 7<br>(COLUMNA 8)   Se<br>esperaba Int o<br>Double. Se encontró:<br>Bool               |
| Operador relacional                      | relationalOperator.s | ERROR:<br>SEMÁNTICO -<br>SENTENCIAS<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LINEA 7<br>(COLUMNA 8)   Se<br>esperaba un Int o<br>Double. Se encontró<br>Array  |
| Se intenta sumar un<br>array con un Int. | sumArrayAndInt.s     | ERROR:<br>SEMÁNTICO -<br>SENTENCIAS<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LINEA 12<br>(COLUMNA 8)   Se<br>esperaba un Int o<br>Double. Se encontró<br>Array |

|                                                              |                                          |                                                                                                                                                                                                 |
|--------------------------------------------------------------|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                              |                                          |                                                                                                                                                                                                 |
| Operador sub (-)<br>sobre tipos<br>incorrectos               | arraySub.s<br><br>stringSum.s            | ERROR:<br>SEMÁNTICO -<br>SENTENCIAS<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 5<br>(COLUMNA 9)   Se<br>esperaba un Int o<br>Double. Se encontró<br>Array    |
| Operador increment<br>y decrement sobre<br>tipos incorrectos | boolIncrement.s<br><br>stringDecrement.s | ERROR:<br>SEMÁNTICO -<br>SENTENCIAS<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 3<br>(COLUMNA 10)   Se<br>esperaba tipo Int o<br>Double. Se encontró:<br>Bool |
| Operador not sobre<br>tipos incorrectos                      | notOperator.s<br><br>arrayNotOperator.s  | ERROR:<br>SEMÁNTICO -<br>SENTENCIAS<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:                                                                                            |



|                                                  |                                               |                                                                                                                                                                                                   |
|--------------------------------------------------|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                  |                                               | LÍNEA 4<br>(COLUMNA 9)   Se<br>esperaba tipo Bool.<br>Se encontró: Int                                                                                                                            |
| Operador de casteo<br>sobre tipos<br>incorrectos | arrayCast.s<br><br>boolCast.s                 | ERROR:<br>SEMÁNTICO -<br>SENTENCIAS<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 10<br>(COLUMNA 14)   Se<br>esperaba un Int o<br>Double. Se encontró<br>Array    |
| Índice de array no<br>entero (Bool/Double)       | arrayIndexNotInt.s<br><br>arrayIndexNotInt2.s | ERROR:<br>SEMÁNTICO -<br>SENTENCIAS<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 12<br>(COLUMNA 12)   El<br>índice debe ser de<br>tipo Int. Se encontró:<br>Bool |
| Asignación de void a<br>una variable             | assignmentWithVoid.<br>s                      | ERROR:<br>SEMÁNTICO -<br>SENTENCIAS<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)                                                                                                                |

|                                                                            |                                                                                      |                                                                                                                                                                                                                             |
|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                            |                                                                                      | DESCRIPCIÓN:  <br>  LÍNEA 20<br>(COLUMNA 8)   El<br>tipo asignado es<br>incorrecto. Se<br>esperaba: Int. Se<br>obtuvo: void.                                                                                                |
| Llamada a<br>constructor de clase<br>que no existe                         | constructorClassInN<br>ewDoesNotExist.s                                              | ERROR:<br>SEMÁNTICO -<br>SENTENCIAS<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 18<br>(COLUMNA 12)   La<br>clase B referenciada<br>no se encuentra<br>declarada                           |
| Llamada a<br>constructor con<br>diferente cantidad o<br>tipo de parámetros | constructorParamete<br>rsWrongQuantity.s<br><br>constructorParamete<br>rsWrongType.s | ERROR:<br>SEMÁNTICO -<br>SENTENCIAS<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 18<br>(COLUMNA 12)   El<br>número de<br>parámetros del<br>constructor no<br>coincide con los<br>brindados |
| Se intenta acceder a                                                       | incorrectChainedArra                                                                 | ERROR:                                                                                                                                                                                                                      |

|                                                                                     |                                                                                                                                                           |                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| una variable luego del array                                                        | yAccess.s                                                                                                                                                 | SEMÁNTICO - SENTENCIAS   NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:     LÍNEA 20 (COLUMNA 10)   Un array solo puede tener un encadenamiento si es de tipo Str              |
| Se intenta acceder a atributos privados desde start y desde métodos de otras clases | incorrectPrivateAccess.s<br><br>incorrectPrivateArrayAccess.s<br><br>accessingPrivateAttribute.s<br><br>accessingPrivateAttributeWithChainedConstructor.s | ERROR: SEMÁNTICO - SENTENCIAS   NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:     LÍNEA 12 (COLUMNA 14)   No se puede acceder a un atributo privado (b) desde otra clase (B). |
| Asignación de diferente tipo                                                        | incorrectTypeInAssignment.s<br><br>stringSum.s                                                                                                            | ERROR: SEMÁNTICO - SENTENCIAS   NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:     LÍNEA 10 (COLUMNA 12)   El tipo asignado es                                                 |

|                                                           |                                                                 |                                                                                                                                                                                                   |
|-----------------------------------------------------------|-----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                           |                                                                 | incorrecto. Se esperaba: Bool. Se obtuvo: Int.                                                                                                                                                    |
| Acceso a atributo de instancia en contexto estático       | instanceAttributeInStaticContext.s                              | ERROR:<br>SEMÁNTICO - SENTENCIAS<br>  NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  <br>  LÍNEA 9 (COLUMNA 8)   El atributo de instancia x es accedido desde un contexto estático.<br>      |
| Llamada a método con parámetros de distinto tipo          | methodWithWrongTypes.s<br><br>methodWithWrongTypesInsideClass.s | ERROR:<br>SEMÁNTICO - SENTENCIAS<br>  NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  <br>  LINEA 18 (COLUMNA 9)   El parámetro 5.5 es de tipo incorrecto. Se obtuvo Double y se esperaba Int |
| Variable no declarada en el ámbito en el que es utilizada | nonDeclaredVariable.s                                           | ERROR:<br>SEMÁNTICO - SENTENCIAS<br>  NÚMERO DE LÍNEA (NÚMERO                                                                                                                                     |

|                                                                                    |                                                                 |                                                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                    |                                                                 | DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 10<br>(COLUMNA 8)   La<br>variable y no ha sido<br>correctamente<br>declarada en este<br>contexto.                                                                                        |
| Método estático<br>llama a un método<br>no estático                                | nonStaticMethodFromStaticMethod.s                               | ERROR:<br>SEMÁNTICO -<br>SENTENCIAS<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 8<br>(COLUMNA 9)   Se<br>realiza una llamada<br>a un método de<br>instancia desde un<br>contexto estático          |
| No hay una<br>expresión booleana<br>dentro de la<br>expresión del if y el<br>while | notBooleanExpressionInIf.s<br><br>notBooleanExpressionInWhile.s | ERROR:<br>SEMÁNTICO -<br>SENTENCIAS<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 12<br>(COLUMNA 14)   La<br>condición del if debe<br>devolver un<br>booleano (true o<br>false). Se encontró<br>Int. |

|                                                                                                                                                                                                  |                                                                                                                                                                                             |                                                                                                                                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Polimorfismo mal hecho. Se intenta acceder a métodos/constructores de B habiendo sido declarado de clase A. Se testea también que se devuelva un subtipo de la clase en el retorno de un método. | <p>incorrectPolymorphism.s</p> <p>polymorphismMethod.s</p> <p>polymorphismParameters1.s</p> <p>polymorphismParameters2.s</p> <p>polymorphismParameters3.s</p> <p>polymorphismInReturn.s</p> | <p>ERROR:</p> <p>SEMÁNTICO - SENTENCIAS   NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  </p> <p>  LINEA 32 (COLUMNA 12)   El tipo asignado es incorrecto. Se esperaba: A. Se obtuvo: C.  </p> |
| Ret en constructor, en start, en un método con retorno void, ret no presente en un método que necesita retorno, o ret en método con diferente tipo de retorno                                    | <p>retInConstructor.s</p> <p>retInStartMethod.s</p> <p>retInVoidMethod.s</p> <p>retNotPresentInMethod.s</p> <p>retWithDifferentTypes.s</p>                                                  | <p>ERROR:</p> <p>SEMÁNTICO - SENTENCIAS   NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  </p> <p>  LÍNEA 8 (COLUMNA 8)   El constructor de A no puede tener ret  </p>                          |
| Se reasigna self                                                                                                                                                                                 | selfAssigned.s                                                                                                                                                                              | <p>ERROR:</p> <p>SEMÁNTICO - SENTENCIAS   NÚMERO DE LÍNEA (NÚMERO DE COLUMNA)   DESCRIPCIÓN:  </p>                                                                                                  |

|                                                      |                                                                                                                           |                                                                                                                                                                                                                      |
|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                      |                                                                                                                           | LÍNEA 11<br>(COLUMNA 8)   Self<br>no puede ser<br>asignado                                                                                                                                                           |
| Método estático con<br>self                          | selfInStaticMethod.s                                                                                                      | ERROR:<br>SEMÁNTICO -<br>SENTENCIAS<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 10<br>(COLUMNA 8)   self<br>está siendo<br>referenciado en un<br>contexto estático                 |
| Start accede a<br>atributos de instancia<br>privados | startAccessingInstanceAttributes.s<br><br>startAccessingInstanceMethods.s<br><br>startAccessingInstancePrivateAttribute.s | ERROR:<br>SEMÁNTICO -<br>SENTENCIAS<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 27<br>(COLUMNA 8)   La<br>variable y no ha sido<br>correctamente<br>declarada en este<br>contexto. |
| Void en<br>encadenamiento                            | voidInChaining.s<br><br>voidInChaining2.s<br><br>voidInChaining3.s                                                        | ERROR:<br>SEMÁNTICO -<br>SENTENCIAS<br>  NÚMERO DE<br>LÍNEA (NÚMERO                                                                                                                                                  |

|                                                                                                                                                                                                                                                        |                                                                                                                            |                                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                                                        |                                                                                                                            | DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LINEA 12<br>(COLUMNA 21)   El<br>tipo void no está<br>permitido en un<br>encadenamiento                                                                                     |
| Encadenamientos<br>con objetos dentro<br>de arrays<br>incorrectos. El<br>método que se<br>encadenó no existe<br>dentro de Str, se<br>intenta acceder a<br>variables o se intenta<br>encadenar con un<br>objeto no<br>encadenable (Int en<br>este caso) | intArrayWithChained<br>Node.s<br><br>strArrayWithChained<br>NodeIncorrect.s<br><br>strArrayWithChained<br>NodeIncorrect2.s | ERROR:<br>SEMÁNTICO -<br>SENTENCIAS<br>  NÚMERO DE<br>LÍNEA (NÚMERO<br>DE COLUMNA)  <br>DESCRIPCIÓN:  <br>  LÍNEA 20<br>(COLUMNA 18)   Un<br>array solo puede<br>tener un<br>encadenamiento si<br>es de tipo Str |

## 2.7. Generación de código

El código se genera utilizando el AST, ya que cada nodo del mismo produce su propio código. Por ejemplo, un nodo asignación producirá todo el código necesario para realizar la misma, llamando a los métodos `codeGen()` de los nodos hijos, finalmente asignando los valores o direcciones correspondientes.

### 2.7.1. Estructura del registro de activación

#### 2.7.1.1. Estructura de registro de activación

El registro de activación elaborado tiene la siguiente estructura:





Figura 10. Estructura del registro de activación.

Por ejemplo, el registro de activación del constructor del array, al momento de la llamada, quedaría de la siguiente manera:

20(\$fp) → \$fp anterior  
 16(\$fp) → self  
 12(\$fp) → tamaño del tipo (4 o 8 si es Double)  
 8(\$fp) → longitud del array  
 4(\$fp) → inicialización  
 0(\$fp) → \$ra

#### 2.7.1.2. Datos almacenados en registro de activación

El registro de activación contiene el framepointer del llamador, el self, los parámetros del método, el return address y las variables locales. En el caso de que no exista el self, la posición se mantiene igualmente, pero vacía para mantener una coherencia global.

Otra consideración especial es que los parámetros se guardan en el orden inverso en el que son declarados. Esto se hace así ya que el framepointer del nuevo método apuntará como posición al RA y seguido de él se colocaran las variables locales en orden. De esta manera, se logra un estándar a la hora de buscar los offsets de los mismos, ya que el primer parámetro tendrá offset +4 (o +8 si es Double) y, a su vez, la primera variable tendrá offset -4.

## 2.7.2. Gestión y resolución de métodos

### 2.7.2.1. Acceso a parámetros

Como se explicó anteriormente, el acceso a parámetros se hace desde el framepointer utilizando el offset. Es decir, el primer parámetro está en el offset +4 del fp (o +8 si es Double), el segundo en la anterior +4 (o +8), y de esta manera sucesivamente.

### 2.7.2.2. Acceso a variables

De manera similar, el acceso a las variables también se realiza desde el framepointer utilizando el offset, pero en este caso la primera variable estará en el offset -4 (o -8 si es Double), y las siguientes en los offsets obtenidos al restar 4 o 8 según corresponda.

### 2.7.2.3. Retorno

Los retornos de los métodos se guardan en el registro \$f0 si son de tipo Double o en el registro \$a0 en otro caso (direcciones o valores).

### 2.7.2.4. Datos del stack ante la llamada a un método

Ante la llamada a un método se incorpora en el stack:

- El framepointer del llamador (para su posterior restauración)
- La dirección al CIR del objeto que representaría el self
- Los parámetros del mismo colocados en orden inverso

### 2.7.2.5. Datos almacenados al inicio de un método

Dentro del método se almacena la siguiente información:

- El return address
- Las variables locales declaradas

## 2.7.3. Estructuras de las clases

### 2.7.3.1. Class Instant Record

El Class Instant Record elaborado tiene la estructura usual. Es decir, al comienzo tiene la dirección a la Vtable de la clase y luego tiene los atributos del objeto.

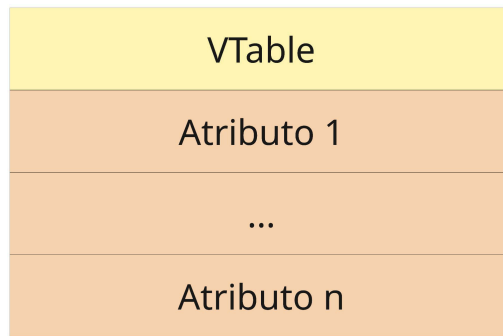


Figura 11. Estructura del Class Instant Record.

#### 2.7.3.2. Virtual table

La Virtual Table guarda las direcciones de todos los métodos en orden de declaración para su posterior acceso.

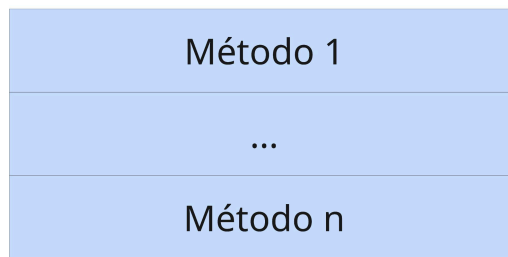


Figura 12. Estructura de la Virtual Table.

#### 2.7.3.3. Creación de objetos de una clase

Para crear un objeto de una clase se reserva memoria en el heap teniendo en cuenta la cantidad de atributos. Por cada double se reservan 8 bytes y en otro caso se reservan 4 bytes. Además, se reservan 4 bytes adicionales para la vtable. Luego de reservada la memoria, se carga la dirección de la vtable en el offset 0 y la dirección de la memoria solicitada queda en el registro \$a0.

#### 2.7.3.4. Acceso a atributos de objetos instanciados

Primero se consigue la dirección del objeto principal la cual estaría almacenada en el registro de activación (ya sea como variable, parámetro o como self) y se carga en el registro \$a0. Luego, en esa dirección se utiliza el offset del atributo para acceder al mismo, el cual queda finalmente almacenado en el registro \$a0.

#### 2.7.3.5. Acceso a los métodos de los objetos instanciados

De manera similar, se accede a la dirección del objeto principal y se busca la dirección de la vtable del mismo, la cual se encuentra en el offset 0 y se carga en el registro \$a0. Luego, utilizando el offset del método se accede a la dirección del mismo, se almacena en \$a0 y, habiendo cargado los parámetros previamente, se hace un jump al registro contenido en \$a0.

#### 2.7.4. Decisiones de diseño

Se tomaron decisiones de diseño adicionales con respecto a diferentes temas:

- Para los objetos de tipo Str y Array se crearon sus correspondientes CIR en la generación de código, mientras que el resto se trató como si fueran valores primitivos. Esto debido a que estas dos clases fueron las únicas que poseían métodos.
- En el CIR del array se guarda además de la Vtable y los elementos del array, el tamaño del mismo, esto con el objetivo de facilitar los chequeos de longitud.
- De manera similar, la clase Str también posee métodos (concat y length) por lo que se almacena la dirección a la vtable.
- Los booleanos en generación de código se representan como los números enteros 0 y 1.
- El nil en generación de código se representa como un 0.

#### 2.7.5. Proceso de generación de código MIPS

Se tomó la decisión de crear un archivo setupCode.asm con el código base del programa. Está compuesto por el código MIPS correspondiente a las clases predefinidas de TinyS que contienen métodos, es decir, Array, Str e IO. Para los mismos, se codificaron los constructores, las vtable y los métodos correspondientes a estas clases, como por ejemplo length de la clase Array. Adicionalmente, se incluyeron los procedimientos para las operaciones relacionales con Double.

De manera similar, se generó un archivo runtimeExceptions.asm, el cual contiene las excepciones que pueden surgir en tiempo de ejecución, como por ejemplo, índice de array fuera

de rango. Ambos archivos se concatenan al final de cada código generado, conformando así el .asm definitivo.

Con respecto al código restante, el mismo es generado mediante un recorrido del Abstract Syntax Tree visitando los nodos correspondientes, los cuales poseen un método `codeGen()` que genera su propio código y lo concatena al archivo final. Al finalizar, el recorrido se obtiene dicho archivo que contiene todo el código generado.

### 2.7.6. Errores detectados

Los errores en esta etapa son aquellos encontrados en tiempo de ejecución, tales como:

- División por cero
- Módulo de cero
- Tamaño del array negativo
- Índice del array fuera de rango
- Índice negativo
- Entrada incorrecta de un boolean (Int distinto de 0 o 1)
- Variable no inicializada

### 2.7.7. Casos de testeo

Tabla 6. Casos de testeo de generación de código.

| Escenario                                                                          | Nombre Test                | Salida esperada                 |
|------------------------------------------------------------------------------------|----------------------------|---------------------------------|
| Se inicializa un array, se le asignan valores, se imprime y se calcula la longitud | arrayTest.s                | [0, 0, 0, 0, 0][0, 1, 0, 2, 3]5 |
| Imprime el valor del array en la posición 0                                        | arrayAccessWithCla<br>ss.s | 0                               |

|                                                   |                                                      |                                                             |
|---------------------------------------------------|------------------------------------------------------|-------------------------------------------------------------|
| Se realiza una búsqueda lineal en un array        | arrayLinealSearch.s                                  | 3                                                           |
| Se calcula el promedio de los valores de un array | averageTest.s                                        | Promedio = 30                                               |
| BubbleSort                                        | BubbleSortWithArray.s                                | [1,2,3,4,5]                                                 |
| Operaciones matemáticas complejas                 | complexMathExpression.s                              | 5<br>7.0                                                    |
| Operaciones con array de tipo Double              | doubleArray2.s<br>doubleArray.s                      | [0.0, 0.0, 0.0, 0.0, 0.0][4.7, 1.5, 0.0, 2.6, 3.74]5        |
| Herencia                                          | doubleInheritanceWithSelf.s<br><br>inheritanceTest.s | 9.4<br>9.4<br>3.8                                           |
| Fibonacci                                         | fibonacciTest.s                                      | Para la entrada 3:<br>f_0=0<br>f_1=1<br>f_2=1<br>f_3=2<br>2 |
| Se realiza una división de Doubles                | firstTest.s                                          | RUNTIME<br>EXCEPTION:<br>división por cero.                 |
| Se realiza un bucle While y se imprime la salida  | forTest.s                                            | 6788                                                        |

|                                                |                     |                                                      |
|------------------------------------------------|---------------------|------------------------------------------------------|
| Se realiza un if-else y se imprimen resultados | ifTest.s            | 6                                                    |
| Bloque de código con class e impl              | ifWithClassTest.s   | 7                                                    |
| Clase con atributo de otra clase               | ifWithNestedClass.S | 6                                                    |
| Se verifica que se ingrese un bool (0 o 1)     | IOBoolIn.s          | BOOOL0<br>o BOOOL1<br>dependiendo el caso            |
| Se ingresa por teclado y se imprime un Double  | inAndOutDouble.s    | 3 + entrada<br>Para entrada 5:<br>8.0                |
| Se ingresa por teclado y se imprime un Str     | inAndOutStr.s       | Para entrada "chau":<br>0<br>chau<br>chauhola        |
| Se prueba el if encadenado                     | nestedIf.s          | par 4<br>impar 3<br>par 2<br>impar 1<br>resultado=10 |
| Se prueba el while encadenado                  | nestedWhile.s       | 1<br>2<br>2<br>4<br>3<br>6                           |
| Se asigna un valor a un atributo de clase      | newClass.s          | 5                                                    |

|                                                                                                 |                            |                                              |
|-------------------------------------------------------------------------------------------------|----------------------------|----------------------------------------------|
| Se asigna nil a una variable                                                                    | nilTest.s                  | soy nil<br>Soy igual a a<br>No soy igual a a |
| Fibonacci polimórfico                                                                           | polymorphicFibonacci.s     | 55<br>55                                     |
| Se prueban los métodos polimórficos                                                             | polymorphismMethodsTest.s  | 12.56<br>9.0                                 |
| Se prueba el polimorfismo                                                                       | polymorphismTest.s         | 3.8                                          |
| Factorial recursivo                                                                             | recursiveFactorial.s       | 6<br>5<br>4<br>3<br>2<br>1<br>720            |
| Se prueban los métodos length y concat de Str                                                   | strMethodsTest.s           | Holachau8                                    |
| Se accede a un elemento de un array que está dentro de una clase                                | arrayAccessWithClasses.s   | 0                                            |
| Se testea que se pueda llamar al método concat desde un objeto Str que es contenido en un array | strArrayWithChainedNode3.s | holaschau                                    |
| Se llama al método length de Str con el                                                         | strArrayWithChainedNode2.s | 4                                            |



|                                                                                                     |                            |                                                                                                        |
|-----------------------------------------------------------------------------------------------------|----------------------------|--------------------------------------------------------------------------------------------------------|
| objeto de un array usando encadenamiento                                                            |                            |                                                                                                        |
| Se llama al método length de Str con un array, el cual se accede con 2 encadenamientos              | strArrayWithChained Node.s | 4                                                                                                      |
| División por cero                                                                                   | divZeroDouble.s            | RUNTIME EXCEPTION: división por cero.                                                                  |
| Entrada por teclado de Bool distinto de 0 o 1                                                       | incorrectIOBoolIn.s        | Si se ingresa número diferente de 0 o 1: RUNTIME EXCEPTION: se esperaba 0 o 1 como entrada de un Bool. |
| Módulo de cero                                                                                      | modZero.s                  | RUNTIME EXCEPTION: división por cero en operación módulo.                                              |
| Tamaño de un array negativo                                                                         | negativeArraySize.s        | RUNTIME EXCEPTION: la longitud del array no puede ser negativa.                                        |
| Caso en que faltan returns en un método o están mal ubicados. No da error pero explota en ejecución | retTest.s                  | 2147479516 (incorrecto)                                                                                |
| Se intenta acceder a                                                                                | testArrayExceptions.       | RUNTIME                                                                                                |

|                                                                |                              |                                                                       |
|----------------------------------------------------------------|------------------------------|-----------------------------------------------------------------------|
| un índice fuera de rango                                       | s                            | EXCEPTION: índice del array fuera de rango.                           |
| Se intenta acceder a una variable de una clase no inicializada | variableClassNotInitalized.s | RUNTIME EXCEPTION: se intenta acceder a una variable no inicializada. |

### 3. Gestión de los tiempos

Antes de iniciar con la implementación de cada etapa, se estudiaba y entendía en profundidad la parte teórica de la misma. Luego, antes de pasar a la codificación correspondiente, se realizaba un diseño previo de la solución de la etapa, a través de esquemas y en forma de pseudocódigo. Esto ocurrió en el análisis sintáctico con la transformación de la gramática, en el análisis semántico con el EDT, la tabla de símbolos y el AST y, también, en la generación de código a través de un ejemplo de código generado para organizar la estructura del mismo. Por lo tanto, al momento de comenzar con la implementación, se poseía una base sólida de conocimientos.

La implementación comenzó durante el cursado de la materia, llegando a finalizar el analizador sintáctico. Luego, se retomó el mismo mejorando lo realizado anteriormente y comenzando con el analizador semántico y posteriormente, el generador de código.

Finalmente, una vez que se terminó con el proyecto, se procedió a realizar el informe final, plasmando en el mismo todo el proceso que se llevó a cabo.

### 4. Conclusión

En conclusión, se logró alcanzar una implementación funcional del compilador del lenguaje TinyS, cumpliendo con los requisitos establecidos. El resultado fue

validado mediante múltiples casos de prueba, en los que se verificó tanto programas correctos como incorrectos brindando los mensajes de error correspondientes.

Como posible mejora, se podría realizar una refactorización del código implementado, con el objetivo de eliminar la duplicación de código.

Este proyecto nos permitió afianzar conocimientos y entender en profundidad el funcionamiento interno de un compilador.

## 5. Referencias y/o bibliografía

**Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008).**

*Compiladores: principios, técnicas y herramientas* (2.<sup>a</sup> ed.). Pearson Addison-Wesley.

**Grune, D., van Reeuwijk, K., Bal, H. E., Jacobs, C. J. H., & Langendoen, K. (2012).**

*Modern compiler design* (2nd ed.). Springer.

**Cooper, K. D., & Torczon, L. (2012).**

*Engineering a compiler* (2nd ed.). Morgan Kaufmann.

## 6. Anexos

**Anexo 1:** Diagrama de clases de la tabla de símbolos (archivo Anexo/symbolTable.png).

**Anexo 2:** Ejemplo de JSON generado en la etapa 3 (archivo Anexo/JsonSymbolTable.png).

**Anexo 3:** Diagrama de clases del árbol sintáctico abstracto (archivo Anexo/ASTClassDiagram.png).

**Anexo 4:** Ejemplo de JSON generado en la etapa 4 (archivo Anexo/JsonAST.json).